Università di Padova

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica



ChatCart: agente conversazionale con interfaccia generativa

Tesi di Laurea

Laureando

Edoardo De Piccoli

Matricola 2101055

Relatore

Prof. Tullio Vardanega

Anno Accademico 2025-2026

© Edoardo De Piccoli. Tutti i diritti riservati. Tesi di Laurea: “ChatCart: agente conversazionale con interfaccia generativa”, Università di Padova, Dipartimento di Matematica “Tullio Levi-Civita”, Corso di Laurea in Informatica.

Sommario

In questa relazione descrivo lo *stage* che ho svolto presso Halue S.r.l., società di consulenza tecnologica con sede operativa ad Abano Terme, dal 14 settembre al 9 novembre 2026.

Con la supervisione del Prof. Tullio Vardanega, relatore, e di Antonio Cavestro, *tutor* aziendale, ho progettato e realizzato ChatCart, il prototipo di un assistente conversazionale per i negozi Shopify che risponde al cliente non soltanto con testo, ma anche con componenti dell'interfaccia che sceglie e popola in tempo reale con i dati del negozio.

Struttura del documento. La relazione si articola in quattro capitoli:

- **Capitolo 1: «Halue e il commercio digitale».** Presento l'azienda, la sua clientela, le tecnologie e i processi che adotta e il suo rapporto con l'innovazione, in particolare con l'intelligenza artificiale.
- **Capitolo 2: «Oltre la conversazione testuale».** Discuto il problema che lo *stage* affronta, il suo posto nella strategia di Halue, gli obiettivi e i vincoli del progetto e le ragioni per cui l'ho scelto.
- **Capitolo 3: «Progetto e sviluppo di ChatCart».** Illustro il metodo di lavoro, l'analisi, l'architettura, la realizzazione dell'agente e del *widget*, le funzionalità sperimentali e le possibili evoluzioni del progetto, la verifica e i risultati ottenuti.
- **Capitolo 4: «Valutazione retrospettiva».** Valuto il grado di soddisfacimento degli obiettivi, la mia maturazione professionale e la distanza tra le competenze che lo *stage* mi ha richiesto e quelle che ho acquisito nel corso di studi.

In appendice raccolgo gli acronimi e le abbreviazioni con la loro forma estesa, il glossario e i riferimenti.

Convenzioni tipografiche. Nel documento adotto le seguenti convenzioni:

- il corsivo indica i termini in lingua straniera, e soltanto questi; i nomi propri di aziende, prodotti e metodi (come Shopify, Slack o Scrum) restano in tondo;
- alla prima occorrenza di un termine presente nel glossario ne do nel testo una definizione essenziale e lo contrassegno con il pedice G; il termine e il pedice sono cliccabili e rimandano all'approfondimento nel glossario;
- figure, tabelle e codici sorgente hanno una numerazione progressiva e una didascalia posta sotto l'elemento, che ne indica la fonte quando non sono di mia realizzazione.

Indice

1 Halue e il commercio digitale **1**

1.1 Profilo aziendale e linee di attività 1

1.2 Clientela e modalità di ingaggio 4

1.3 Ecosistema tecnologico 6

1.4 Processi di sviluppo e gestione dei progetti 7

1.4.1 Organizzazione del lavoro e gestione dei progetti 7

1.4.2 Dal *ticket* al rilascio 8

1.5 Innovazione e intelligenza artificiale 10

2 Oltre la conversazione testuale **12**

2.1 Il ruolo degli *stage* nella strategia aziendale 12

2.2 I limiti degli assistenti conversazionali testuali 15

2.3 Il commercio agentico nella visione di Halue 18

2.4 Obiettivi e vincoli del progetto 20

2.5 Motivazioni e obiettivi personali 23

3 Progetto e sviluppo di ChatCart **26**

3.1 Metodo di lavoro 26

3.1.1 Pianificazione e interazione con il *tutor* aziendale 26

3.1.2 Organizzazione del lavoro 28

3.1.3 Strumenti e assistenti di programmazione 29

3.2 Studio preliminare e analisi 31

3.2.1 L'ecosistema delle applicazioni Shopify 31

3.2.2 Un'analisi dei requisiti iterativa 34

3.3 Architettura del sistema 38

3.3.1 Visione d'insieme 38

3.3.2 Il contratto tra agente e *widget* 41

3.4 Realizzazione 44
3.4.1 L'agente conversazionale 44
3.4.2 Il *widget* e l'integrazione con il tema del negozio 47
3.4.3 Metriche di utilizzo e analisi delle conversazioni 49
3.5 Sperimentazioni e sviluppi futuri 52
3.6 Verifica e validazione 54
3.7 Risultati 56
3.7.1 Il prodotto dal punto di vista dell'utente 56
3.7.2 Risultati quantitativi 61

4 Valutazione retrospettiva **63**
4.1 Soddisfacimento degli obiettivi 63
4.2 Maturazione professionale 66
4.3 Formazione universitaria e lavoro 69

Acronimi e abbreviazioni **72**

Glossario **74**

Riferimenti **85**

Elenco delle figure

1.1 Le linee di attività di Halue 2
1.2 Le integrazioni di un negozio Shopify 4
1.3 Esempio di architettura di un progetto di intelligenza artificiale. . . . 7
1.4 Il ciclo di vita di un *ticket* in Halue, come diagramma di attività UML. 9
1.5 La quota del tempo che Halue dedica alla ricerca e sviluppo. 10
1.6 Il ciclo di lavoro con gli strumenti di intelligenza artificiale in Halue. . 10

2.1 Un flusso di conversazione predefinito e il ciclo di un agente 13
2.2 Confronto tra un assistente testuale e ChatCart 15
2.3 I tre approcci all'interfaccia generativa 18
2.4 Le due direzioni del commercio agentico 19
2.5 L'architettura su due livelli prevista dal piano di lavoro 20
2.6 La ripartizione delle ore prevista dal piano di lavoro 23

3.1 Ore pianificate e stimate per attività 28
3.2 Installazione e accesso al negozio 32
3.3 I casi d'uso del cliente 35
3.4 I casi d'uso del commerciante 36
3.5 Architettura di ChatCart 39
3.6 Tre *pattern* del *server* 41
3.7 Il contratto condiviso degli strumenti 42
3.8 Il percorso di una risposta 44
3.9 La sincronizzazione del carrello 48
3.10 Dagli eventi alle misure e all'analisi 50
3.11 Il widget nel negozio di sviluppo 57
3.12 Dalla variante al carrello 58
3.13 La dashboard del commerciante 59
3.14 Il dettaglio di una conversazione 60

4.1 La tassonomia di Bloom rivista 69

Elenco delle tabelle

1.1 Gli strumenti di coordinamento del *team* di Halue. 8
1.2 Le pratiche di Halue lungo il ciclo di un *ticket*. 9

2.1 Gli obiettivi obbligatori del piano di lavoro. 21
2.2 Gli obiettivi desiderabili e facoltativi del piano di lavoro. 21
2.3 I miei obiettivi personali per lo *stage*. 25

3.1 Le modalità di coordinamento durante lo *stage*. 27
3.2 Gli strumenti principali impiegati durante lo *stage*. 30
3.3 I punti di accesso ai dati e al carrello di Shopify. 33
3.4 I venti requisiti principali selezionati dalla ricognizione finale. Gli identificativi mantengono la numerazione originale. 38
3.5 I *pattern* adottati e il problema concreto che risolvono. 40
3.6 La divisione delle regole tra istruzioni al modello e codice. 45
3.7 Tre usi della stessa conversazione conservata nella base di dati. 46
3.8 Le strategie con cui ChatCart aggiorna il tema dopo una scrittura della chat. 48
3.9 Le tre dimensioni dell'analisi automatica di una conversazione. 51
3.10 Le sperimentazioni escluse dalla versione finale. Il confronto sul valore resta qualitativo, basato sulle prove del prototipo. 53
3.11 La strategia delle prove automatiche sulle cuciture del sistema. 55
3.12 Copertura dei venti requisiti principali nella versione finale. 61
3.13 Copertura delle righe per area, misurata con Vitest sulla versione finale. L'avvio e il pannello del *widget* comprendono pulsante di apertura, intestazione, campo di testo e pannello del carrello. 62

4.1 Lo stato degli obiettivi del piano di lavoro e dei miei obiettivi personali nella versione finale. 64

4.2 I prodotti attesi del piano di lavoro e ciò che ho consegnato. 66

Elenco dei codici sorgente

3.1 Un esempio del contratto tipizzato 43

1 Halue e il commercio digitale

Presento Halue S.r.l., l'azienda presso cui ho svolto lo *stage*, descrivendone le attività, i clienti, le tecnologie, i processi e il rapporto con l'innovazione. Il quadro nasce dall'esperienza diretta delle otto settimane di lavoro, dai colloqui con i colleghi, da un'intervista dedicata al *tutor* aziendale, Antonio Cavestro, e dalle informazioni pubbliche del sito dell'azienda[1].

1.1 Profilo aziendale e linee di attività

Halue è una società di consulenza tecnologica nata nel 2022, con sede legale a Parma e sede operativa ad Abano Terme, in provincia di Padova. È nata come *boutique* di consulenza, cioè una società piccola e specializzata in pochi ambiti, sull'*e-commerce*$_{\text{G}}$ e sui sistemi di *Customer Relationship Management* (CRM)$_{\text{G}}$, con Shopify e Salesforce come piattaforme di riferimento. Il primo ambito riguarda la vendita di prodotti e servizi tramite canali digitali. Il secondo riguarda la gestione delle relazioni delle aziende con i propri clienti, dalle vendite all'assistenza al *marketing*.

Negli anni successivi Halue ha esteso la propria attività all'intelligenza artificiale, a partire dai progetti stessi. I negozi dei suoi clienti hanno cataloghi ampi, contenuti in più lingue e percorsi d'acquisto complessi. In questi punti un modello linguistico collegato ai dati del negozio aiuta sia chi acquista sia chi gestisce il negozio. Nello stesso periodo le piattaforme di commercio elettronico hanno iniziato a offrire strumenti e protocolli pensati per gli agenti di intelligenza artificiale, che presento nella sezione 1.3. La figura 1.1 riassume le linee di attività di oggi, che descrivo più avanti in questa sezione.

Dal 2024 Halue è una Società Benefit$_{\text{G}}$, una forma giuridica introdotta in Italia nel 2015 per le società che oltre al profitto perseguono obiettivi di beneficio comune dichiarati nello statuto e ne rendono conto ogni anno in una relazione d'impatto. Gli



Figura 1.1: Le linee di attività di Halue. La consulenza tecnologica fa da cornice a tutte; l'area di ciascun blocco segue l'ordine di peso delle linee, senza riprodurne una misura.

obiettivi di Halue riguardano l'ambiente, le persone e le comunità. L'azienda misura le proprie pratiche con il *B Impact Assessment*$_{G}$, un questionario di valutazione che le esamina in cinque aree: *governance*, lavoratori, clienti, comunità e ambiente. Tra i risultati del 2025, Halue riporta[2]:

- un codice etico e politiche interne sulla segnalazione di illeciti, sugli abusi e sulle discriminazioni;
- linee guida per rendere sostenibile il lavoro da remoto;
- una collaborazione per la tutela della fauna marina nel Delta del Po.

Oggi l'azienda conta dieci persone, con i seguenti ruoli:

- *Founder & Senior Consultant*$_{G}$: il fondatore Valentino Baraldo, che svolge anche attività di consulenza;
- *Solution & Technical Architect*$_{G}$: definisce le soluzioni e l'architettura tecnica;
- *Senior Project Manager*$_{G}$ e *Project Management Assistant*$_{G}$: due ruoli dedicati alla gestione dei progetti;
- *Senior Software Engineer*$_{G}$, *Software Engineer*$_{G}$ e *Junior Developer*$_{G}$: cinque sviluppatori con diversi livelli di esperienza;
- *UX/UI Digital Consultant*$_{G}$: cura la progettazione dell'esperienza utente e dell'interfaccia.

In un'azienda di queste dimensioni ogni ruolo copre un'intera fase del progetto: la consulenza iniziale, l'architettura, la gestione, lo sviluppo e la progettazione dell'interfaccia. Ogni persona lavora per più clienti nello stesso periodo.

Halue lavora su commessa: costruisce soluzioni su misura per i singoli clienti e non vende prodotti propri, come applicazioni pubblicate sullo Shopify App Store. Le sue competenze crescono quindi progetto dopo progetto, guidate dalle richieste dei clienti, che l'azienda segue da vicino in un rapporto di lavoro continuo, possibile per un'azienda delle sue dimensioni.

La consulenza tecnologica accompagna tutte le linee di attività dell'azienda, dalla valutazione iniziale alla scelta dei sistemi. Le linee sono, in ordine di peso:

1. l'*e-commerce* su Shopify e su Salesforce Commerce Cloud, sia verso il consumatore finale (B2C$_G$) sia verso altre aziende (B2B$_G$). Nel B2B il negozio gestisce anche listini riservati, ordini minimi e condizioni di pagamento per ciascuna azienda cliente;
2. l'integrazione di sistemi, cioè il collegamento del negozio con la logistica, con i servizi di terze parti e con i sistemi gestionali (ERP$_G$), che riuniscono in un'unica base di dati amministrazione, acquisti, magazzino e vendite e con cui il negozio sincronizza ordini e disponibilità dei prodotti. Il collegamento può essere punto a punto, costruito per ogni coppia di sistemi, oppure passare per una piattaforma di integrazione erogata come servizio (iPaaS$_G$), che offre connettori già pronti e strumenti per trasformare i dati;
3. i servizi di intelligenza artificiale, dalla valutazione di quanto un'azienda sia pronta ad adottarla, anche per la qualità e l'organizzazione dei suoi dati, fino allo sviluppo di agenti su misura;
4. i sistemi CRM su Salesforce, compresi i prodotti per l'assistenza ai clienti e per il *marketing*;
5. la formazione, rivolta sia ai clienti sia ai dipendenti. Per i clienti, Halue prepara incontri, come *workshop* formativi sull'intelligenza artificiale, tenuti direttamente dal fondatore, che spiegano il funzionamento dei modelli linguistici e rendono il cliente consapevole delle loro possibilità e dei loro limiti. I dipendenti seguono una formazione interna sui metodi agili, sulle piattaforme e sulla tecnologia, e percorsi personali per crescere come professionisti, anche oltre il loro ruolo in azienda;
6. la progettazione dell'interfaccia e dell'esperienza utente.

Le prime due linee si incontrano in quasi ogni progetto: il negozio raramente lavora da solo, e scambia di continuo dati con i sistemi che l'azienda cliente usa già.

Le due piattaforme di *e-commerce* hanno pesato in modo diverso nella storia di Halue. Nei primi anni l'azienda si è specializzata anche su Salesforce Commerce



Figura 1.2: Un esempio di integrazioni punto a punto di un negozio Shopify con CRM, gestionale ERP, logistica e servizi di terze parti. Le finestre sono illustrative; le frecce verso l'esterno indicano i dati che il negozio invia, quelle verso il centro i dati che riceve. Con una piattaforma iPaaS gli stessi flussi passano per un unico punto di raccordo invece che per un collegamento dedicato a ogni sistema.

Cloud, la piattaforma di commercio elettronico della *suite* di Salesforce, e ne segue ancora i clienti. Oggi la specializzazione principale è Shopify Plus, che presento nella sezione 1.2.

L'intelligenza artificiale è insieme una linea a sé e, sempre di più, una competenza che entra nei progetti di *e-commerce* e di integrazione.

1.2 Clientela e modalità di ingaggio

Halue lavora soltanto con aziende private, per lo più aziende italiane di medie e grandi dimensioni, e non con la pubblica amministrazione. Sono marchi strutturati della moda, dello sport, del commercio al dettaglio e della cosmetica professionale, con fatturati che vanno da decine a diverse centinaia di milioni di euro all'anno, e vendono *online* su più mercati, con lingue e valute diverse.

Vendere su più mercati moltiplica il lavoro tecnico. Ogni mercato ha la sua lingua, la sua valuta, i suoi prezzi e le sue tasse, spesso anche un magazzino proprio, e ogni negozio deve restare allineato con il sistema gestionale dell'azienda. Per questo molti clienti di Halue usano Shopify Plus$_{G}$, la proposta di Shopify per le aziende di grandi dimensioni, che aggiunge al negozio, tra l'altro, funzioni dedicate alla vendita B2B[3].

Alcuni clienti hanno decine di negozi attivi, uno per mercato o per canale, ciascuno con il proprio catalogo e il proprio magazzino.

Halue è *Shopify Plus Partner* dal 2023. Il programma partner di Shopify assegna alle agenzie un livello in base ai negozi che hanno avviato o trasferito sulla piattaforma e al volume di vendite di quei negozi. Il livello dà all'agenzia visibilità nell'elenco ufficiale dei partner, che i commercianti consultano per scegliere a chi affidare il proprio negozio. Nel suo profilo Halue indica come specializzazioni, tra le altre, lo sviluppo di applicazioni e API su misura, la personalizzazione della pagina di pagamento e dell'area personale dei clienti e l'ottimizzazione per i motori di ricerca[4].

Alcune delle richieste dei clienti che ho potuto notare in prima persona durante le riunioni quotidiane con il *team* sono:

- la migrazione di un negozio verso Shopify da altre piattaforme, come Salesforce Commerce Cloud, WooCommerce e Magento. Oltre a catalogo, clienti e ordini, la migrazione trasferisce le integrazioni con i sistemi esterni e conserva gli indirizzi delle pagine, perché i collegamenti esistenti e il posizionamento nei motori di ricerca non vadano persi;
- il *redesign*$_{\text{G}}$ di un negozio esistente;
- lo sviluppo di interfacce su misura, con *landing page*$_{\text{G}}$ dinamiche (pagine che accolgono chi arriva da una campagna o da un annuncio e lo guidano verso un acquisto, con contenuti che cambiano in base al mercato o al visitatore) e configuratori 3D;
- l'espansione su nuovi mercati e la manutenzione periodica dei negozi in produzione, con aggiornamenti di configurazione, correzioni ed evoluzioni.

I progetti arrivano soprattutto per passaparola, da clienti e contatti che segnalano l'azienda, e in parte da gare e bandi. Le formule contrattuali sono tre, ciascuna adatta a un tipo di lavoro:

- a corpo, con un prezzo fissato in anticipo, per i progetti dal perimetro ben definito. Questa formula richiede una stima accurata prima di iniziare, e nella sezione 1.4.2 descrivo il metodo con cui Halue la ottiene;
- a tempo e materiali, per i lavori il cui perimetro si chiarisce strada facendo. Il cliente paga il tempo impiegato, calcolato sul costo giornaliero di ciascun dipendente moltiplicato per il numero dei dipendenti coinvolti e per le giornate di lavoro;
- i contratti di manutenzione e assistenza per i negozi già in produzione, che coprono le segnalazioni e le evoluzioni dopo la consegna.

Le modifiche chieste a progetto avviato diventano *change request* e seguono lo stesso percorso di stima del progetto iniziale. Il rapporto con un cliente prosegue quindi oltre la consegna, e con alcuni clienti è diventato una collaborazione duratura, comprensiva di più progetti successivi alla prima collaborazione con l'azienda.

Il fondatore e il *Senior Project Manager* sono le figure che seguono i clienti con più continuità. Quando un cliente collabora da tempo con uno sviluppatore, per esempio un *Senior Software Engineer*, le richieste di nuove funzionalità e gli aggiornamenti sull'avanzamento passano direttamente da quella persona, nelle *call* e nel canale Slack dedicato al marchio.

1.3 Ecosistema tecnologico

Le interfacce dei negozi Shopify rivolte ai clienti, dette *storefront*$_G$, si possono sviluppare in due modi:

- con un tema della piattaforma: Shopify genera le pagine a partire da modelli scritti in Liquid$_G$, il suo linguaggio di *templating*, e dai dati del negozio;
- con un'architettura *headless*$_G$: l'interfaccia è un'applicazione separata, costruita per esempio con Hydrogen$_G$, il *framework* di Shopify basato su React, e accede al negozio tramite API$_G$, cioè interfacce con cui un programma mette a disposizione di altri programmi dati e operazioni, secondo un contratto definito.

Shopify offre due famiglie di API: le Admin API, per gestire i dati e le operazioni del negozio, e le Storefront API, per catalogo, carrello e acquisto. Entrambe usano GraphQL$_G$, un linguaggio di interrogazione in cui il *client* chiede con una sola richiesta esattamente i campi che gli servono. Un tema Liquid riceve invece i dati del negozio durante la generazione delle pagine. Per aggiungere funzionalità, Halue sviluppa applicazioni personalizzate in Node.js e TypeScript, integrabili con entrambe le modalità.

Nei progetti di intelligenza artificiale cambia soprattutto l'architettura. Al centro c'è un modello linguistico di grandi dimensioni (LLM)$_G$, addestrato a elaborare e generare testo, che chiede all'agente che lo ospita di invocare uno strumento, ne riceve il risultato e ripete il ciclo finché non formula la risposta (figura 1.3). Gli strumenti si possono scrivere su misura per il progetto oppure ottenere tramite MCP$_G$, un protocollo aperto con cui un *server* espone strumenti e risorse, come il catalogo

di un negozio, a qualunque agente lo supporti, senza un'integrazione dedicata per ciascuno.

In ChatCart ho sviluppato una ricerca nel catalogo su misura al posto del *server* MCP di Shopify: da questa scelta nasce il contratto tra agente e *widget* che descrivo nella sezione 3.3.2.

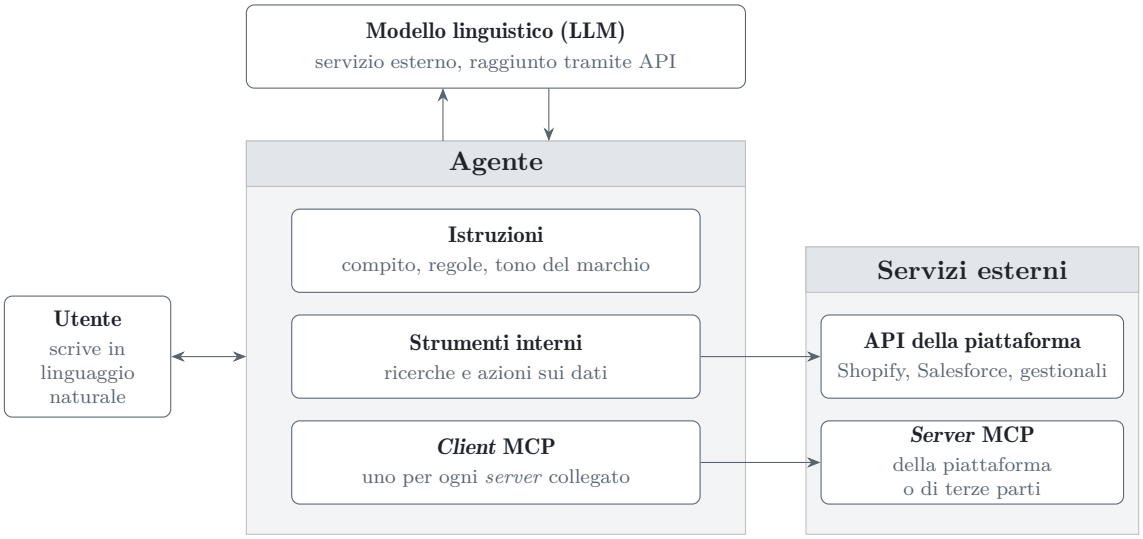


Figura 1.3: Esempio di architettura di un progetto di intelligenza artificiale.

1.4 Processi di sviluppo e gestione dei progetti

1.4.1 Organizzazione del lavoro e gestione dei progetti

Il metodo di riferimento è Scrum$_G$, un *framework* agile per lo sviluppo iterativo, adattato al lavoro di un'agenzia. Il lavoro procede per *sprint*$_G$, cicli brevi al termine dei quali il *team* presenta i risultati al cliente in una *sprint review* e ne discute internamente in una retrospettiva. Mancano le figure canoniche di Scrum, il *Product Owner* e lo *Scrum Master*, e le loro responsabilità passano a due ruoli:

- il *team lead*$_G$, che cura l'analisi funzionale, le stime per il cliente e il coordinamento tecnico;
- il *project/service manager*$_G$, che pianifica le attività e la capacità del *team*.

Accanto a loro lavorano gli sviluppatori e chi progetta l'interfaccia. In un'agenzia lo stesso gruppo di persone lavora per più clienti nello stesso periodo, perciò nessuno è assegnato a un progetto in modo stabile: ogni mese un *demand planning* condiviso ripartisce il *team* sui clienti, e un rapporto sullo stato di avanzamento dei lavori

(SAL)$_G$ riporta con un semaforo lo stato di ciascun progetto, le scadenze, i rischi e i punti aperti.

Il *team* si coordina con gli strumenti della tabella 1.1 e con uno *stand-up*$_G$ quotidiano, una riunione mattutina di quindici-venti minuti in cui ciascuno espone su cosa ha lavorato il giorno prima e su cosa lavorerà quel giorno.

Strumento	**Uso**
Google Meet	riunioni a distanza, tra cui lo *stand-up*
Jira	registro delle richieste e *board* di avanzamento dei progetti
Slack	comunicazione con i clienti, in un canale per ciascun marchio
Email	resoconti delle attività e variazioni contrattuali
Confluence e Google Drive	documentazione e conoscenza interna

Tabella 1.1: Gli strumenti di coordinamento del *team* di Halue.

Gli orari e il luogo di lavoro sono flessibili: i dipendenti lavorano in sede indicativamente tra il 30% e il 40% del tempo e da remoto per il resto, e ognuno gestisce la propria presenza in autonomia. L'inserimento di un nuovo membro del *team* parte dalla documentazione e dalle procedure interne e prosegue con piccoli *ticket*$_G$, le unità di lavoro che Jira registra con un identificativo, una descrizione e uno stato, su un progetto reale e con l'affiancamento di un collega.

1.4.2 Dal *ticket* al rilascio

La figura 1.4 mostra il percorso di una richiesta del cliente, dal *ticket* su Jira alla manutenzione.

Gli *Epic* raccolgono insiemi di funzionalità, le *Story* descrivono singole funzionalità, le *Change Request* le modifiche chieste a progetto avviato e i *Bug* i difetti.

La tabella 1.2 raccoglie, nell'ordine della figura 1.4, le pratiche che accompagnano il ciclo. Il metodo è rigido dove la struttura si ripaga sempre, cioè nella tracciabilità dalla richiesta al codice e nelle stime, e flessibile dove i clienti sono diversi tra loro, come la copertura dei *test* o i livelli di servizio, che si decidono progetto per progetto. La stessa impostazione ha guidato il mio *stage*, per esempio nella scelta di concentrare i *test* automatici solo dove servivano (sezione 3.6).

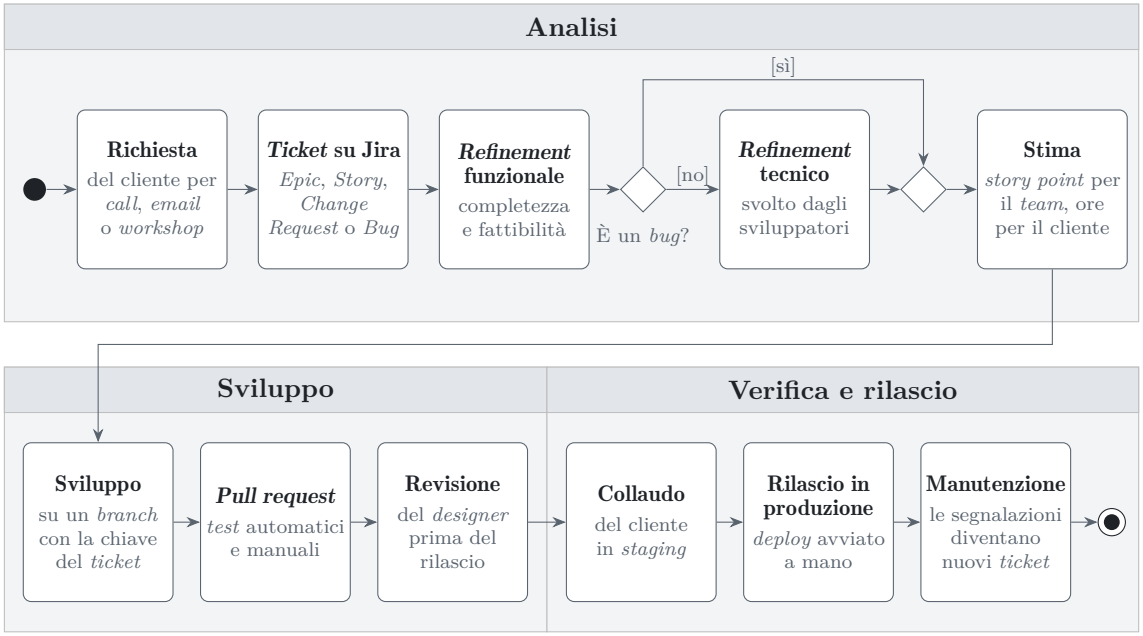


Figura 1.4: Il ciclo di vita di un *ticket* in Halue, come diagramma di attività UML.

Ambito	Pratica
Stima per il *team*	*story point*$_G$, una misura della complessità relativa sui numeri di Fibonacci da 1 a 13; oltre 13 il *ticket* si divide
Stima per il cliente	ore, con un metodo interno che pesa complessità, chiarezza della richiesta, *team* coinvolto e confidenza nella stima
Decisioni di architettura	scritte nel *ticket*, oppure in un ADR$_G$, un documento breve con contesto, scelta e motivazione, se valgono per tutto il progetto
Progetto grafico	da Figma, collegato al *ticket*, in versione per dispositivi mobili e per *desktop*
Codice	una *pull request*$_G$, cioè una proposta di integrazione delle modifiche, per ogni *ticket*, con la sua chiave nei messaggi di *commit*$_G$, le registrazioni delle modifiche nella storia del codice; stile controllato da ESLint e Prettier
Versioni	versionamento semantico (SemVer)$_G$, che distingue modifiche incompatibili, nuove funzionalità e correzioni, con *tag* e *release* su GitHub
Test	nessuna copertura minima prefissata
Collaudo	in *staging*$_G$ o *preview*, un ambiente separato che riproduce le condizioni di produzione
Deploy	automatizzato, secondo una procedura scritta
Manutenzione	segnalazioni da Jira, Slack o *email*; priorità concordate caso per caso, senza livelli di servizio formali
Retrospettive	anche alla chiusura di un progetto o di una *release* importante

Tabella 1.2: Le pratiche di Halue lungo il ciclo di un *ticket*.

1.5 Innovazione e intelligenza artificiale

Halue pratica uno *scouting* tecnologico continuo e dedica una quota del tempo a progetti di ricerca e sviluppo, tra cui quelli di intelligenza artificiale (figura 1.5). I dipendenti dedicano allo studio una parte rilevante del proprio tempo.

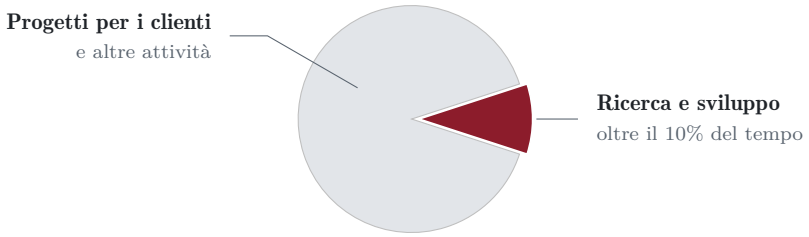


Figura 1.5: La quota del tempo che Halue dedica alla ricerca e sviluppo.

L'intelligenza artificiale è il campo in cui questa propensione si vede di più. Il *team* segue da vicino le novità del settore, testa regolarmente nuovi strumenti di automazione basati sull'IA e li porta nel lavoro di ogni giorno. Nello sviluppo l'IA esegue una prima revisione automatica del codice e analizza i *changelog* delle tecnologie in uso. Per i processi ricorrenti l'azienda usa agenti di IA guidati da *skill*$_{\text{G}}$ interne, cioè documenti di istruzioni, a volte accompagnati da *script* e modelli, che descrivono come svolgere un processo e che l'agente consulta quando il compito lo richiede. In tutti questi usi vale la stessa regola: l'IA propone e la persona decide (figura 1.6).

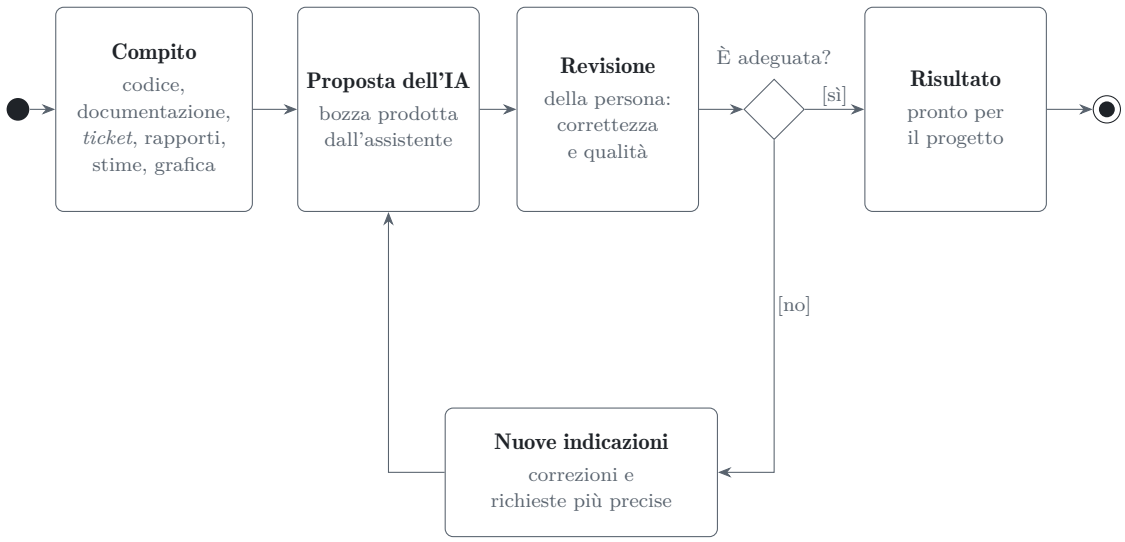


Figura 1.6: Il ciclo di lavoro con gli strumenti di intelligenza artificiale in Halue.

L'azienda riscontra una riduzione dei tempi, ma non dispone ancora di misure precise su questo beneficio e sull'impatto sulla qualità. Nel *team* è condivisa un'opinione su

cui torno nella sezione 4.2: nelle mani di un programmatore esperto l'IA è un buon programmatore *junior*, ma non può ancora sostituire un *senior*.

Nei progetti di IA per i clienti Halue adotta alcune regole che valgono anche per il mio progetto:

- i dati vengono sempre dal catalogo reale, tramite strumenti e ricerca, senza affidarsi alla memoria del modello;
- le azioni con effetti esterni restano sotto controllo umano;
- la qualità delle risposte si valuta con *test* manuali su casi d'uso reali;
- il modello si sceglie in base al compito, in modo da tenere sotto controllo il consumo dei *token*$_{\text{G}}$, i frammenti di testo in cui il modello scompone ciò che legge e scrive e su cui si calcola il costo del suo utilizzo.

Questa propensione all'innovazione passa anche dagli *stage*, che Halue usa per esplorare temi che i progetti per i clienti non lasciano il tempo di approfondire. Approfondisco questo ruolo nella sezione 2.1.

2 Oltre la conversazione testuale

Discuto le ragioni per cui Halue ha proposto questo *stage* e per cui io l'ho scelto. Tratto il ruolo degli *stage* in azienda, il problema che il progetto affronta, il suo posto nella strategia di Halue, gli obiettivi e i vincoli fissati dal piano di lavoro e le mie motivazioni.

2.1 Il ruolo degli *stage* nella strategia aziendale

Halue ospita in media tre o quattro *stage* all'anno, ed è soprattutto attraverso gli *stage* che collabora con le università. Gli stagisti provengono per lo più dall'Università di Padova, dai corsi di laurea triennale e magistrale. L'azienda attribuisce a queste esperienze due scopi:

- svolgere ricerca e sviluppo su temi che i progetti per i clienti non lasciano il tempo di esplorare;
- conoscere persone da inserire nel *team*, valutando al termine di ogni percorso la possibilità di proseguire la collaborazione con lo stagista.

Il primo scopo spiega la forma che gli *stage* assumono. Uno stagista lavora a tempo pieno su un solo tema per circa due mesi, da 300 a 320 ore lavorative, con una continuità che i dipendenti non hanno, perché l'azienda li ripartisce ogni mese tra più clienti (sezione 1.4.1). La quota di tempo che Halue dedica alla ricerca e sviluppo (sezione 1.5) è frammentata, per i dipendenti, tra un progetto e l'altro; in uno *stage* si concentra invece su un unico tema. Halue affida quindi agli stagisti temi nuovi, su cui non ha ancora un'esperienza consolidata, e un *tutor* che li segue ogni giorno. Il secondo scopo spiega perché lo stagista partecipa ai momenti di coordinamento del *team* e ne adotta gli strumenti, dato che l'azienda valuta il suo modo di lavorare oltre ai risultati che produce.

Alcuni degli *stage* che hanno preceduto il mio riguardavano già gli assistenti conversazionali. Uno di questi, svolto alcuni anni prima, aveva progettato un assistente basato su flussi di conversazione predefiniti. In un sistema di questo tipo il progettista definisce in anticipo gli stati della conversazione e i passaggi tra l'uno e l'altro. A ogni stato l'assistente riconosce, con un modello addestrato a classificare le frasi, l'intenzione espressa dall'utente, la associa a uno dei passaggi previsti e gli propone messaggi predefiniti che lo conducono verso una risposta o un'azione. Il comportamento è deterministico, perché a parità di richiesta l'assistente segue sempre lo stesso percorso, e una richiesta che il progettista non ha previsto riceve soltanto una risposta di ripiego.

Un agente basato su un modello linguistico funziona in un altro modo. Il progettista definisce gli strumenti a disposizione e le istruzioni da seguire, e il modello decide a ogni turno quali strumenti invocare e come comporre la risposta. Il comportamento diventa non deterministico, perché la stessa richiesta può ricevere risposte diverse, ma l'agente affronta anche richieste che nessuno ha previsto. Questo cambiamento, insieme agli strumenti di sviluppo basati sull'IA, ha esteso ciò che una persona può realizzare nelle ore di uno *stage*. Rispetto a quel progetto, ChatCart non segue percorsi stabiliti: cerca nel catalogo, compone l'interfaccia e modifica il carrello in base alla conversazione.

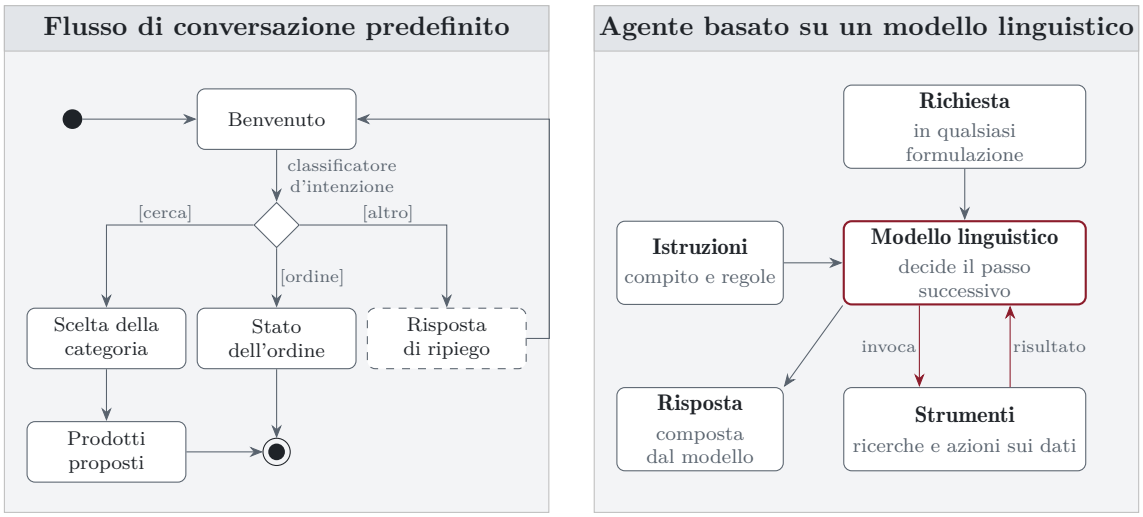


Figura 2.1: A sinistra un esempio di flusso predefinito come diagramma degli stati UML: le guardie tra parentesi quadre sono le intenzioni riconosciute dal classificatore, e ogni richiesta che non rientra tra quelle previste riceve la risposta di ripiego e torna al benvenuto. A destra il ciclo di un agente, in cui il modello invoca gli strumenti e ne legge i risultati finché non dispone di ciò che serve per rispondere.

Ogni *stage* si inserisce in un percorso che lo precede e lo segue. Il tema nasce da un'esigenza interna oppure dall'interesse che può avere per un laureando in infor-

matica. Nel mio caso il percorso è iniziato con la candidatura a luglio 2026 e con un colloquio conoscitivo ad agosto con il *tutor* e con il fondatore. Dopo il colloquio Halue mi ha proposto alcuni temi, tutti di intelligenza artificiale, tra cui:

- uno *storefront* con interfaccia generativa, in cui un agente costruisce la pagina del negozio in tempo reale in base a ciò che il cliente cerca;
- lo sviluppo di un agente interno all'azienda, integrato in profondità con la conoscenza e con gli strumenti di Halue. Il tema si concentrava sulla gestione del contesto dell'agente, cioè sulla scelta delle informazioni da fornire al modello insieme a ogni richiesta, che non possono essere troppe, perché il modello ne elabora una quantità limitata. Per sceglierle serve un sistema di ricerca semantica$_G$, che recupera le informazioni in base al loro significato e non soltanto alle parole che contengono;
- il monitoraggio dei sistemi di IA in produzione, cioè il controllo del comportamento degli agenti già in uso presso l'azienda o i suoi clienti.

I temi coprono tutto il ciclo di un sistema di IA: l'esperienza di chi acquista nei negozi dei clienti di Halue, i processi interni, in cui l'azienda usa già agenti di IA (sezione 1.5), e il funzionamento dei sistemi dopo il rilascio. Ho scelto il primo e l'ho declinato in un assistente di chat, per le ragioni che espongo nella sezione 2.5. ChatCart tocca però, su scala ridotta, anche gli altri due temi:

- l'agente lavora con il contesto del negozio, cioè il catalogo e le pagine informative, che recupera con strumenti dedicati quando la conversazione li richiede;
- le metriche e l'analisi delle conversazioni (OB5, tabella 2.1) sono una prima forma di monitoraggio di un sistema di IA in uso.

Ciascuno dei tre ambiti si presta quindi a un'espansione in profondità: un'interfaccia generativa che compone intere pagine del negozio, un contesto esteso agli ordini e ai dati dei clienti, un monitoraggio che valuta la qualità delle risposte e i costi nel tempo. Ho poi scritto con il *tutor* il piano di lavoro.

Al termine lo stagista presenta il lavoro a tutto il *team* e partecipa a una retrospettiva, simile a quelle con cui Halue chiude gli *sprint* e i progetti per i clienti, in cui si discute che cosa ha funzionato e che cosa correggere. Il lavoro può quindi avere due esiti:

- l'azienda lo archivia come base di conoscenza sul tema;
- l'azienda lo rivede e ne riusa una parte nel codice e nei progetti di produzione.

La scelta dipende dal tipo di progetto e dalla qualità del prodotto. Più avanti descrivo i possibili utilizzi di ChatCart per Halue (sezione 2.3) e le direzioni in cui si può sviluppare (sezione 3.5).

2.2 I limiti degli assistenti conversazionali testuali

Molti negozi *online* offrono ai clienti un *chatbot*$_G$, un programma che conversa con l'utente in linguaggio naturale. Di solito compare come *widget*$_G$, un riquadro sovrapposto alle pagine del negozio che il cliente apre con un pulsante. Questi assistenti rispondono quasi sempre con solo testo. Il testo è adatto alle domande frequenti, ma è poco efficace quando il cliente deve confrontare prodotti, scegliere una taglia o controllare un ordine, cioè nelle operazioni che le pagine di un negozio risolvono con immagini, prezzi, selettori e pulsanti. Un assistente che descrive a parole tre prodotti costringe il cliente ad abbandonare la conversazione per vederli, e per il commerciante questo si traduce in conversazioni che non si concludono con un acquisto. La figura 2.2 mette a confronto le due risposte alla stessa richiesta.

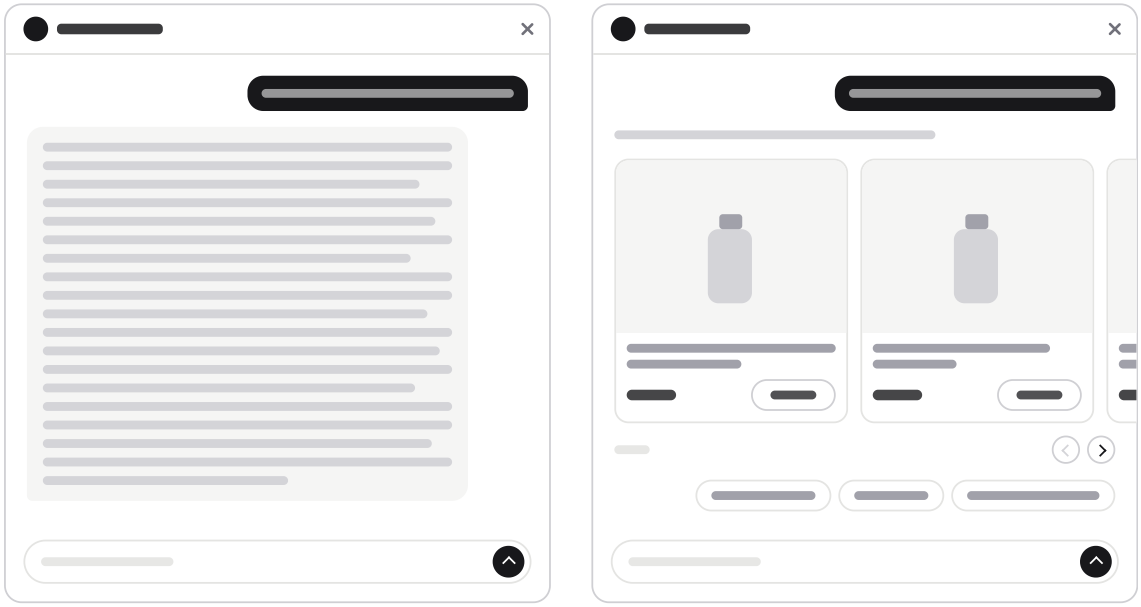


Figura 2.2: *Mockup* della risposta alla stessa richiesta di un assistente testuale, a sinistra, e di ChatCart, a destra.

Il potenziale di un assistente efficace è misurabile. Uno studio sperimentale condotto tra il 2023 e il 2024 su una delle maggiori piattaforme internazionali di commercio elettronico[5] ha confrontato sette applicazioni dell'IA generativa, ciascuna con un gruppo di clienti che la usava e un gruppo di controllo. L'effetto maggiore è quello dell'assistente che risponde alle domande dei clienti prima dell'acquisto,

che ha aumentato le vendite del 16,3%. L'aumento deriva da un numero maggiore di clienti che acquistano, mentre il valore medio del carrello resta invariato. I clienti che traggono più vantaggio dall'assistente sono quelli che conoscono meno la piattaforma.

Le soluzioni disponibili sul mercato appartengono a cinque tipi, e ciascuno lascia scoperta una parte del problema. I primi due sono prodotti di terze parti rivolti a chi acquista:

- **Gli assistenti generici per l'assistenza clienti** risolvono richieste di supporto su più canali a partire dai documenti dell'azienda. Conoscono poco il catalogo e rispondono con testo, quindi sono adatti alla gestione di un reso più che alla scelta di un prodotto.
- **Gli assistenti alla vendita preconfezionati** si installano nel negozio e consigliano prodotti. L'interfaccia, il modello e il comportamento sono decisi dal fornitore e restano uguali per tutti i negozi che li adottano.

Gli altri tre li offre Shopify, da sola o insieme a Google, e non si rivolgono direttamente a chi acquista nel negozio:

- **Sidekick**[6], l'assistente che Shopify integra nel pannello di amministrazione, supporta il commerciante nella gestione del negozio e non si rivolge a chi acquista.
- **Il *server* MCP del negozio**[7], che Shopify attiva per ogni negozio, espone agli agenti la ricerca nel catalogo, il carrello e le condizioni di vendita. È un componente destinato a chi sviluppa un agente, perché fornisce dati e operazioni senza un'interfaccia, e la sua ricerca è la stessa per ogni catalogo.
- **UCP$_{\mathbf{G}}$**[8], uno standard aperto che Shopify e Google hanno pubblicato nel gennaio 2026, permette a un agente di scoprire che cosa offre un negozio e di completare un acquisto. È pensato per gli assistenti di terze parti, per cui la conversazione avviene fuori dal negozio, in un'interfaccia che il marchio non controlla.

Nessuna di queste soluzioni soddisfa insieme i tre requisiti a cui i clienti di Halue attribuiscono più valore:

- l'identità visiva del marchio, perché l'assistente fa parte del negozio e deve avere il suo stesso aspetto;
- i dati specifici del catalogo, con le varianti, le collezioni e le informazioni che distinguono i prodotti di quel marchio;
- un'esperienza d'acquisto che si svolge interamente nel negozio, dalla domanda al carrello.

Un prodotto preconfezionato deve adattarsi a tutti i commercianti, e per questo lascia poco spazio alla personalizzazione. Halue costruisce soluzioni per il singolo cliente, perciò ha scelto di esplorare un assistente su misura basato su un'interfaccia generativa$_G$, cioè un'interfaccia che il modello compone durante la conversazione invece di restare fissata da chi la sviluppa. Il termine comprende approcci che si distinguono per quanto controllo lasciano al modello:

- il modello risponde soltanto con testo, e l'interfaccia resta quella fissa della chat;
- il modello sceglie un componente da una libreria predefinita, come una scheda prodotto, e lo popola in tempo reale con i dati del negozio, mentre aspetto e comportamento del componente restano decisi da chi lo sviluppa;
- il modello genera da sé la struttura dell'interfaccia, per esempio sotto forma di codice o di una descrizione degli elementi da disporre, e può quindi produrre forme che nessuno ha previsto.

Halue ha scelto il secondo approccio. Il marchio conserva il controllo sull'aspetto dei componenti, e il modello decide soltanto quale mostrare e con quali dati. Le scelte del modello restano non deterministiche, ma ognuna produce un componente progettato e verificato in anticipo: un errore riguarda al più il componente o i dati scelti, mentre la grafica e il funzionamento dei pulsanti restano quelli previsti. Il terzo approccio offre più libertà, ma rende difficile garantire la coerenza con il tema del negozio e la correttezza di prezzi, varianti e pulsanti d'acquisto, che in un negozio hanno conseguenze dirette sulle vendite.

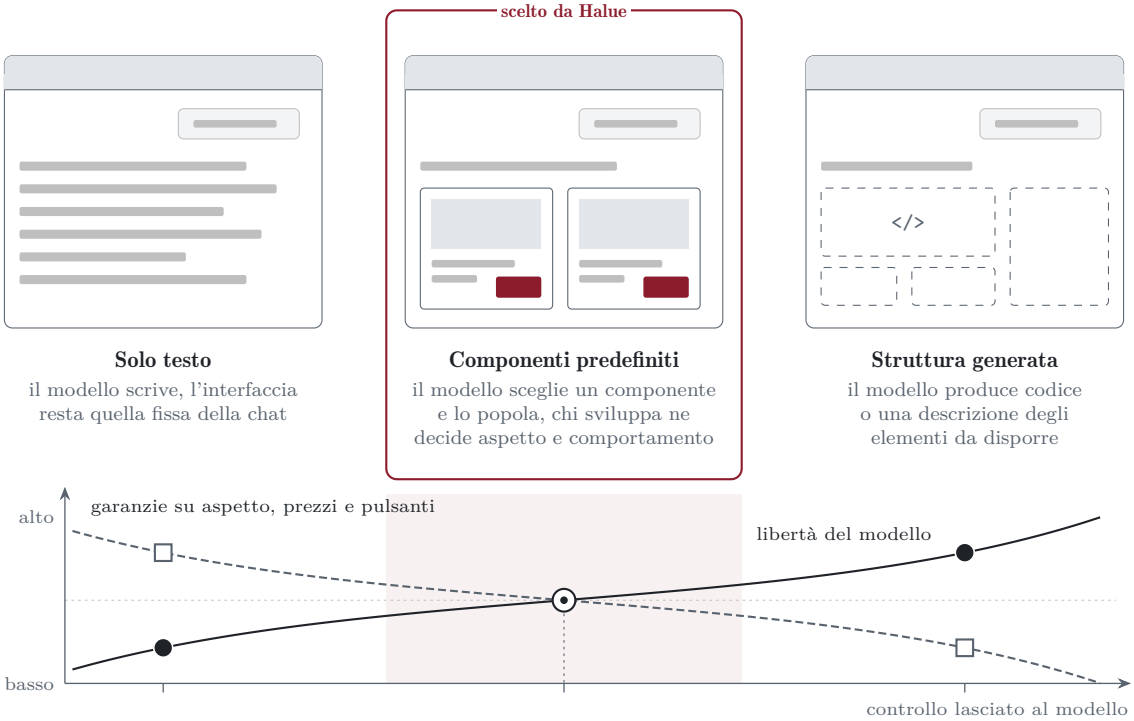


Figura 2.3: I tre approcci all'interfaccia generativa, in ordine di controllo lasciato al modello. Le finestre sono schematiche: in bordeaux i pulsanti d'acquisto, che nel secondo approccio fanno parte del componente e non dipendono dal modello; i riquadri tratteggiati indicano elementi di cui il modello decide forma e posizione. Il grafico è qualitativo e confronta gli approcci senza misurarli: l'approccio scelto da Halue si colloca dove le curve si incrociano, cioè dove la libertà del modello e le garanzie si bilanciano.

2.3 Il commercio agentico nella visione di Halue

Il *server* MCP del negozio e UCP, che ho presentato nella sezione 2.2 tra le soluzioni disponibili, sono due elementi di un cambiamento più ampio, il commercio agentico$_G$: l'insieme delle esperienze d'acquisto in cui un agente di intelligenza artificiale accompagna il cliente nella ricerca, nella scelta e nell'acquisto di un prodotto. Queste soluzioni mostrano le due direzioni in cui questo cambiamento si sviluppa:

- nella prima il negozio si apre agli assistenti generici di terze parti, che lo raggiungono attraverso protocolli come MCP e UCP; il cliente conversa altrove e il negozio fornisce il catalogo e il pagamento;
- nella seconda l'assistente appartiene al marchio e opera dentro il negozio, con la sua interfaccia e con strumenti realizzati attorno al suo catalogo.

Le piattaforme promuovono la prima direzione, poiché qualsiasi commerciante la può adottare senza alcuno sviluppo dedicato. In cambio il negozio diventa una fonte di dati per un assistente che il marchio non controlla. È l'assistente a decidere come

presentare i prodotti, quali proporre accanto a quello cercato e in quale ordine, e la relazione con il cliente passa per un'interfaccia esterna. La seconda direzione lascia questa relazione al marchio, ma richiede un progetto su misura per ogni negozio.

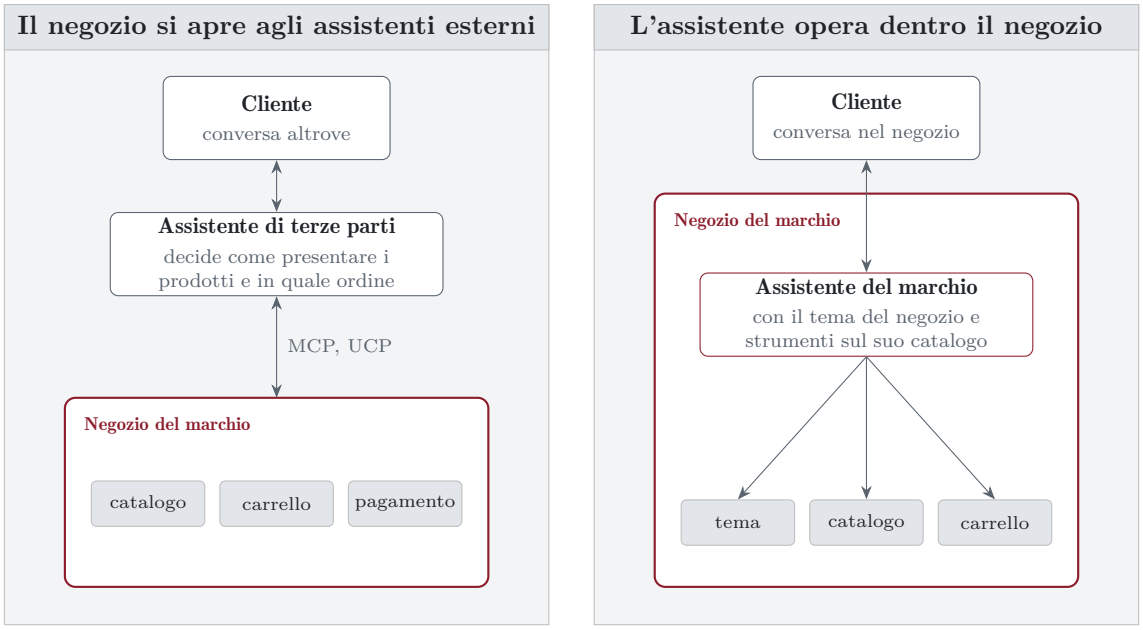


Figura 2.4: Le due direzioni del commercio agentico. Il bordo bordeaux delimita ciò che il marchio controlla: nella prima direzione soltanto i dati e il pagamento, nella seconda anche la conversazione. Nella seconda gli strumenti dell'assistente sono costruiti sul tema e sul catalogo del negozio, mentre il *server* MCP espone le stesse operazioni a qualsiasi agente.

Un assistente con l'aspetto del marchio e la conoscenza del suo catalogo, come quello che serve ai clienti di Halue, appartiene alla seconda direzione. Halue ha scelto di esplorarla con ChatCart, pur seguendo già i nuovi protocolli nei progetti dei clienti: le due direzioni possono convivere, perché lo stesso negozio può esporre il catalogo agli assistenti esterni e offrire ai visitatori un assistente proprio.

Oltre alle ragioni di prodotto, la scelta dipende anche dal modello di attività dell'azienda. Halue lavora su commessa e non vende prodotti propri (sezione 1.1), quindi innova attraverso le competenze che porta nei progetti dei clienti. Un assistente del marchio si costruisce sul catalogo, sul tema e sui processi di un singolo negozio, cioè sul lavoro che Halue svolge già per i suoi clienti Shopify Plus. In questa direzione un'azienda delle sue dimensioni può offrire un servizio che le piattaforme non coprono, mettendo a frutto la conoscenza approfondita che ha dei negozi dei propri clienti.

Halue sviluppa già agenti di IA collegati ai dati dei negozi e li usa nei propri processi interni (sezione 1.5). Un assistente che conversa con chi visita il negozio pone però

problemi propri: deve integrarsi con il tema del negozio, rispondere in pochi secondi e comportarsi in modo prevedibile con qualsiasi interlocutore. Lo *stage* doveva approfondire questi aspetti su un caso abbastanza circoscritto da poterlo completare nella durata prevista.

Il risultato è per Halue una base di ricerca e sviluppo. L'obiettivo dichiarato è arrivare a proporre eventualmente questo tipo di soluzione ai clienti come servizio, con tempi e forma ancora da stabilire. Il primo passo è stata la presentazione al *team* del 9 novembre 2026, l'ultimo giorno di *stage*: trenta minuti sul prodotto, quindici sulla parte tecnica e quindici di domande. Le domande hanno indicato che cosa interessa a chi riprenderà il lavoro, cioè le scelte di progettazione e architetturali, come la gestione dello stato nella sincronizzazione del carrello, e il modo in cui ho istruito l'agente perché si comportasse come previsto.

2.4 Obiettivi e vincoli del progetto

Il piano di lavoro, che ho scritto con il *tutor* ad agosto 2026, divide il prototipo in due livelli. L'*app layer* è il *widget* di chat nel negozio e si limita a mostrare la conversazione e i componenti. L'*agent layer* è il *backend*, cioè il programma eseguito su un *server*, che ospita l'agente e conserva lo stato della conversazione. Per interrogare le API di Shopify l'agente usa il *tool calling*$_{\mathrm{G}}$, il meccanismo con cui un modello linguistico chiede al programma che lo ospita di eseguire una funzione e ne riceve il risultato.

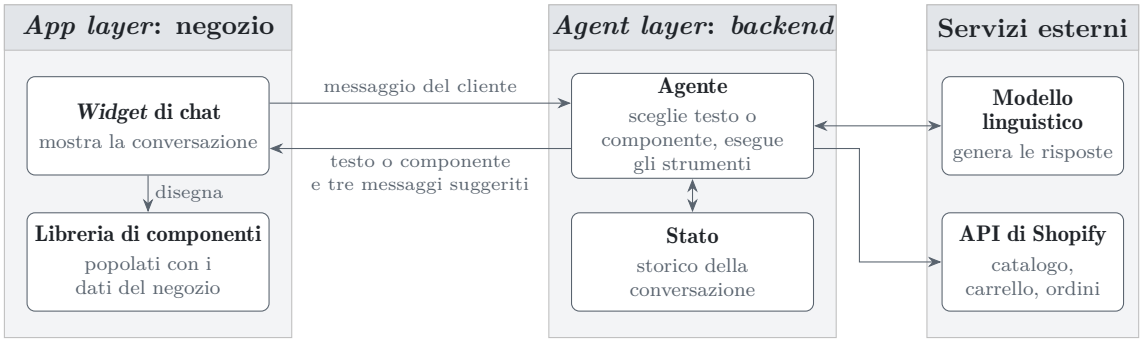


Figura 2.5: L'architettura su due livelli prevista dal piano di lavoro. Il *widget* non contiene logica conversazionale: ogni decisione spetta all'agente, che può interrogare più volte modello e API prima di rispondere.

Ogni obiettivo ha un identificativo, che uso nel resto della relazione, e uno di tre livelli di priorità. Gli obiettivi obbligatori (tabella 2.1) sono vincolanti. Chiedono di dimostrare che l'approccio è fattibile e di misurarne l'efficacia.

ID	Obiettivo
OB1	*App layer*: *widget* di chat in React e TypeScript integrato nel negozio
OB2	*Agent layer*: *backend* Node.js con un agente dotato di *tool calling*, in grado di interrogare le API di Shopify
OB3	Risposte in linguaggio naturale (domande generali, sui prodotti, di chiarimento) oppure con un componente predefinito, scelto e popolato in base alla conversazione
OB4	Libreria di almeno due o tre componenti flessibili e tre messaggi suggeriti per ogni risposta
OB5	Raccolta di metriche di coinvolgimento e di conversione sull'uso del *widget*

Tabella 2.1: Gli obiettivi obbligatori del piano di lavoro.

Gli obiettivi desiderabili hanno un valore aggiunto riconoscibile, quelli facoltativi un valore aggiunto non prioritario (tabella 2.2). Riprendono tre aspetti che il *tutor* ha indicato come interessanti per l'azienda:

- il supporto a più mercati, che riguarda molti clienti di Halue;
- i temi Hydrogen, in cui riusare i componenti React è più semplice e integrarsi con il carrello è più complesso;
- una ricerca nel catalogo scritta su misura, perché il *tutor* giudicava elementare quella del *server* MCP di Shopify.

ID	Priorità	Obiettivo
DE1	desiderabile	Supporto a Shopify Markets$_G$, la funzione con cui un negozio vende in più paesi con lingue e valute diverse
DE2	desiderabile	Componente per lo stato dell'ordine e il supporto dopo la vendita
FA1	facoltativo	Prova di fattibilità dell'integrazione su Hydrogen, con il riuso dei componenti del *widget*
FA2	facoltativo	Confronto sperimentale tra la ricerca su misura, il *server* MCP di Shopify e un eventuale *server* MCP proprio

Tabella 2.2: Gli obiettivi desiderabili e facoltativi del piano di lavoro.

Gli obiettivi traducono in prodotto i requisiti della sezione 2.2. OB1 porta l'assistente dentro il negozio, così che l'esperienza d'acquisto vi si svolga per intero. OB2 collega l'agente ai dati reali del catalogo, secondo la prima delle regole che Halue segue nei progetti di IA (sezione 1.5). OB3 e OB4 realizzano l'interfaccia generativa con componenti predefiniti, che rispettano l'identità visiva del marchio, e DE1 estende lo stesso comportamento ai negozi che vendono su più mercati. OB5 serve

a verificare se l'assistente produce il beneficio che lo studio citato nella sezione 2.2 attribuisce a un assistente efficace, cioè più clienti che arrivano all'acquisto.

Per l'azienda il criterio di successo era un prototipo funzionante dall'inizio alla fine su un negozio di esempio, accompagnato da una valutazione onesta di ciò che funziona e di ciò che non funziona. Nella sezione 4.1 valuto in che misura ho soddisfatto ogni obiettivo.

Il progetto aveva sei vincoli, ciascuno con una conseguenza sul lavoro:

- **Quantità di lavoro**: da 300 a 320 ore, al massimo a tempo pieno, dal 14 settembre al 9 novembre 2026. Con una durata fissa dovevo decidere presto che cosa approfondire e che cosa lasciare agli sviluppi futuri.
- **Modalità di lavoro**: da remoto, con un giorno a settimana in sede, un allineamento quotidiano con il *tutor* e una riunione settimanale di un'ora. Il confronto passava quindi soprattutto per brevi riunioni a distanza e per i messaggi su Slack.
- **Tecnologie**: TypeScript in tutto il progetto, dal *widget* al *backend*. Il piano rimandava alle prime settimane la scelta del modello linguistico e degli strumenti per costruire l'agente, che il *tutor* ha lasciato a me.
- **Piattaforma**: prima i temi Liquid, i più diffusi, e Hydrogen soltanto come eventuale approfondimento finale. Il *widget* doveva quindi funzionare all'interno di temi scritti da altri.
- **Ambiente**: un unico negozio di sviluppo, senza clienti né traffico reali, con due temi installati per le prove. L'ho allestito come un negozio di cosmetica botanica con circa quaranta prodotti, perché il comportamento di un assistente dipende dal catalogo su cui opera. Varianti, collezioni, confezioni regalo, prodotti esauriti e archiviati lo avvicinano a un caso reale. Senza traffico, però, le metriche di OB5 vengono soltanto dalle prove.
- **Natura del prodotto**: un prototipo di ricerca e sviluppo, essenziale ma solido, senza i requisiti di scala e di costo di un prodotto in produzione. Per questo non ho ottimizzato il consumo di *token* né la capacità di servire molti negozi insieme.

Il piano ripartisce 310 ore indicative tra dieci attività (figura 2.6) e prevede di rifinire la ripartizione con il *tutor* nelle prime settimane. Nella sezione 3.1.1 la confronto con il tempo che ho effettivamente impiegato.

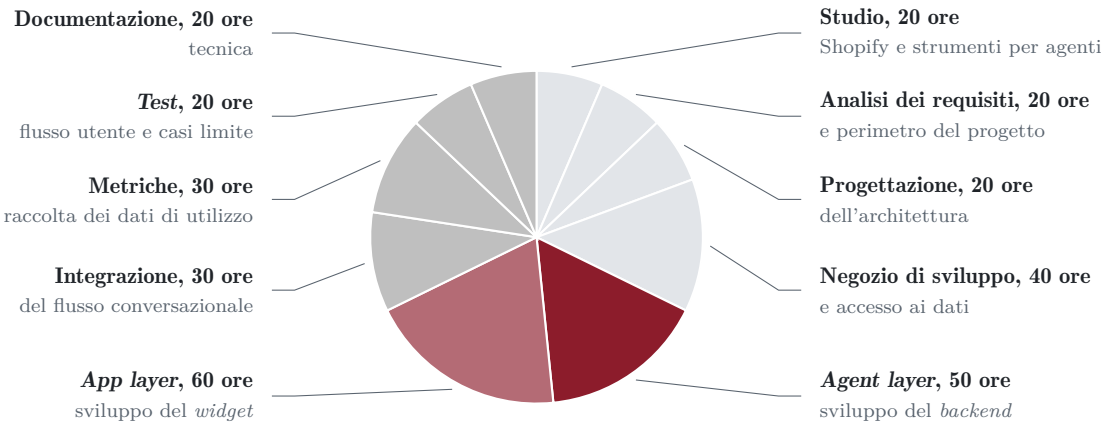


Figura 2.6: La ripartizione delle ore prevista dal piano di lavoro, in senso orario dall'alto. In grigio chiaro le attività che precedono la scrittura del codice, in bordeaux lo sviluppo dei due livelli, in grigio medio le attività che lo completano.

2.5 Motivazioni e obiettivi personali

Da marzo 2026 lavoro in proprio come sviluppatore di negozi Shopify. Il mio cliente principale è Dr Kampes[9], un marchio per cui ho seguito quasi ogni aspetto del negozio: ho creato e configurato il negozio, gestito e ampliato il catalogo, impostato le regole di spedizione, realizzato lo *storefront* e collegato il CRM, con le automazioni successive all'acquisto e quelle di *marketing*. Questo lavoro mi ha dato una visione ampia della piattaforma, ma lasciava una lacuna nella mia esperienza: lo sviluppo di applicazioni su misura e la gestione del negozio attraverso le Admin API di Shopify.

Prima di scegliere Halue ho sostenuto un colloquio con un'altra azienda, esterna alle offerte dell'ateneo, e ho valutato le altre proposte di *stage*. Erano pensate per il lavoro, e per un potenziale inserimento, in aziende grandi e strutturate, mentre io, per esperienza personale, ho sempre prediletto gli ambienti piccoli e dinamici, in cui posso sperimentare creativamente e procedere per iterazioni rapide, come quelli

delle *startup*$_{\text{G}}$, le giovani imprese nate per sviluppare un prodotto innovativo. La proposta di Halue rispondeva a questa preferenza e offriva tre vantaggi:

- riguardava Shopify, la piattaforma su cui già lavoravo, in un'azienda che la usa ogni giorno, e mi permetteva di passare dall'uso dei suoi strumenti alla costruzione di applicazioni;
- mi garantiva due mesi a tempo pieno su tecnologie che già mi interessavano, un'occasione che il lavoro in proprio concede di rado;
- prevedeva il lavoro soprattutto da remoto, che Halue mi ha proposto già al colloquio. Per me era una condizione decisiva: il lavoro da remoto corrisponde al mio modo di lavorare, e intendo privilegiarlo anche nei lavori futuri per la mobilità e le scelte di vita che consente.

Tra i temi proposti ho scelto l'interfaccia generativa. La gestione del contesto mi interessava molto e apriva diverse strade di integrazione con altri strumenti e sistemi, ma comportava un rischio. Aziende come Anthropic[10] e OpenAI[11] aggiornano i propri assistenti a ritmo serrato, e vi aggiungono una dopo l'altra funzioni come la ricerca nei documenti e negli strumenti di un'azienda. Un sistema costruito in due mesi poteva diventare superfluo con un solo aggiornamento di un assistente generico. Un assistente di vendita nel negozio di un marchio dipende invece da elementi che solo quel negozio possiede, cioè il catalogo, il tema e il carrello, e un assistente generico lo sostituisce con più difficoltà. Risponde inoltre a un problema di conversione, con metriche misurabili (sezione 2.2), ed è più semplice da offrire ai clienti come servizio personalizzato.

L'idea dell'interfaccia generativa è di Halue, che la proponeva come uno *storefront* costruito in tempo reale da un agente. Vi ho riconosciuto un'esigenza di Dr Kampes, che mi aveva parlato di un assistente capace di aiutare i visitatori del negozio e di indirizzarli verso l'acquisto. Ho quindi proposto di declinare il tema in un assistente di chat che sceglie e popola con i dati del negozio componenti come schede prodotto, tabelle di confronto, guide alle taglie e lo stato di un ordine, e che accompagna il cliente dalla scoperta dei prodotti all'assistenza dopo la vendita. Un assistente di questo tipo si aggiunge al tema esistente di un negozio, e si applica quindi anche ai negozi che Halue e io, su scala diversa, seguiamo già.

Il tema toccava anche altri miei interessi:

- imparare a sviluppare applicazioni Shopify, per estendere gli *storefront* con soluzioni su misura;
- collegare i negozi ai miei agenti di IA attraverso le Storefront API e le Admin API (sezione 1.3);
- conoscere la piattaforma a fondo, capendo come si incastrano i suoi componenti, per costruire da zero ciò che finora avevo soltanto usato.

Volevo infine mettere alla prova in un progetto reale ciò che ho imparato nel corso di studi: i *design pattern*$_{\text{G}}$, cioè soluzioni riconosciute a problemi ricorrenti di progettazione, il rigore nella pianificazione e una progettazione che facilita le estensioni future e la manutenzione.

Da queste ragioni ho ricavato quattro obiettivi personali (tabella 2.3), che nella sezione 4.1 valuto insieme a quelli del piano.

ID	Obiettivo
OP1	Estendere le mie competenze su Shopify dai temi allo sviluppo di applicazioni
OP2	Acquisire esperienza nella progettazione di sistemi basati su agenti
OP3	Ricavare dal progetto conoscenze e soluzioni riutilizzabili nella mia attività professionale
OP4	Gestire in autonomia il lavoro da remoto, dove nessuno segue la mia giornata in modo continuo e spetta a me organizzarla, stabilire le priorità e prevedere le pause

Tabella 2.3: I miei obiettivi personali per lo *stage*.

Questo capitolo e il precedente definiscono il contesto del progetto: l'azienda che lo ha proposto, il problema che affronta, gli obiettivi e i vincoli fissati dal piano di lavoro e le ragioni per cui l'ho scelto. Il progetto di *stage* nasce dall'incontro di tre esigenze: Halue voleva svolgere una ricerca propria sul commercio agentico, gli assistenti disponibili sul mercato rispondono soltanto con testo o scendono a compromessi significativi sull'identità del marchio e sui dati del catalogo, e io cercavo l'occasione di approfondire l'ecosistema Shopify, dallo sviluppo di applicazioni su misura con le Storefront API e le Admin API alla progettazione di agenti basati su modelli linguistici. Nel capitolo 3 racconto la progettazione e lo sviluppo di ChatCart, dalle prime settimane di pianificazione alla presentazione al *team*.

3 Progetto e sviluppo di ChatCart

Descrivo come ho trasformato gli obiettivi del piano di lavoro in ChatCart: il metodo con cui ho organizzato lo *stage*, lo studio della piattaforma e dei requisiti, le scelte architetturali e realizzative, le sperimentazioni abbandonate, la verifica e i risultati del prototipo.

3.1 Metodo di lavoro

3.1.1 Pianificazione e interazione con il *tutor* aziendale

Il piano di lavoro assegnava 310 ore indicative a dieci attività, senza distribuirle in un calendario settimanale. La scelta era coerente con la natura esplorativa del progetto, perché Halue non sapeva ancora quale forma avrebbe preso un assistente con interfaccia generativa, e un calendario dettagliato avrebbe fissato in anticipo decisioni che dipendevano da ciò che avrei scoperto lavorando. Io e il *tutor* abbiamo quindi tenuto fissi gli obiettivi e stabilito di volta in volta le attività successive in base allo stato del prototipo. Un procedimento di questo tipo richiede confronti frequenti, che il piano prevedeva quotidiani e settimanali e che ho adattato al lavoro prevalentemente da remoto (tabella 3.1).

Momento previsto	**Svolgimento effettivo**
Allineamento quotidiano con il *team*	Ho partecipato allo *stand-up* mattutino, riportando attività concluse, lavoro previsto e impedimenti.
Allineamento quotidiano con il *tutor*	Ho inviato un resoconto scritto di fine giornata nel canale Slack dedicato.
Revisione settimanale di un'ora	Io e il *tutor* abbiamo discusso il prototipo, i problemi aperti e le attività successive.
Supporto su richiesta	Ho chiesto riunioni aggiuntive quando una scelta bloccava il lavoro; in alcuni confronti è intervenuto anche il fondatore.

Momento previsto	Svolgimento effettivo
Presenza in sede	Dopo i primi due giorni in ufficio, sono stato in sede in media un giorno a settimana.

Tabella 3.1: Le modalità di coordinamento durante lo *stage*.

Il lavoro da remoto rendeva il confronto meno immediato che in ufficio, e nelle prime settimane ponevo le domande al *tutor* troppo tardi, dopo aver dedicato tempo a soluzioni che un confronto anticipato avrebbe corretto. Ho quindi iniziato a preparare ogni confronto per iscritto, indicando il risultato atteso, le alternative già provate e le decisioni da prendere insieme. La forma scritta mi obbligava a definire il problema prima della riunione e lasciava traccia della decisione presa. Per la sincronizzazione del carrello, per esempio, ho presentato tre possibilità: specializzare il *widget* per un tema, supportare una famiglia di temi oppure offrire un'integrazione generica. Da quel confronto è nata la combinazione di adattatori e ripiego descritta nella sezione 3.4.2; ciò che ho imparato dalla comunicazione a distanza è oggetto della sezione 4.2.

Il primo passaggio importante è arrivato presto. Nella prima settimana ho studiato l'ecosistema Shopify e definito il perimetro del progetto; nella seconda ho completato un primo prototipo funzionante, con cui ho verificato le scelte architetturali concordate con il *tutor*. Alla fine della terza settimana, dopo circa sette giorni passati a lavorare sul codice, avevo una prima versione che copriva gli obiettivi obbligatori OB1–OB5. Il *tutor* l'ha valutata positivamente e, con più di metà dello *stage* ancora da svolgere, abbiamo deciso di dedicare il tempo restante all'esperienza d'uso e alla sperimentazione, che Halue considerava la parte di maggior valore del progetto. La prima versione è diventata così una base da mettere alla prova, e funzionalità e architettura hanno continuato a cambiare fino alle ultime settimane.

Per capire quanto questa decisione abbia modificato la distribuzione del lavoro, ho ripartito a consuntivo le 304 ore dello *stage* tra le attività del piano, basandomi sulla storia delle modifiche al codice e sulle date dei documenti di progetto. La figura 3.1 riporta il risultato in ore intere e indicative.

La progettazione ha richiesto il doppio delle ore previste, perché vi sono tornato ogni volta che una prova mostrava un limite della soluzione adottata. La prima realizzazione dell'agente e dell'accesso al negozio ha richiesto invece circa la metà del tempo previsto, perché il modello ufficiale di Shopify per le applicazioni forniva già autenticazione, sessioni e integrazione con l'amministrazione. Lo sviluppo del-

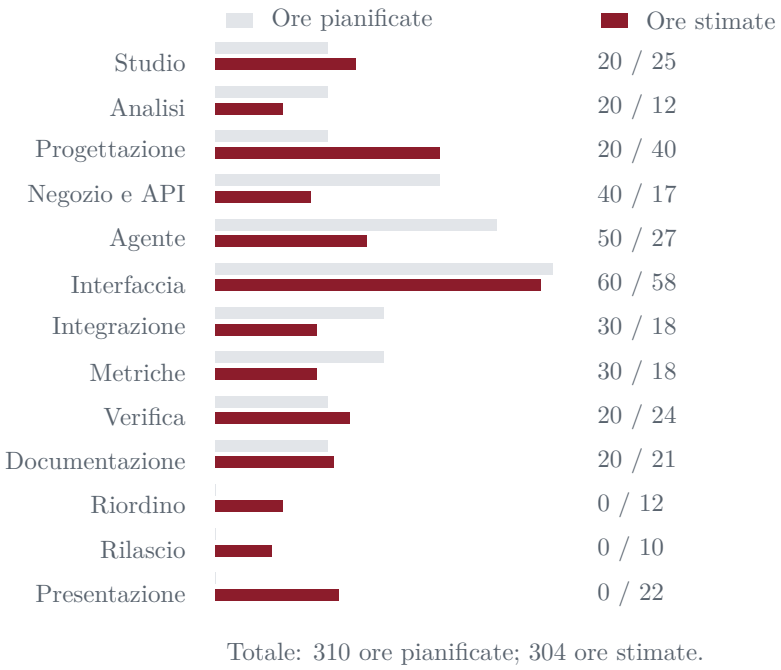


Figura 3.1: Le ore del piano e la mia stima a consuntivo, in valori indicativi. Le ultime tre attività non avevano una voce autonoma nel piano. Le barre confrontano quantità di lavoro, senza attribuire al piano un calendario che non prevedeva.

l'interfaccia si è esteso lungo quasi tutto lo *stage*, comprese le sperimentazioni poi abbandonate, e il tempo risparmiato nelle prime settimane è andato al riordino del codice, al rilascio e alla presentazione finale al resto del *team*, attività che il piano non prevedeva come voci autonome. La distribuzione effettiva riflette quindi la decisione presa alla fine della terza settimana, che ha trasformato un piano pensato per costruire un prodotto in un percorso di ricerca e sviluppo, in cui provavo ogni soluzione, la valutavo con il *tutor* e, quando serviva, la rivedevo.

3.1.2 Organizzazione del lavoro

Lavorando quasi sempre da remoto, su un progetto senza un calendario prefissato, dovevo stabilire ogni giorno a che cosa dedicare il tempo. Per questo ho organizzato il lavoro intorno a un contesto scritto e aggiornato quotidianamente. La mattina registravo un *brain dump*$_{\text{G}}$ vocale, cioè una raccolta libera e parlata di attività, dubbi e idee, e lo trascrivevo con strumenti miei basati su un modello di trascrizione. Dalla trascrizione estraevo le azioni, assegnavo loro una priorità e le collocavo in blocchi di tempo del calendario, così da ripartire la mattina successiva da ciò che avevo concluso e da ciò che restava aperto. Il procedimento è vicino al metodo Getting

Things Done di David Allen[18], che propone di raccogliere e organizzare fuori dalla memoria ciò che richiede attenzione.

Ho gestito le attività personali, come le riunioni e i resoconti di fine giornata, in Obsidian, un'applicazione per note in formato Markdown, e quelle di sviluppo in file Markdown conservati accanto al codice, così che anche gli assistenti IA potessero consultarle. Distinguevo le attività organizzative da quelle esecutive e, tra queste ultime, anticipavo quelle da cui dipendeva il lavoro successivo. Il mio calendario, gestito tramite Apple Calendar e Apple Reminders, restava comunque modificabile, perché una prova poteva richiedere più tempo del previsto e un confronto con il *tutor* poteva cambiare le priorità.

In un progetto esplorativo il rischio maggiore era dedicare molte ore a una modifica estesa basata su un'ipotesi sbagliata. Per questo adattavo la preparazione all'incertezza e all'estensione della modifica. Per una correzione circoscritta bastava una nota, mentre per una modifica estesa preparavo una breve progettazione, in cui esaminavo i moduli coinvolti, confrontavo le soluzioni, descrivevo quella scelta in pseudocodice e cercavo lacune e casi limite con un assistente IA. Tra il 14 e il 23 ottobre questo lavoro ha prodotto sei specifiche e cinque piani di implementazione, dedicati soprattutto alla sincronizzazione del carrello e alla riorganizzazione dell'applicazione.

Poiché le soluzioni cambiavano spesso, dovevo anche poter ricostruire quando e perché avessi preso una decisione. Ho quindi registrato le modifiche con messaggi secondo Conventional Commits$_G$, la convenzione che antepone alla descrizione il tipo di intervento, per esempio una funzionalità, una correzione o un riordino. Questa disciplina mi ha permesso di ricostruire l'evoluzione delle scelte e di collegare ogni prova al codice che la realizzava. I documenti di progetto fissavano le intenzioni, mentre la storia delle modifiche documentava ciò che avevo effettivamente realizzato.

3.1.3 Strumenti e assistenti di programmazione

La tabella 3.2 riassume gli strumenti con cui ho sviluppato, verificato e pubblicato ChatCart. Presento invece le librerie su cui si basa il prodotto insieme all'architettura, nella sezione 3.3.1, perché la loro scelta deriva dalle decisioni progettuali.

Attività	**Strumenti**	**Uso**
Sviluppo	Zed, Shopify CLI, mise	Modifica del codice, avvio sul negozio di prova e versioni degli strumenti fissate per il progetto.

Attività	Strumenti	Uso
Coordinamento	Slack, Meet, Obsidian	Resoconti, revisioni e organizzazione personale.
Versionamento	Git e GitHub	Storia delle modifiche e conservazione del codice.
Verifica	Vitest, happy-dom, TypeScript, ESLint	Prove automatiche, simulazione della pagina, controllo dei tipi e analisi statica.
Rilascio	ngrok, Vercel, Neon	Accesso all'applicazione locale e pubblicazione di applicazione e base di dati.
Assistenti IA	Claude Code, Codex, OpenCode	Studio, confronto di soluzioni, prototipi e preparazione dei casi di prova.

Tabella 3.2: Gli strumenti principali impiegati durante lo *stage*.

Durante lo sviluppo l'applicazione era in esecuzione sul mio computer, mentre Shopify deve poterla raggiungere attraverso Internet per inoltrarle le richieste provenienti dal negozio. Per collegare le due parti serviva un *tunnel*$_G$, un servizio che espone un indirizzo pubblico e inoltra le richieste alla macchina locale. Ho sostituito il *tunnel* predefinito della Shopify CLI con ngrok, perché il primo accumulava i frammenti della risposta dell'agente e li consegnava al *widget* tutti insieme, impedendo la visualizzazione progressiva della risposta. Il problema mi ha mostrato presto che dovevo controllare l'intero percorso della risposta, compresi gli intermediari esterni al mio codice.

Gli assistenti IA di programmazione mi sono serviti soprattutto nelle attività in cui il progetto esplorativo richiedeva di provare molte strade in poco tempo: trovare la documentazione pertinente, discutere alternative progettuali, realizzare piccoli prototipi, preparare i dati del negozio e scrivere prove a partire dai casi limite che definivo io. Il primo prototipo architetturale riguardava l'elenco condiviso degli strumenti dell'agente, diventato poi il contratto della sezione 3.3.2; poterlo provare rapidamente mi ha permesso di valutarne la fattibilità prima di incorporarlo nel progetto.

Ho inoltre collegato il negozio di sviluppo ad assistenti IA con accesso alle Admin API, che lo hanno popolato con prodotti, varianti, collezioni e pagine, e per provare l'interfaccia ho usato assistenti IA capaci di controllare il *browser*, a cui assegnavo percorsi d'uso diversi. Lo stile, le animazioni e la naturalezza delle interazioni li ho

valutati personalmente, e ho riletto e corretto i *test* che gli assistenti IA mi aiutavano a preparare. I prototipi servivano a scegliere tra alternative e a scoprire i problemi in anticipo, mentre l'implementazione definitiva di ChatCart l'ho scritta a mano. Nella sezione 4.2 valuto come questo uso degli assistenti IA abbia cambiato il mio modo di programmare.

3.2 Studio preliminare e analisi

3.2.1 L'ecosistema delle applicazioni Shopify

Prima dello *stage* conoscevo Shopify dal lato dei temi, su cui avevo lavorato per Dr Kampes, ma non avevo mai costruito un'applicazione. Per progettare ChatCart dovevo prima rispondere a tre domande: dove viene eseguito un programma che estende un negozio, come raggiunge le pagine viste dal cliente e con quali credenziali ne legge i dati. Ho dedicato la prima settimana soprattutto alla documentazione ufficiale e il 22 settembre ho creato il progetto dal modello ufficiale di Shopify per React Router[19], che risolve già autenticazione, gestione delle sessioni e integrazione nell'amministrazione. Partendo da quella base, ho tolto gli esempi e vi ho inserito il *widget*, concentrandomi sul prodotto anziché sull'infrastruttura dell'applicazione.

Shopify non esegue il codice delle applicazioni. Il *server* di un'applicazione è ospitato altrove, e la piattaforma ne conosce soltanto l'indirizzo, i permessi e le estensioni, descritti in una configurazione che pubblico con la Shopify CLI. Nel rilascio finale il *server* di ChatCart è ospitato da Vercel, un servizio che esegue applicazioni web senza richiedere la gestione di una macchina propria, mentre Shopify conserva la configurazione e i file dell'estensione. Tra i modelli di distribuzione ho scelto quello *custom*, riservato a un solo negozio[17], perché corrisponde al servizio su misura che Halue ipotizzava di offrire ai propri clienti.

Il secondo problema era portare il *widget* nelle pagine del negozio senza intervenire sul tema, che appartiene al commerciante e che il commerciante può aggiornare indipendentemente da ChatCart. Per questo ho usato una *theme app extension*$_{\text{G}}$, un'estensione che aggiunge elementi al tema senza modificarne i file[22]. Il suo blocco *app embed* compare su tutte le pagine quando il commerciante lo attiva nell'*editor*, carica i file del *widget* e gli consegna il contesto della pagina; disattivarlo rimuove l'inserimento senza lasciare tracce nel tema. Restano escluse le pagine di pagamento, che Shopify gestisce in un ambiente separato.

Il terzo problema riguardava l'accesso ai dati. Il commerciante apre l'applicazione dentro l'amministrazione del negozio, dove la *dashboard*$_{G}$, una pagina di riepilogo delle misure e delle conversazioni, occupa un riquadro incorporato. In quel momento Shopify attesta l'identità di chi la apre, e il *server* la scambia con un *token* di accesso$_{G}$, cioè una credenziale limitata ai permessi approvati all'installazione. Ho richiesto soltanto i permessi necessari a leggere i contenuti pubblici del negozio e a configurare il collegamento con il *widget*, così che ChatCart non possa modificare dati che non usa.

La figura 3.2 sintetizza questo *token exchange*[20]. Il flusso applica OAuth$_{G}$, lo standard che concede a un'applicazione un accesso limitato ai dati senza consegnarle la password dell'utente; l'identità arriva in un JWT, *JSON Web Token*, un dato firmato da Shopify che il *server* verifica prima di procedere. Nella base di dati conservo una sessione *offline*, legata al negozio anziché al commerciante, perché l'agente deve leggere il catalogo anche quando nessuno è collegato all'amministrazione.

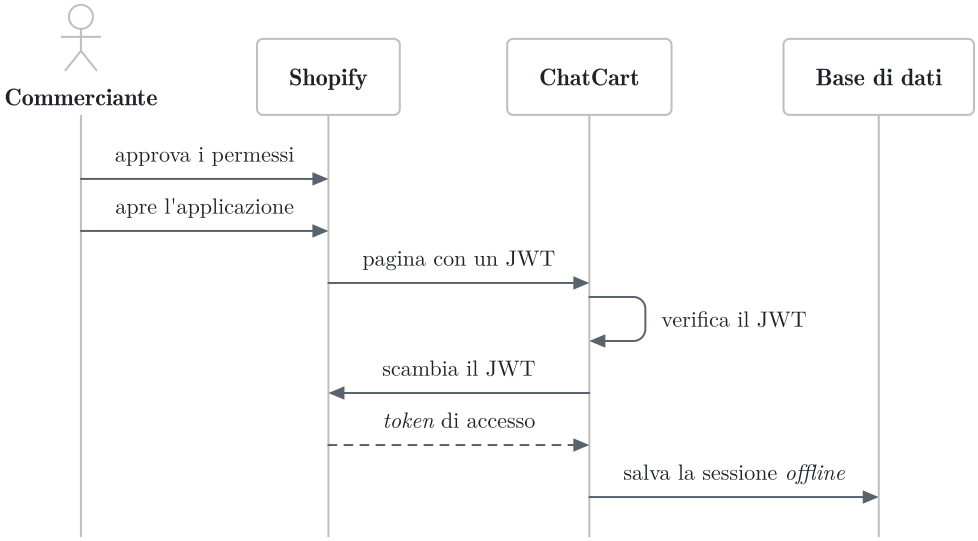


Figura 3.2: Lo scambio delle credenziali alla prima apertura dell'applicazione. Il *server* verifica il JWT e conserva la sessione del negozio; nelle aperture successive riusa la sessione valida.

Restava da collegare il *widget* al *server* senza esporre questa credenziale al *browser* del cliente. La soluzione è l'*app proxy*$_{G}$, un percorso nel dominio del negozio attraverso cui Shopify inoltra le richieste all'applicazione, aggiungendo una firma[21]. Il *widget* chiama quindi lo stesso dominio della pagina, e il *server* verifica la firma, riconosce il negozio e recupera la sessione con cui interrogare il catalogo; una richiesta con firma errata o senza una sessione valida non raggiunge l'agente.

Attraverso lo stesso *proxy* passa lo *streaming*$_{G}$, l'invio della risposta in frammenti mentre il modello la produce. Una risposta dell'agente può richiedere più letture del negozio prima di concludersi, e mostrarla progressivamente fa comparire le prime parole prima che la risposta sia completa e segnala al cliente che l'agente sta lavorando. Ho usato SSE$_{G}$, *Server-Sent Events*, il formato con cui un *server* trasmette una sequenza di eventi sulla stessa risposta HTTP, e ho verificato che i frammenti attraversassero il *proxy* senza essere trattenuti, sia nell'ambiente locale sia nel rilascio finale.

Infine dovevo scegliere con quali API leggere i dati e modificare il carrello (tabella 3.3). Il *server* legge prodotti, collezioni e pagine con le Storefront API, che restituiscono i contenuti pubblici del negozio nel paese e nella lingua del cliente[23]. Il *widget* modifica invece il carrello con la AJAX Cart API, la stessa usata dal tema, che opera sul carrello identificato dal *cookie* del cliente e può restituire nella stessa risposta le sezioni HTML da aggiornare[14]. Usare l'API del tema mi ha evitato di creare un secondo carrello da mantenere allineato con il primo.

API	**Funzione**	**Uso in ChatCart**
Storefront API	Catalogo, collezioni e pagine rivolte al cliente.	Il *server* legge i dati nel paese e nella lingua del cliente.
AJAX Cart API	Lettura e scrittura del carrello associato al *browser*.	Tema e *widget* lavorano sullo stesso carrello.
Section Rendering API	HTML aggiornato delle sezioni del tema.	Gli adattatori aggiornano l'interfaccia del tema dopo una scrittura della chat.
Admin API	Gestione e configurazione del negozio.	L'ho usata nelle prove di personalizzazione e per preparare i dati; il prodotto finale non la interroga.

Tabella 3.3: I punti di accesso ai dati e al carrello di Shopify.

Lo studio della piattaforma ha separato il lavoro nuovo da ciò che già sapevo. Per l'estensione e il carrello ho riusato Liquid e la conoscenza dei temi, mentre ho dovuto imparare l'autenticazione delle applicazioni, la gestione delle sessioni e il collegamento tra negozio e *server*, conoscenze che ho poi usato anche per dare agli assistenti IA l'accesso al negozio di prova.

3.2.2 Un'analisi dei requisiti iterativa

Il piano fissava obiettivi chiari ma nessuna specifica delle funzioni, perché Halue voleva esplorare il valore di un assistente con interfaccia generativa prima di stabilirne la forma. Un'analisi dei requisiti completa svolta all'inizio avrebbe quindi fissato le scelte proprio quando avevo meno informazioni, e io e il *tutor* abbiamo preferito un'analisi iterativa, in cui le prove sul prototipo correggevano via via ciò che il prodotto doveva fare.

Questo modo di procedere corrisponde al ciclo costruire, misurare e imparare che Eric Ries propone per le condizioni d'incertezza[24]. Nel mio caso la misura aveva però un limite, perché io e il *tutor* provavamo il prototipo sul negozio di sviluppo, senza osservare clienti reali. Le revisioni ci aiutavano a scartare le soluzioni troppo complesse o poco convincenti, ma non potevano dimostrare un beneficio sulle vendite.

Per non perdere il legame con il piano mentre le soluzioni cambiavano, ho distinto tre livelli di descrizione. Gli obiettivi del piano fissavano lo scopo, i casi d'uso$_G$ descrivevano le interazioni con cui un attore, cioè un ruolo esterno al sistema, raggiunge un risultato, e i requisiti precisavano le proprietà da verificare. OB4, per esempio, richiede componenti interattivi; il caso d'uso UC6 descrive l'aggiunta di un prodotto al carrello dalla chat; RFOB6 richiede che il cliente possa scegliere la variante e aggiungerla al carrello.

Ho rappresentato i sedici casi d'uso finali in due diagrammi UML, uno per ciascun attore primario, cioè per chi usa ChatCart per uno scopo proprio. Il cliente conversa e prepara l'acquisto nel *widget* (figura 3.3), mentre il commerciante attiva l'assistente nel tema e ne osserva i risultati nella *dashboard* (figura 3.4). Shopify e il fornitore dei modelli sono invece attori secondari, ai quali ChatCart si rivolge per portare a termine un caso d'uso; l'agente non compare tra gli attori, perché fa parte del sistema.

Nel diagramma del commerciante la relazione «*extend*» indica un caso d'uso facoltativo, che si innesta in un altro in un punto preciso: il commerciante legge una conversazione (UC15) o ne avvia l'analisi (UC16) soltanto dall'elenco delle conversazioni (UC14).

La distinzione si è rivelata utile, perché i casi d'uso sono rimasti relativamente stabili mentre cambiavano le soluzioni che li realizzavano. Il carrello, per esempio, funzionava già nella prima versione, ma non si integrava in tempo reale con il tema. L'ho

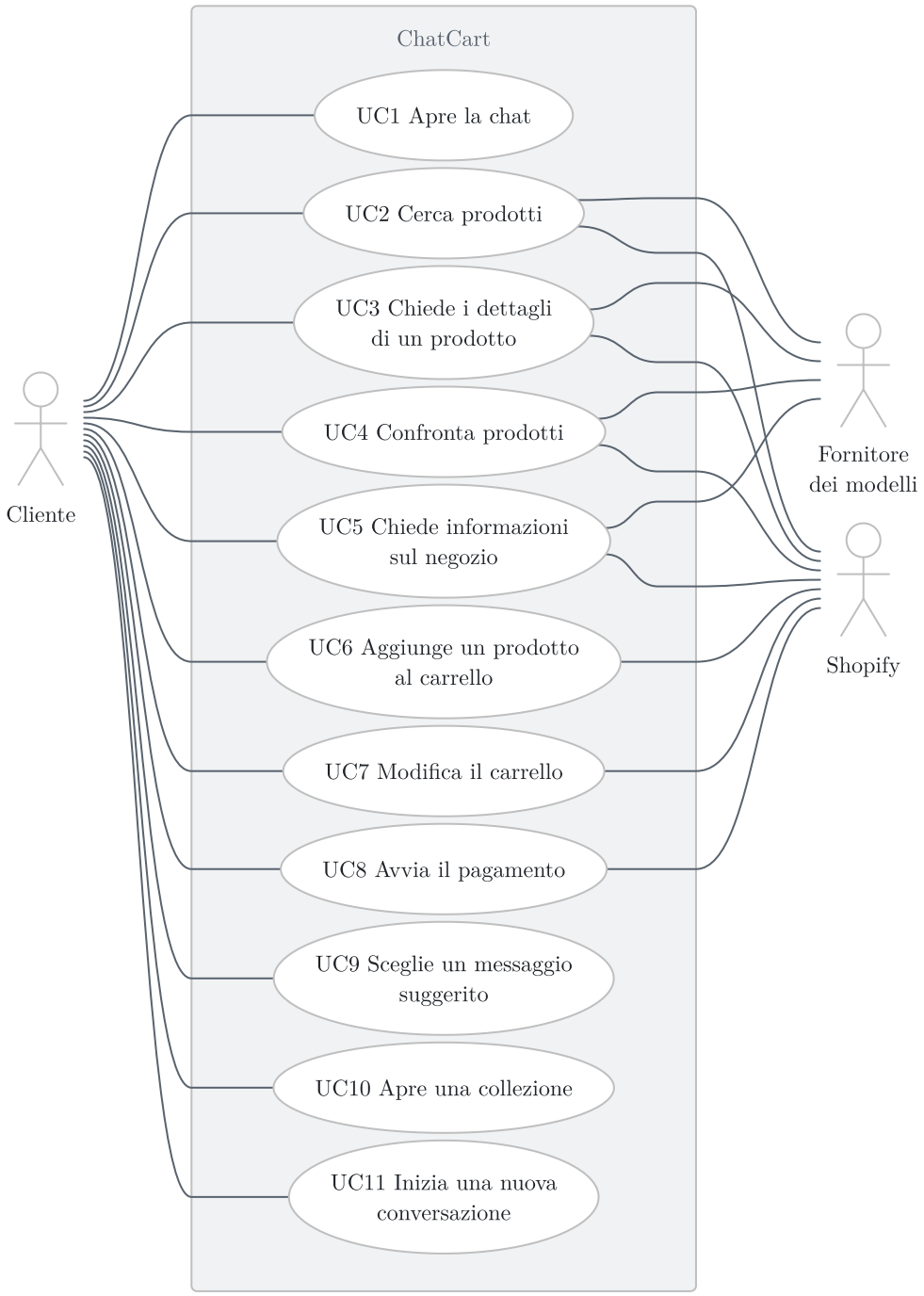


Figura 3.3: I casi d'uso del cliente. Shopify partecipa quando ChatCart legge catalogo, pagine o carrello e quando il pagamento prosegue nella sua procedura; il fornitore dei modelli partecipa alle risposte dell'agente. L'aggiunta al carrello parte da una scheda prodotto, che può comparire nelle risposte di casi d'uso diversi, e per questo è collegata direttamente al cliente.

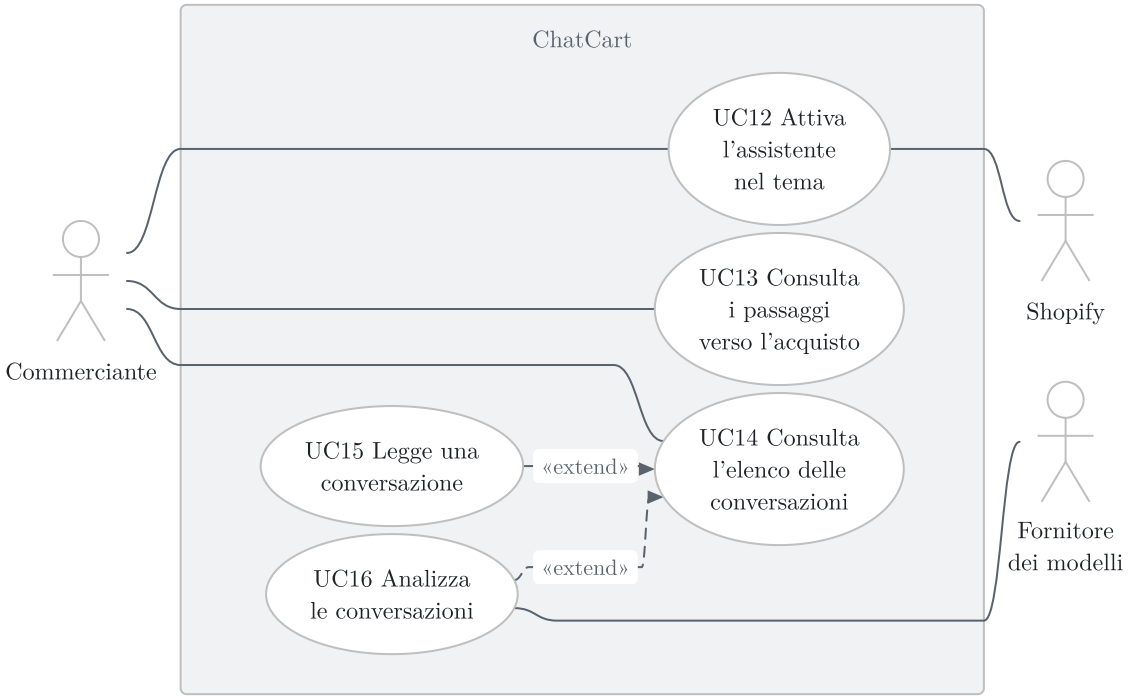


Figura 3.4: I casi d'uso del commerciante. L'attivazione avviene nell'*editor* del tema di Shopify, gli altri casi d'uso nella *dashboard*; il fornitore dei modelli partecipa all'analisi, affidata al modello Jev (sezione 3.4.3).

tolto il 15 ottobre e, dopo il confronto con il *tutor*, l'ho implementato nuovamente, senza che cambiassero il caso d'uso e l'obiettivo a cui rispondeva. La seconda implementazione è descritta nella sezione 3.4.2.

Il 9 ottobre ho raccolto il comportamento atteso dell'agente in una specifica di lavoro, accompagnata da scenari di prova. Alcune richieste descrivevano capacità più estese di quelle necessarie a dimostrare il valore del prototipo, come leggere altri contenuti del negozio, proporre raccomandazioni e gestire ricerche più articolate, e riducendo il perimetro le ho spostate tra gli sviluppi futuri. La ricognizione finale conta 45 requisiti; la tabella 3.4 ne presenta una selezione di 20, sufficiente a descrivere il nucleo del prodotto, le scelte architetturali e i limiti più significativi.

Ogni identificativo è formato dalla lettera R, dal tipo, dalla priorità e da un numero progressivo. Il tipo è F per un requisito funzionale, Q per una proprietà di qualità e V per un vincolo; la priorità è OB per obbligatorio, DE per desiderabile e OP per opzionale. La selezione conserva i numeri della ricognizione completa, e per questo la numerazione non è continua; nella colonna dei casi d'uso riporto gli identificativi delle due figure precedenti. Ho considerato obbligatorio il carrello nella chat, che deriva da OB4, perché porta l'interfaccia generativa fino all'acquisto; le voci desiderabili descrivono invece funzioni migliorative, che io e il *tutor* potevamo escludere

dal perimetro senza compromettere gli obiettivi obbligatori. Valuto la copertura di questa selezione nella sezione 3.7.2.

Requisito	Proprietà richiesta	Caso d'uso	Obiettivo
RFOB2	Il cliente riceve la risposta in *streaming*, con l'indicazione dell'attività dell'agente.	UC2–UC5	OB1, OB2
RFOB3	La conversazione riprende quando il cliente torna o cambia pagina.	UC1	nessuno
RFOB5	Ogni messaggio include paese, lingua e pagina corrente.	UC2–UC5	OB2, DE1
RFOB6	Il cliente sceglie la variante e aggiunge il prodotto dalla scheda.	UC6	OB4
RFOB9	Chat e tema mantengono il carrello allineato nei due sensi.	UC6, UC7	OB4
RFOB10	L'agente mostra da uno a tre prodotti con immagine, prezzo e punti di forza.	UC2, UC4	OB3, OB4
RFOB11	Le risposte sul negozio hanno una scheda con testo e fonte.	UC5	OB3, OB4
RFOB12	L'agente consulta il negozio entro dieci passi e conclude con al più un componente.	UC2–UC5	OB2
RFOB13	La risposta assume una delle tre forme: testo, testo e prodotti, scheda sul negozio.	UC2–UC5	OB3
RFOB15	Prima di citare prezzo o disponibilità, l'agente rilegge il prodotto e la variante.	UC2–UC4	OB3
RFOB22	La *dashboard* mostra passaggi verso l'acquisto e relativi tassi a trenta giorni.	UC13	OB5
RFOB24	Il commerciante rilegge messaggi, componenti e azioni in un'unica cronologia.	UC15	OB5
RFDE3	Testi, prezzi e collegamenti rispettano lingua e mercato del cliente.	UC1–UC10	DE1
RFDE5	Un componente mostra lo stato di un ordine.	nessuno	DE2
RFDE7	Un modello assegna punteggio ed etichette alle conversazioni concluse.	UC16	OB5
RFDE8	Le metriche confrontano sessioni con e senza assistente e valore medio dell'ordine.	UC13	OB5
RQOB2	L'agente afferma fatti letti nel negozio durante la risposta e dichiara i dati mancanti.	UC2–UC5	OB3
RQDE1	Gli errori del modello o degli strumenti lasciano al cliente un avviso.	UC1–UC11	nessuno

Requisito	Proprietà richiesta	Caso d'uso	Obiettivo
RVOB2	Ogni componente ha una sola definizione nell'elenco condiviso tra agente e *widget*.	UC2, UC4, UC5, UC10	OB4
RVOP1	Il *widget* funziona anche su uno *storefront* Hydrogen.	nessuno	FA1

Tabella 3.4: I venti requisiti principali selezionati dalla ricognizione finale. Gli identificativi mantengono la numerazione originale.

3.3 Architettura del sistema

3.3.1 Visione d'insieme

Lo studio della piattaforma e i requisiti imponevano all'architettura tre condizioni: le credenziali del negozio non dovevano raggiungere il *browser*, l'agente doveva affermare soltanto fatti letti nel negozio (RQOB2) e agente e *widget* dovevano interpretare ogni componente nello stesso modo (RVOB2).

Il piano di lavoro prevedeva due livelli, l'*app layer* nel negozio e l'*agent layer* sul *server*. Li ho mantenuti perché corrispondono a due ambienti con un diverso grado di fiducia. Il *browser* del cliente esegue codice che chiunque può ispezionare, mentre il *server* custodisce le credenziali del negozio e l'accesso ai modelli. Vi ho aggiunto la persistenza delle conversazioni e l'interfaccia del commerciante, richieste da OB5. La figura 3.5 mostra gli ambienti e i collegamenti che li uniscono; il carrello resta del negozio, e il *widget* vi opera direttamente senza passare dall'agente.

Il *server* è una sola applicazione React Router, in cui le rotte autenticano le richieste e chiamano i moduli dell'agente, delle conversazioni e della *dashboard*, separati per responsabilità così che una modifica alla lettura delle conversazioni non tocchi il ciclo agentico. Il *widget* è un'applicazione React distinta, compilata con Vite e caricata dall'estensione del tema. Accanto a esso il tema resta un programma indipendente, che legge e modifica lo stesso carrello senza condividere codice con il *widget*. Mantenere allineate le due interfacce è il problema centrale della sezione 3.4.2.

Poiché *server* e *widget* sono programmi distinti, il rischio principale era che interpretassero in modo diverso gli stessi dati. Per questo ho usato TypeScript in entrambi e ho raccolto nella cartella `shared/` le definizioni comuni (strumenti, messaggi, pro-

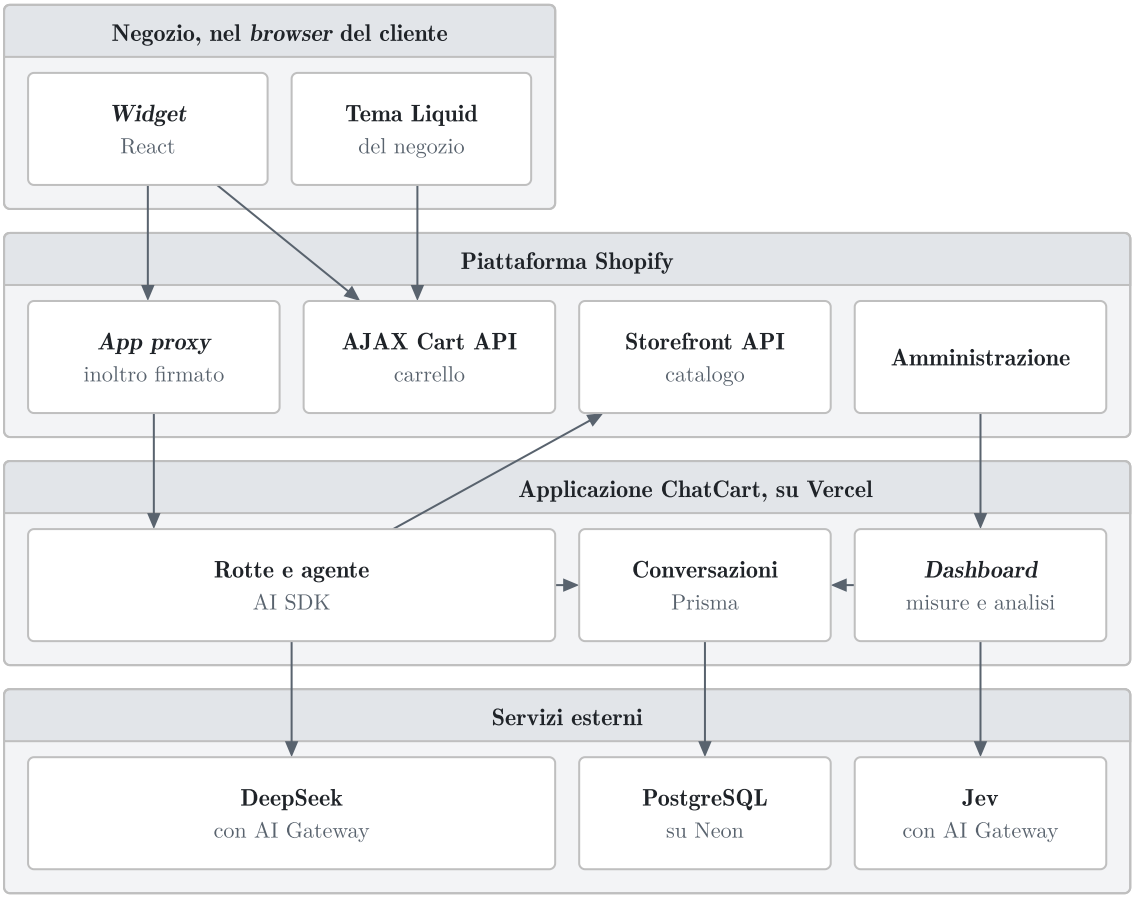


Figura 3.5: Gli ambienti e i collegamenti del prototipo. Il *widget* raggiunge il *server* solo attraverso l'*app proxy* e condivide con il tema il carrello della AJAX Cart API. La *dashboard* appartiene all'applicazione, ma Shopify la mostra al commerciante nell'amministrazione; i due modelli si raggiungono attraverso l'AI Gateway di Vercel.

dotti mostrati, contesto del cliente ed eventi), così che il compilatore verifichi il contratto tra i due programmi. L'AI SDK gestisce il rapporto con i modelli e il ciclo agentico, Zod descrive e valida gli schemi dei dati, Prisma collega l'applicazione alla base di dati PostgreSQL.

Il codice segue lo stile funzionale già adottato da React, React Router e AI SDK, e anche i *design pattern* hanno preso questa forma, con interfacce espresse come tipi TypeScript, adattatori come oggetti e la creazione degli strumenti come funzione. Seguendo un'indicazione del *tutor*, ho introdotto le astrazioni solo quando un problema concreto le rendeva utili, perché in un prototipo che cambiava ogni settimana avrebbero fissato una suddivisione dei moduli che le prove successive potevano smentire.

La tabella 3.5 raccoglie i *pattern* introdotti in questo modo, ciascuno con il problema concreto che risolveva. Il contratto condiviso e la sincronizzazione del carrello richie-

dono un approfondimento, perché sono le parti in cui l'organizzazione del codice incide di più sull'estensibilità del prodotto.

Pattern	**Idea e applicazione nel progetto**
registry$_{\text{G}}$	Un elenco indicizzato concentra le informazioni su ogni voce. Gli elenchi degli strumenti e degli eventi obbligano tutte le parti a riconoscere gli stessi nomi (sezione 3.3.2, sezione 3.4.3).
adapter$_{\text{G}}$ e *strategy*$_{\text{G}}$	Un'interfaccia comune adatta componenti diversi; l'implementazione scelta determina l'algoritmo. Gli adattatori dei temi aggiornano il carrello senza esporre le differenze al codice della chat (sezione 3.4.2).
decorator$_{\text{G}}$	Un involucro aggiunge un comportamento conservando l'interfaccia originale. Osservo le richieste del tema senza cambiarne il risultato (sezione 3.4.2).
observer$_{\text{G}}$	Un soggetto notifica i cambiamenti ai propri iscritti. Tutti i componenti del carrello ricevono lo stesso aggiornamento (sezione 3.4.2).
factory$_{\text{G}}$	Una funzione costruisce gli oggetti da usare. `createTools` crea gli strumenti con il negozio e il mercato della richiesta.
dependency injection$_{\text{G}}$	Una funzione riceve le dipendenze dall'esterno. Nei *test* passo modelli e accessi alle API controllati (sezione 3.6).
repository$_{\text{G}}$	Un modulo nasconde il modo di conservare i dati. Salvataggio e lettura delle conversazioni attraversano la stessa interfaccia (sezione 3.4.3).

Tabella 3.5: I *pattern* adottati e il problema concreto che risolvono.

La *factory* degli strumenti nasce da un problema di isolamento. Ogni strumento deve usare il negozio autenticato e il paese e la lingua del cliente corrente. Affidarli agli argomenti scelti dal modello gli avrebbe permesso di indicare un mercato sbagliato, e conservarli in variabili condivise tra le richieste avrebbe mescolato i dati di richieste concorrenti. `createTools` li riceve invece dal programma a ogni richiesta (figura 3.6). Con la *dependency injection* anche il modello arriva come parametro, e le prove lo sostituiscono con uno simulato; il *repository* raccoglie invece in un unico modulo gli accessi alle conversazioni.

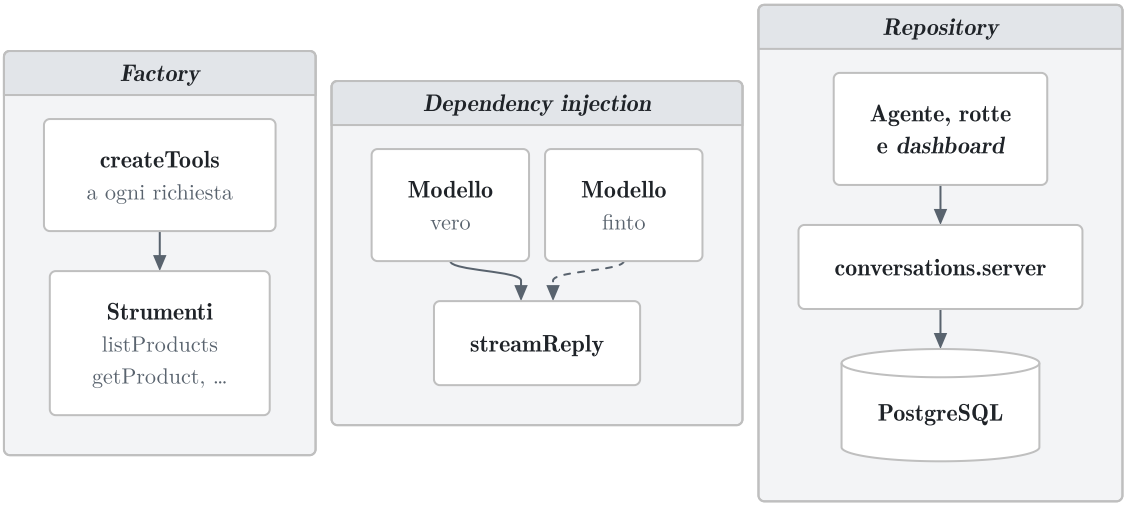


Figura 3.6: Tre *pattern* del *server*. Nelle prove `streamReply` riceve un modello simulato, che restituisce risposte prestabilite. Le misure della *dashboard* interrogano invece Prisma direttamente: il *funnel* legge in un'unica interrogazione tutte le conversazioni degli ultimi trenta giorni, con il numero dei messaggi del cliente e i tipi di evento, mentre il *repository* opera su una conversazione alla volta.

3.3.2 Il contratto tra agente e *widget*

Nel modello di interfaccia generativa scelto per ChatCart il modello linguistico seleziona componenti predefiniti e li popola con i dati del negozio (sezione 2.2). Agente e *widget* devono quindi concordare sul nome di ogni componente, sugli argomenti con cui richiederlo e sui dati necessari a mostrarlo. Poiché i due programmi si compilano e si pubblicano separatamente, un campo cambiato soltanto sul *server* potrebbe lasciare nel *browser* un componente vuoto senza che alcun errore lo segnali. Per questo ho progettato il contratto in modo che queste divergenze emergano durante la compilazione, prima del rilascio.

La definizione comune è un elenco in `shared/tools.ts`, che per ciascuno degli otto strumenti contiene nome, categoria, schema in ingresso e, quando serve, schema in uscita. Le due categorie corrispondono ai due modi in cui il modello usa il negozio:

- i *data tool*$_{\text{G}}$ leggono i dati per il modello: elenco e dettaglio dei prodotti, collezioni, elenco e testo delle pagine del negozio;
- i *display tool*$_{\text{G}}$ restituiscono i dati dei componenti visibili: schede prodotto, scheda informativa e collegamenti alle collezioni. La loro invocazione conclude la risposta.

Il risultato dei *data tool* resta interno al ragionamento del modello, e il *widget* ne mostra soltanto un'indicazione di avanzamento, come «Cerco i prodotti».

Per evitare due descrizioni dello stesso dato da tenere allineate, ho affidato a un unico schema Zod sia la validazione sia il controllo dei tipi (figura 3.7). Durante l'esecuzione l'AI SDK presenta lo schema in ingresso al modello come descrizione degli argomenti e lo usa per verificarli prima di eseguire lo strumento; durante la compilazione TypeScript ne ricava un tipo statico con `z.infer`.

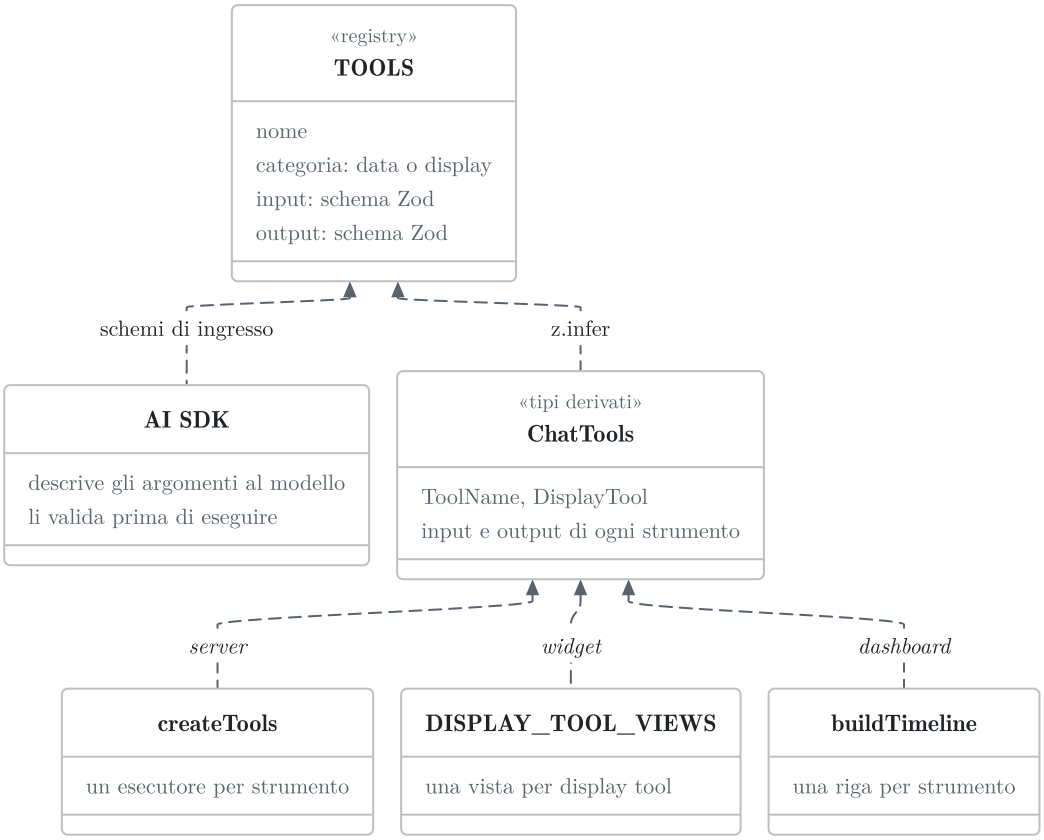


Figura 3.7: Il *registry* `TOOLS` e chi ne dipende. L'AI SDK usa gli schemi di ingresso durante l'esecuzione; *server*, *widget* e *dashboard* dipendono dai tipi derivati, che legano ogni risultato al suo esecutore, alla sua vista e alla sua riga nella cronologia.

Per il risultato dei *display tool* lo schema serve soprattutto a ricavare il tipo atteso dalle viste, perché questi dati li costruisce il mio codice a partire dal negozio e il controllo statico di TypeScript basta a verificarne la coerenza con i componenti. Gli argomenti scelti dal modello richiedono invece anche la validazione durante l'esecuzione, perché il modello li genera soltanto mentre il programma è in esecuzione, fuori dalla portata del controllo dei tipi svolto in compilazione. Il contratto garantisce quindi la forma dei dati, senza dimostrare che il modello abbia scelto il prodotto giusto o scritto una spiegazione corretta.

Il codice 3.1 mostra un caso ridotto. Lo schema della scheda informativa stabilisce gli argomenti e il risultato, e il tipo della vista usa direttamente lo schema in uscita.

Il testo della risposta arriva dal modello, mentre titolo e indirizzo della fonte arrivano dalla pagina del negozio letta dal mio codice.

```
const faqTool = {
  input: z.object({
    handle: z.string(), answer: z.string(),
  }),
  output: z.object({
    title: z.string(), answer: z.string(), url: z.string(),
  }),
};
type FaqOutput = z.infer<typeof faqTool.output>;
type FaqView = (props: { output: FaqOutput }) => ReactNode;
```

Codice sorgente 3.1: La definizione di un *display tool* e il tipo dei dati della sua vista, semplificati. L'elenco completo comprende anche la categoria e gli altri strumenti; z.infer ricava il tipo dallo schema.

Dallo stesso *registry* ricavo anche le parti del programma che dipendono dalla categoria degli strumenti, come la condizione di arresto dell'agente al primo *display tool*; se a una voce mancano l'esecutore, la vista o la riga della cronologia, il compilatore lo segnala.

Il vantaggio di questo meccanismo si è visto il 28 ottobre, quando ho aggiunto i collegamenti alle collezioni. Ho scritto la definizione condivisa, l'esecutore e le due rappresentazioni, e il compilatore ha segnalato ogni punto che dipendeva dal nuovo strumento e non lo gestiva ancora. Restavano da aggiornare le istruzioni del modello e la misura dei clic, che il compilatore non può controllare perché dipendono dal significato della funzionalità.

Il contratto è arrivato a questa forma in due passi. Il 5 ottobre ho ricavato i tipi comuni dal codice dell'agente, rendendo però la parte condivisa dipendente dal *server*, e il 15 ottobre ho spostato la definizione in un elenco Zod indipendente.

Resta aperta la compatibilità tra rilasci, perché Shopify pubblica il *widget* separatamente dal *server*. Un servizio esteso a più negozi richiederebbe di versionare il contratto, mentre già oggi un componente non più riconosciuto non interrompe la riapertura di una vecchia conversazione. Definito che cosa agente e *widget* si scambiano, restava da realizzare ciascuna delle due parti.

3.4 Realizzazione

3.4.1 L'agente conversazionale

Un modello linguistico lasciato libero può inventare prodotti e prezzi, consultare il catalogo senza concludere o mostrare componenti incoerenti con il testo. Realizzare l'agente, che risponde agli obiettivi OB2 e OB3, ha significato soprattutto decidere quali di questi rischi affidare alle istruzioni e quali al codice. Ho costruito l'agente con l'AI SDK di Vercel[12], che gestisce il ciclo di richieste al modello ed esecuzioni degli strumenti. A ogni messaggio del cliente l'agente ricostruisce la storia da inviare al modello, prepara gli strumenti per il negozio e avvia il modello (figura 3.8).

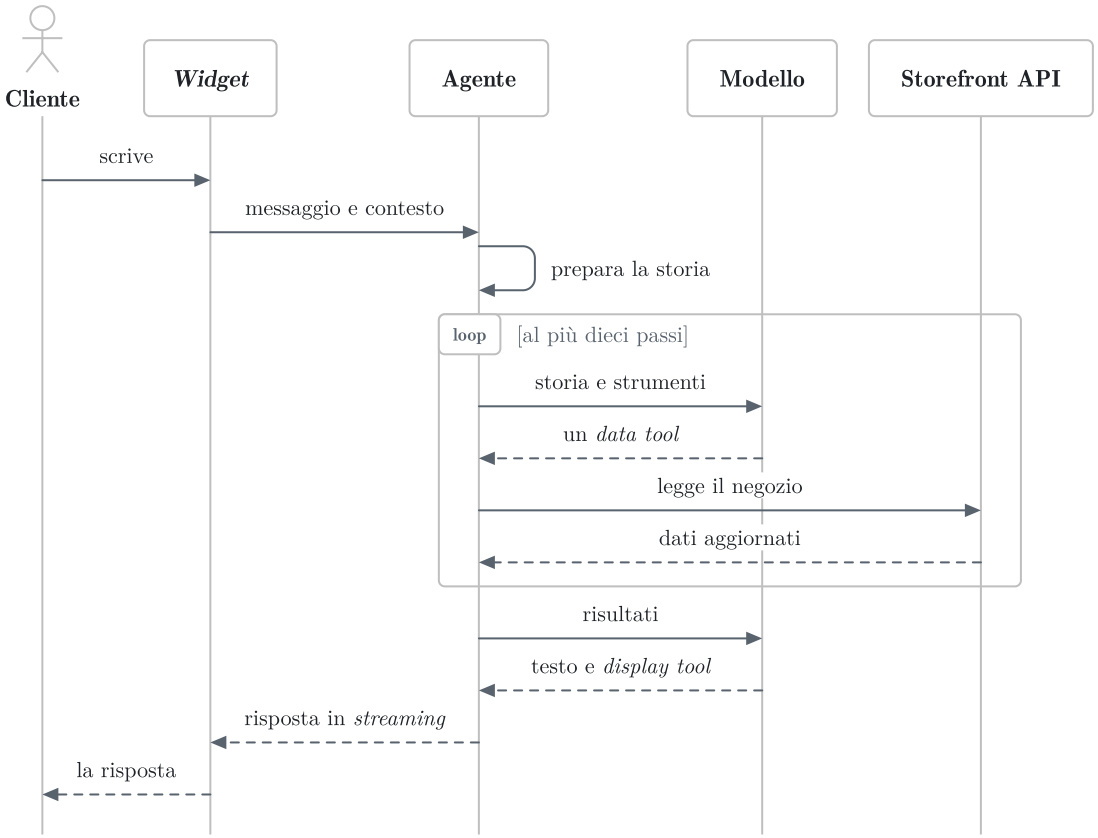


Figura 3.8: Una risposta con lettura dei dati e componente finale. Il ciclo ripete le letture finché il modello chiede dati; il *server* inoltra i frammenti al *widget* man mano che li riceve. Per leggibilità il diagramma raccoglie nell'agente il *proxy* e la persistenza, e omette i messaggi suggeriti.

Il primo vincolo riguarda la durata del ciclo, che termina al primo *display tool*, così che il componente resti l'ultimo elemento visibile, oppure dopo dieci passi, perché il modello non consulti il catalogo senza concludere; entro questi limiti sceglie liberamente il percorso.

Il secondo vincolo riguarda l'affidabilità dei dati mostrati, richiesta da RQOB2. Ho separato la selezione dei prodotti dalla costruzione delle schede. Il modello indica gli identificativi e scrive i punti di forza, mentre il codice rilegge il negozio, ricava titolo, immagine, prezzo e varianti e scarta gli identificativi inesistenti. Un prodotto inventato dal modello non può quindi diventare una scheda acquistabile, e i dati economici del componente provengono sempre da Shopify; la pertinenza del consiglio e la correttezza delle frasi restano invece responsabilità del modello.

Lo stesso criterio guida la divisione delle regole tra istruzioni e codice (tabella 3.6). Le istruzioni di sistema, o $prompt_{\mathrm{G}}$ di sistema, precedono la conversazione e descrivono il ruolo dell'assistente e il comportamento richiesto, e vi colloco le regole che richiedono un giudizio sul dialogo. Nel codice fisso invece i vincoli strutturali che devono valere a ogni esecuzione, perché il modello può disattendere un'istruzione, mentre una condizione verificata dal programma vale sempre.

Controllo	**Esempi**	**Motivazione**
Istruzioni	Chiarire una richiesta vaga, proporre prodotti pertinenti, dichiarare i dati mancanti, suggerire collezioni utili.	Il comportamento dipende dal significato della conversazione.
Ciclo agentico	Massimo dieci passi; conclusione al primo *display tool*.	Il codice limita il percorso e la posizione del componente.
Schemi Zod	Da una a tre schede prodotto; tre punti di forza per scheda.	Gli argomenti devono rispettare la forma ammessa prima dell'esecuzione.
Lettura del negozio	Titolo, prezzo, disponibilità e immagine ricavati dalle API.	Il modello sceglie il prodotto, il programma costruisce la scheda da dati effettivi.

Tabella 3.6: La divisione delle regole tra istruzioni al modello e codice.

I collegamenti alle collezioni restano nelle istruzioni, perché imporli dopo ogni prodotto li renderebbe ripetitivi; il limite sul numero di schede è invece uno schema che il programma verifica senza interpretare il dialogo.

Ogni messaggio porta con sé il paese, la lingua e la pagina corrente del cliente, che entrano nelle interrogazioni delle Storefront API e aiutano a interpretare riferimenti come «questo prodotto». Alla prima risposta, se il cliente non si trova sulla pagina di un prodotto, di una collezione o di una ricerca, il codice impone come primo passo la lettura del catalogo, perché l'agente proponga scelte che il negozio offre davvero.

Anche la storia della conversazione ha richiesto una scelta, perché modello, *widget* e *dashboard* ne hanno bisogni diversi. Conservo i messaggi nella forma completa prodotta dall'AI SDK, comprese le parti degli strumenti, e ne ricavo tre rappresentazioni (tabella 3.7). Il modello riceve gli ultimi venti messaggi a partire da un messaggio del cliente, senza i risultati dei *data tool* e senza i dati delle varianti che servono solo al *widget*. Prezzi e scorte possono cambiare, così come il mercato del cliente, e togliere le letture precedenti obbliga il modello a consultare di nuovo il negozio.

Destinatario	**Rappresentazione**	**Scopo**
Modello	Storia ridotta, senza risultati dei *data tool* né dettagli necessari solo all'interfaccia.	Conservare il dialogo senza riusare dati commerciali superati.
Widget	Messaggi e risultati dei componenti.	Riaprire la conversazione con schede e varianti utilizzabili.
Dashboard	Messaggi e componenti trasformati in righe, insieme agli eventi del cliente.	Capire che cosa l'assistente ha proposto e come il cliente ha reagito.

Tabella 3.7: Tre usi della stessa conversazione conservata nella base di dati.

Alla risposta aggiungo i messaggi suggeriti con una seconda chiamata al modello, che produce un *output* strutturato$_{\text{G}}$, cioè dati conformi a uno schema anziché testo libero. Se la chiamata fallisce, la risposta resta disponibile senza suggerimenti: ho preferito rinunciare a un elemento accessorio piuttosto che ritardare la risposta.

Come modello ho usato DeepSeek v4.1 Flash fin dal primo sviluppo, cercando un rapporto tra capacità e costo adatto a un prototipo, senza un confronto sistematico con altri modelli. Ho invece provato diversi livelli di ragionamento$_{\text{G}}$, cioè quantità diverse di elaborazione intermedia prima della risposta, e ho scelto quello medio, perché il livello ridotto, pur più rapido, a volte ometteva i componenti, che sono il centro dell'esperienza proposta. Sono osservazioni qualitative, senza una misura sistematica della latenza.

L'attesa maggiore rendeva ancora più importante lo *streaming*. Nel primo rilascio di prova, su un *server* virtuale gestito da me, il *proxy* che riceveva le richieste tratteneva la risposta, e nel rilascio definitivo ho controllato che arrivasse progressivamente fino alla chat, dove il *widget* la trasforma in interfaccia.

3.4.2 Il *widget* e l'integrazione con il tema del negozio

Il *widget* realizza l'*app layer* e gli obiettivi OB1, OB4 e DE1, e deve funzionare dentro il tema del negozio, un ambiente che non controllo, con fogli di stile e *script* propri. Il blocco Liquid dell'estensione prepara un elemento radice con lingua, paese e contesto della pagina e carica i file JavaScript e CSS compilati da Vite, così che il *widget* conosca il contesto senza una richiesta aggiuntiva; nomi di classe propri separano il suo stile da quello del tema.

I componenti della conversazione seguono il contratto della sezione 3.3.2 (schede prodotto, scheda informativa e collegamenti alle collezioni), e a essi si aggiungono i suggerimenti e il pannello del carrello. Ne mostro la disposizione e il percorso d'uso nella sezione 3.7.1; qui approfondisco il problema che ha richiesto più progettazione, cioè far condividere a chat e tema lo stesso carrello e lo stesso stato visibile.

Ho stabilito che il cliente acquisti soltanto attraverso i pulsanti dei componenti. Nella versione finale l'agente non vede né modifica il carrello. Consiglia i prodotti, mentre il cliente sceglie la variante e conferma ogni scrittura, così che una decisione del modello linguistico non produca da sola un effetto sull'acquisto. Il *widget* assume quindi più responsabilità di quanto prevedesse il piano iniziale, che lo presentava soprattutto come livello di visualizzazione, perché legge e scrive da sé il carrello di Shopify. Valuto questo scostamento nella sezione 4.1.

La AJAX Cart API conserva già un solo carrello per il cliente, ma non aggiorna tutte le interfacce che lo mostrano. Dopo una scrittura della chat l'icona e il pannello del tema possono mostrare ancora il contenuto precedente, e lo stesso accade alla chat dopo una scrittura del tema. Il problema riguarda quindi le interfacce che mostrano il carrello, mentre il dato resta unico. La figura 3.9 mostra i moduli che realizzano i due versi dell'aggiornamento e i *pattern* che vi applico.

La prima versione del carrello usava regole legate al tema di prova, e per questo l'ho implementata nuovamente (sezione 3.2.2). Il *tutor* ha proposto l'*adapter*, e io ho scelto di combinare due adattatori dedicati con un ripiego generico; ho scartato un secondo carrello sulla Storefront API, che avrebbe aggiunto un altro dato da sincronizzare con quello del negozio.

La prima direzione parte dalla chat. Quando il cliente aggiunge o modifica un articolo, il *widget* chiede alla AJAX Cart API anche le sezioni HTML necessarie ad aggiornare il tema, emette l'evento standard introdotto da Shopify per le modifiche al carrello[13] e affida le differenze all'adattatore scelto. L'evento parte anche per

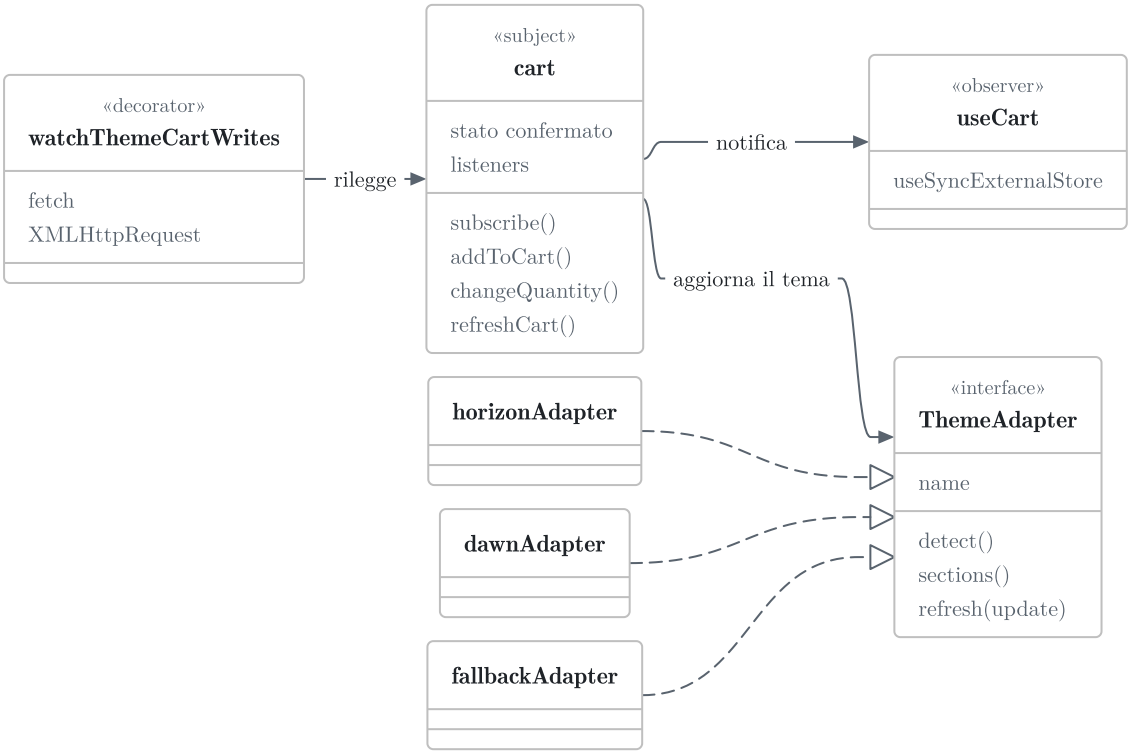


Figura 3.9: I moduli del carrello nel *widget*, con il ruolo di ciascuno nei *pattern*. Le linee tratteggiate indicano le tre implementazioni dell'interfaccia degli adattatori; quella di ripiego riceve gli elementi del carrello indicati dal commerciante.

i temi che non lo ascoltano, perché fa parte del contratto della piattaforma. La tabella 3.8 confronta le tre strategie.

Tema	Aggiornamento	Compromesso
Horizon	Fornisce le sezioni all'evento standard; il tema le usa per ricostruire il carrello.	Sfrutta l'integrazione prevista dai temi più recenti.
Dawn	Aggiorna icona e riepilogo e usa il meccanismo di notifica della famiglia Dawn per il pannello laterale.	Richiede un adattamento alle convenzioni del tema.
Ripiego	Rilegge la pagina e copia gli elementi del carrello indicati dal commerciante.	Funziona senza un adattatore specifico, con configurazione e una lettura aggiuntiva.

Tabella 3.8: Le strategie con cui ChatCart aggiorna il tema dopo una scrittura della chat.

Il codice del carrello ignora quale adattatore stia usando, perché quello selezionato è la *strategy* dell'aggiornamento. Lo scelgo alla prima scrittura, quando gli *script* del tema hanno avuto il tempo di inizializzarsi, riconoscendo elementi e funzioni

caratteristici anziché il solo nome del tema, così da supportare anche i temi derivati dalle famiglie note.

Ho tolto l'astrazione quando avevo un solo comportamento e l'ho reintrodotta quando sono diventati tre. Oggi aggiungere il supporto a un tema significa fornire un'altra implementazione dell'interfaccia. Gli errori degli adattatori restano isolati, perché un aggiornamento visivo fallito non deve annullare una scrittura già confermata da Shopify.

La direzione opposta richiede di accorgersi delle azioni del tema, e per questo applico un *decorator* alle funzioni di rete della pagina. Il *decorator* conserva il comportamento originale di `fetch` e di `XMLHttpRequest` e, dopo una scrittura del tema riuscita, rilegge il carrello. Le richieste del *widget* usano la funzione originale, così da non essere scambiate per scritture del tema, e quando la scheda del *browser* torna visibile aggiorno di nuovo lo stato per cogliere le modifiche fatte in un'altra scheda.

Resta da distribuire lo stato a tutti i componenti della chat. All'inizio ogni scheda e ogni pannello conservava una propria copia del carrello, che poteva divergere dalle altre; oggi un unico modulo contiene l'ultimo carrello confermato, l'operazione in corso e l'eventuale errore, e notifica ogni cambiamento ai componenti React iscritti con `useSyncExternalStore`, secondo il *pattern observer*.

La chat aggiorna il proprio stato e il tema soltanto dopo la risposta di Shopify, ed esegue le scritture una alla volta. Se Shopify rifiuta la quantità richiesta, mostro l'errore e rileggo il carrello, perché il negozio potrebbe aver accettato una parte dell'operazione. Queste regole privilegiano la coerenza tra le due interfacce rispetto a un aggiornamento ottimistico, che mostra subito il risultato atteso e lo corregge se Shopify lo rifiuta.

Per la localizzazione, infine, la direttiva `@inContext` porta paese e lingua in ogni lettura della Storefront API e Shopify ricava la valuta dal paese, così che prezzi e collegamenti restino nel mercato del cliente fino al pagamento; i testi fissi del *widget* sono in italiano e in inglese, ciascuna lingua in un proprio file di traduzione. Per aggiungerne una, lo sviluppatore scrive un nuovo file e lo registra nel modulo delle traduzioni, senza modificare i componenti.

3.4.3 Metriche di utilizzo e analisi delle conversazioni

Un assistente di vendita è utile al commerciante solo se il commerciante può capire come i clienti lo usano e se li aiuta davvero. Per OB5 ho quindi costruito due

strumenti complementari: i conteggi delle azioni del cliente, che mostrano come viene usata l'interfaccia, e l'analisi del contenuto delle conversazioni, che aiuta a individuare quelle da rileggere. La figura 3.10 mostra il percorso comune dei dati e i tre usi che ne fa la *dashboard*.

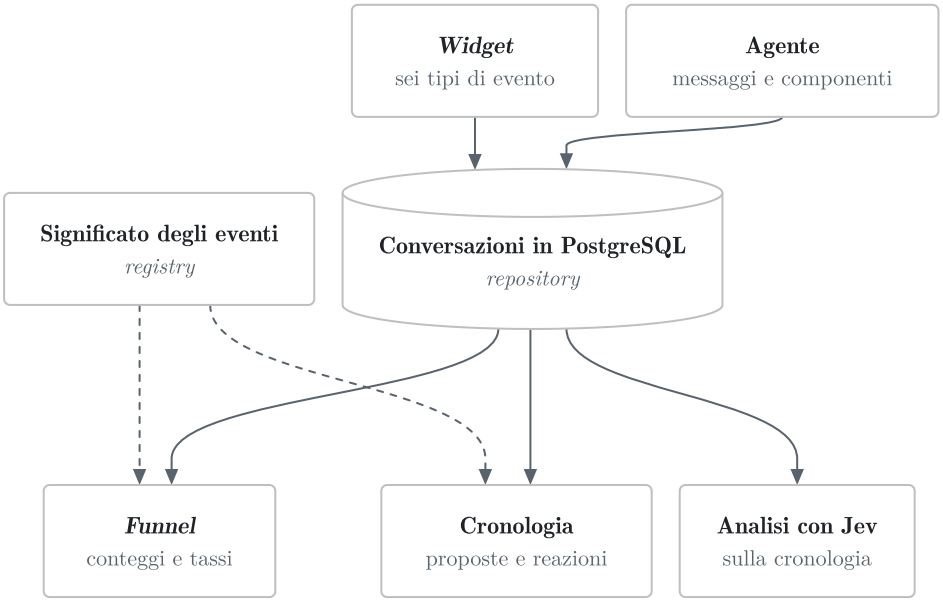


Figura 3.10: Messaggi ed eventi alimentano conteggi, cronologia e analisi; le linee tratteggiate indicano dove il *registry* dei significati stabilisce come ogni evento entra nel *funnel* e nella cronologia. L'analisi automatica aiuta la lettura, senza sostituire il giudizio del commerciante.

Il *server* sa che cosa l'agente ha mostrato, ma soltanto il *widget* osserva come il cliente usa i componenti. Per questo registro sei eventi (apertura della chat, clic su prodotto, aggiunta al carrello, clic su scheda informativa, clic su collezione e clic al pagamento) e li invio attraverso l'*app proxy* con richieste che non bloccano l'interfaccia e che si completano anche quando il cliente lascia la pagina, per esempio passando al pagamento. Lo schema condiviso ne valida tipo e dati, e un *registry* ne fissa il significato per misure e cronologia.

Conservo conversazioni, messaggi ed eventi in PostgreSQL tramite Prisma, un ORM$_G$, *Object-Relational Mapper*, che genera dallo schema i tipi e le funzioni di accesso ai dati; ogni conversazione contiene messaggi, eventi e risultati dell'analisi. Il modulo delle conversazioni applica il *repository* e concentra due regole: considera significative solo le conversazioni in cui il cliente ha scritto e aggiorna l'ultima attività con ogni nuovo messaggio o evento, così che le pagine e l'analisi usino lo stesso criterio di attività.

La *dashboard* calcola un *funnel*$_{\text{G}}$, cioè la sequenza di passaggi che avvicinano il visitatore all'acquisto. Per gli ultimi trenta giorni la *dashboard* mostra caricamento del *widget*, apertura, conversazione avviata, uso di un componente e clic al pagamento, con i tassi di coinvolgimento, di uso dei componenti e di conversione. Conto le conversazioni che raggiungono ciascun passaggio, perché più clic dello stesso cliente non devono diventare più conversioni. L'uso dei componenti rende osservabile la parte generativa dell'interfaccia, anche se non ne isola il contributo causale.

Per l'analisi serviva un modello capace di classificare molte conversazioni secondo criteri uniformi, un compito diverso da quello dell'assistente di vendita, che richiede la capacità generativa e l'uso degli strumenti. Ho scelto Jev, il modello decisionale «System One» di TypeSafe AI[15], che riceve una situazione e alcune domande con le risposte ammesse, descritte da me nella richiesta, e restituisce scelte o punteggi con le relative probabilità, senza generare testo.

Considero conclusa una conversazione dopo trenta minuti senza attività. Dal pulsante «Analizza» il commerciante avvia l'elaborazione di quelle concluse e non ancora analizzate, o aggiornate dopo l'ultima analisi, e con la funzione `experimental_evaluate` dell'AI SDK[16] pongo nella stessa chiamata le tre domande della tabella 3.9.

Indicatore	**Domanda**	**Risposta ammessa**
Qualità dell'aiuto	Quanto l'assistente ha aiutato il cliente a raggiungere ciò che voleva?	Un punteggio mostrato su cinque livelli.
Intenzione	Che cosa cercava principalmente di fare il cliente?	Una tra nove categorie, come ricerca precisa, confronto o informazioni sul negozio.
Motivo del giudizio	Quale problema o punto di forza spiega meglio il risultato?	Una tra quattordici etichette, sette problemi e sette punti di forza.

Tabella 3.9: Le tre dimensioni dell'analisi automatica di una conversazione.

Ho tenuto separati intenzione e giudizio, perché la prima classifica la richiesta del cliente, per esempio la ricerca di un regalo, mentre il secondo valuta la risposta dell'assistente, e la stessa intenzione può ricevere un aiuto buono o scarso. La terza domanda motiva il giudizio con un'etichetta, dato che un punteggio isolato non orienterebbe la lettura; il commerciante può poi verificarlo nella cronologia.

Lo stesso compito potrebbe svolgerlo un modello generativo con risposta strutturata; ho preferito sperimentare un modello progettato per scegliere tra opzioni note. La garanzia riguarda però solo la forma della risposta, perché Jev può scegliere una categoria ammessa e sbagliata: l'analisi resta uno strumento di consultazione, che il commerciante verifica tornando ai messaggi.

Su Vercel il *server* non può proseguire un lavoro dopo aver inviato la risposta, e ho quindi elaborato le conversazioni in sequenza dentro la richiesta avviata dal pulsante, una soluzione adatta al negozio di prova che con molte conversazioni richiederebbe un'elaborazione separata. Anche il *funnel* va letto con cautela, perché dimostra che le misure funzionano, ma un clic al pagamento non prova un ordine concluso né un aumento delle vendite.

3.5 Sperimentazioni e sviluppi futuri

La prima versione completa copriva gli obiettivi obbligatori, ma non diceva quale esperienza d'uso fosse più efficace. Dopo il 2 ottobre io e il *tutor* abbiamo quindi verificato alcune ipotesi, accettando di scartare parte del lavoro. La tabella 3.10 raccoglie le prove rimaste fuori dal prodotto finale, con l'idea di partenza e il motivo della rinuncia.

Prova e periodo	**Ipotesi**	**Esito e decisione**
Inviti contestuali (8–19/10)	Una domanda legata alla pagina incoraggia l'apertura della chat.	Una chiamata al modello per ogni visita e regole aggiuntive portavano poco valore nelle prove; abbiamo tolto gli inviti.
Personalizzazione manuale (30/09–1/10)	Il commerciante riproduce l'identità del marchio con impostazioni visive.	La soluzione copriva solo parte dell'aspetto e non riusava i componenti del tema.
Stile ricavato dal pagamento (2/10)	La configurazione del pagamento rappresenta l'identità del negozio.	Il pagamento ha uno stile indipendente dal tema; non era una fonte sufficiente.
Stile ricavato dal tema (5–15/10)	Un modello ricava l'aspetto dalla configurazione del tema.	I risultati non giustificavano la complessità; il 15/10 abbiamo tolto la personalizzazione.
Pannello affiancato (14–15/10)	Restringere la pagina permette una conversazione più immersiva.	Ogni tema richiedeva correzioni per pannelli e navigazione; sono tornato al pannello flottante.

Prova e periodo	Ipotesi	Esito e decisione
Storia delle visite (15–21/10)	Le pagine precedenti migliorano il contesto dell'agente.	Era utile soprattutto agli inviti; senza di essi migliorava poco le risposte.
Jev per scegliere i componenti (solo progettato)	Un modello decisionale sceglie che cosa mostrare.	Avrei dovuto sostituire il ciclo dell'AI SDK con uno mio; ho rinunciato prima di implementarlo.

Tabella 3.10: Le sperimentazioni escluse dalla versione finale. Il confronto sul valore resta qualitativo, basato sulle prove del prototipo.

La personalizzazione visiva mostra il criterio con cui abbiamo ridotto il perimetro. Il *widget* deve rispettare il *design system*$_{\text{G}}$ del marchio, cioè le regole comuni per colori, caratteri, spaziature e componenti, e per ricavarlo ho provato tre fonti successive: le scelte manuali del commerciante, le impostazioni del pagamento e i file di configurazione del tema interpretati da un modello. La terza strada richiedeva anche di ricostruire i valori predefiniti e di interpretare convenzioni diverse tra temi, e la complessità cresceva più della qualità ottenuta.

Alla fine ho scelto una configurazione di variabili CSS che lo sviluppatore adatta al cliente. La soluzione è coerente con i progetti su misura di Halue, in cui chi integra ChatCart conosce il negozio e può intervenire nel codice, mentre l'adattamento automatico servirebbe soprattutto a un'applicazione da installare senza assistenza in molti negozi diversi. La rinuncia dipende quindi anche dal modo in cui Halue potrebbe proporre il servizio.

Le altre prove hanno messo in luce costi meno evidenti. Gli inviti contestuali generavano contenuti prima che il cliente aprisse la chat, e facevano quindi crescere le chiamate al modello con il traffico delle pagine anche quando nessuno usava l'assistente; il pannello affiancato dipendeva invece dalla struttura di ciascun tema. Ho conservato l'integrazione con il tema dove produceva un vantaggio centrale, come nel carrello, e ridotto le funzioni che richiedevano lo stesso sforzo per un beneficio marginale.

Le rinunce e i limiti emersi indicano anche dove il prototipo potrebbe evolvere, secondo tre priorità:

1. **Portare il prototipo su negozi reali**. Servono una gestione del catalogo con ricerca mirata e paginazione, la separazione dei dati tra negozi, limiti alle richieste e un'elaborazione dell'analisi adatta a volumi maggiori. Oggi

l'agente legge al più i primi 250 prodotti e li filtra in memoria; questo limite è accettabile sul catalogo di prova ma richiede una scelta diversa su cataloghi grandi. Il traffico reale permetterebbe anche un confronto con l'esperienza senza assistente.

2. **Estendere ciò che l'agente conosce**. Le informazioni nei campi personalizzati dei prodotti, le raccomandazioni e gli articoli del negozio potrebbero arricchire i consigli. Lo stato degli ordini (DE2) aggiungerebbe il supporto dopo l'acquisto, con la necessità di riconoscere il cliente e accedere ai suoi dati.
3. **Verificare altre integrazioni**. Hydrogen (FA1) richiede di ripensare il carrello sulla Storefront API; il confronto con il *server* MCP di Shopify (FA2) permetterebbe di valutare il costo degli strumenti su misura. Sul prodotto attuale, eseguire automaticamente le prove dei temi a ogni rilascio aiuterebbe a scoprire cambiamenti nelle loro interfacce.

Questi sviluppi restano proposte, distinte dalle capacità che ho verificato nel prototipo; il modo in cui le ho verificate è l'oggetto della sezione successiva.

3.6 Verifica e validazione

Le parti di ChatCart che possono cambiare senza preavviso sono quelle che non controllo, cioè il modello, le API di Shopify e i temi, che possono cambiare indipendentemente da ChatCart, mentre la struttura interna dei moduli cambiava spesso per mia scelta. Ho quindi concentrato le prove automatiche sulle cuciture$_G$, i punti in cui il mio codice incontra quei sistemi, così da fissare il comportamento ai confini e lasciare libera l'organizzazione interna. La tabella 3.11 riassume i confini, le tecniche e ciò che ciascuna prova può dimostrare.

Confine	Tecnica	Comportamento verificato
Temi	HTML modellato su Dawn e Horizon in happy-dom, una simulazione della pagina.	Riconoscimento del tema, aggiornamento delle sezioni, rilevazione delle scritture esterne.
API di Shopify	Risposte controllate del *client* Storefront e delle richieste al carrello.	Lettura dei dati, costruzione dei componenti, trattamento degli errori.
Modello	Modelli simulati dell'AI SDK, con risposta e chiamate osservabili.	Istruzioni, contesto, storia inviata e gestione del ciclo; la prova non valuta la scelta semantica del modello reale.

Confine	Tecnica	Comportamento verificato
Base di dati	PostgreSQL locale con migrazioni e dati di prova.	Interrogazioni del *funnel*, ordine e contenuto delle conversazioni, stato dell'analisi.
Rotte del proxy	Richieste costruite come quelle del *widget*.	Lettura degli argomenti, chiamata dei moduli e risposta della rotta.

Tabella 3.11: La strategia delle prove automatiche sulle cuciture del sistema.

Eseguo le prove TypeScript con Vitest e sostituisco soltanto i sistemi esterni al processo, mantenendo reali i moduli interni. Un *mock*$_{\text{G}}$, cioè un sostituto con risposte stabilite dalla prova, applicato a un modulo interno legherebbe il *test* al modo in cui ho diviso il codice; in questo modo, quando ho riorganizzato il codice e introdotto i *pattern*, i *test* esistenti hanno continuato a verificare lo stesso comportamento senza bisogno di modifiche.

Grazie alla *dependency injection* (figura 3.6) riproduco senza rete un modello che non risponde, una variante esaurita o un catalogo incompleto.

La base di dati richiede un criterio diverso, perché i calcoli del *funnel* e della selezione delle conversazioni stanno anche nelle interrogazioni, e simulare Prisma dimostrerebbe soltanto che il programma invoca certe funzioni. Ho quindi usato una base di dati reale e, quando la produzione è passata a PostgreSQL, ho portato anche le prove su un PostgreSQL locale con le stesse migrazioni. Verifico così schema e interrogazioni sullo stesso sistema usato nel rilascio.

Ho scelto i casi a partire dalle regole e dai limiti del prodotto, come una variante non disponibile, un tema sconosciuto, una conversazione cancellata durante la risposta, due aggiornamenti successivi del carrello. Ho lasciato invece alla verifica manuale gran parte della disposizione visiva del pannello e della *dashboard*.

Le prove con un modello simulato non dimostrano che l'agente reale sappia consigliare bene. Per questo ho svolto conversazioni di prova sul negozio di sviluppo. Il comportamento richiesto nelle istruzioni non ha però una verifica automatica continuativa, e questo limita l'interpretazione della copertura dei requisiti conversazionali.

Alle prove automatiche ho affiancato il controllo dei tipi in modalità rigorosa e un *linter*$_{\text{G}}$, uno strumento di analisi statica che segnala costrutti scorretti o sospetti, e ho verificato a mano il percorso completo su Dawn e Horizon, da *desktop* e da *mobile*.

Ho ripetuto queste verifiche dopo aver pubblicato ChatCart su Vercel e Neon a fine ottobre, servizi scelti con il *tutor* dopo un primo tentativo su un *server* gestito da me.

La validazione ha accompagnato lo sviluppo attraverso il confronto con il *tutor*, che il 2 ottobre ha valutato la prima versione completa e il 14 ottobre ha mostrato il lavoro al fondatore; il 9 novembre ho presentato il prodotto a tutto il *team*, con una dimostrazione funzionale e una discussione tecnica. Questi incontri hanno confermato l'utilità del prototipo per l'esplorazione di Halue, ma non equivalgono a una validazione commerciale con visitatori e ordini reali.

3.7 Risultati

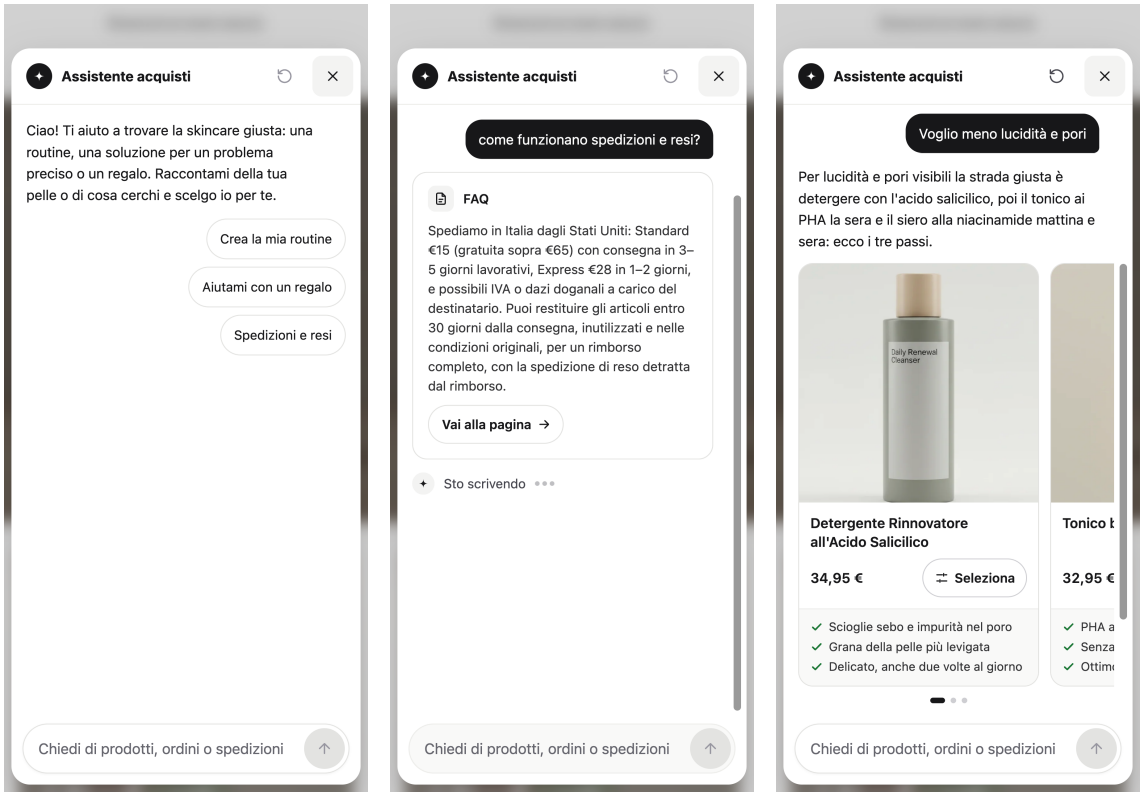
3.7.1 Il prodotto dal punto di vista dell'utente

Al termine dello *stage* ChatCart è un prototipo utilizzabile dall'inizio alla fine su un negozio di sviluppo con circa quaranta prodotti cosmetici, varianti e collezioni. Le schermate seguenti ripercorrono l'esperienza dei due attori. Il cliente può cercare e confrontare prodotti, chiedere informazioni e raggiungere il *checkout*$_{\mathrm{G}}$, la procedura di pagamento di Shopify, passando dalla chat allo stesso carrello del negozio (figura 3.11).

Il cliente apre il pannello e scrive ciò che cerca, oppure sceglie uno dei suggerimenti iniziali, e la risposta compare progressivamente insieme ai componenti pertinenti: schede di prodotto per i consigli d'acquisto, una scheda con il collegamento alla fonte per le informazioni sul negozio. Se cambia pagina, il cliente ritrova la conversazione; da *mobile* il pannello occupa lo schermo, mentre da *desktop* resta sovrapposto alla pagina.

Dalla scheda il cliente seleziona una variante e la aggiunge al carrello, poi ne modifica la quantità o prosegue al pagamento. La figura 3.12 mostra questi comandi nella stessa conversazione; ogni modifica raggiunge anche il carrello del tema, nel modo descritto nella sezione 3.4.2.

Il commerciante attiva il *widget* dall'*editor* del tema, senza modificarne il codice, e configura il ripiego quando il tema lo richiede. Dall'amministrazione consulta poi la *dashboard* della figura 3.13. Il riepilogo dei passaggi mostra in quale passaggio i clienti abbandonano la chat, e l'elenco affianca agli esiti osservati i punteggi e le etichette dell'analisi automatica, per orientare la rilettura.

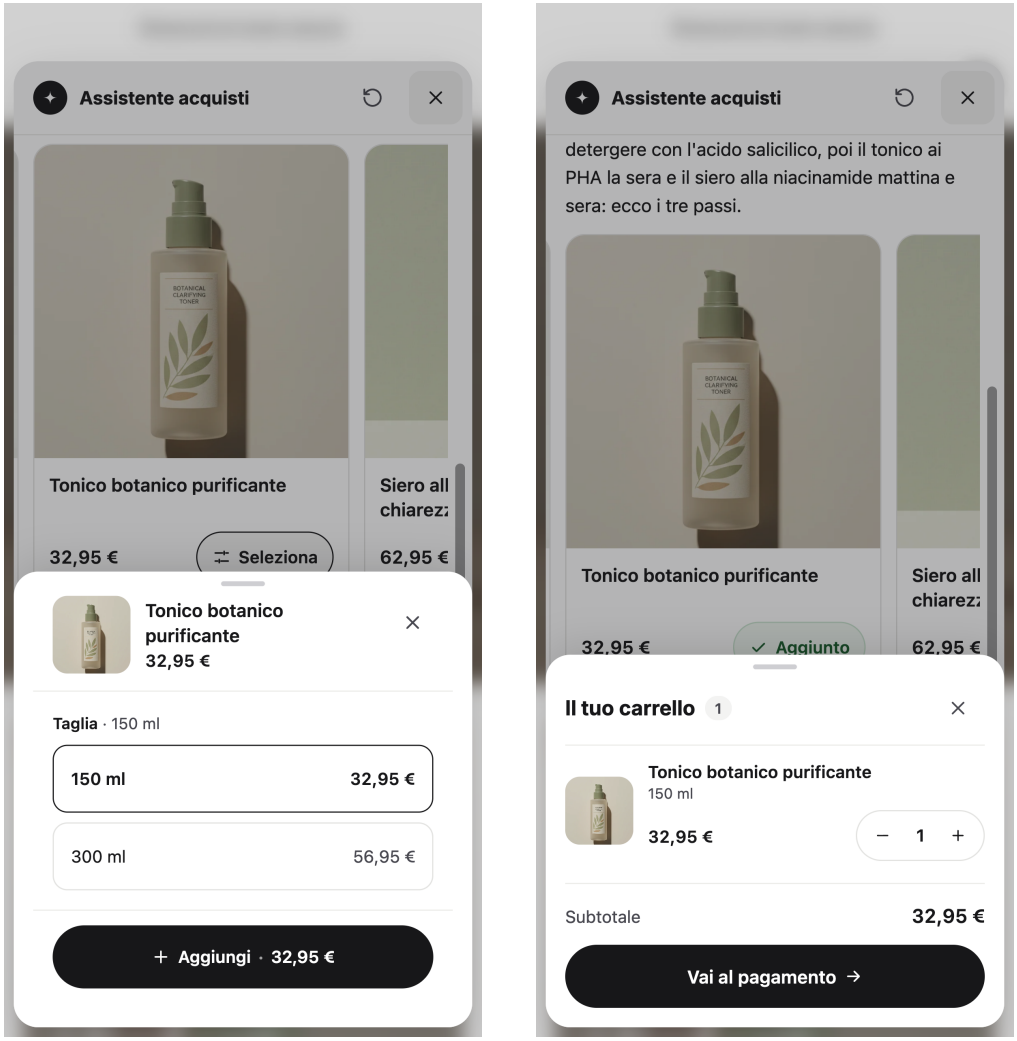


(a) Avvio della chat (b) Informazioni del negozio (c) Prodotti consigliati

Figura 3.11: Il *widget* da *mobile*: benvenuto con proposte iniziali, risposta su spedizioni e resi con collegamento alla pagina del negozio, consigli con schede di prodotto scorribili. Ho acquisito le schermate sul negozio di sviluppo.

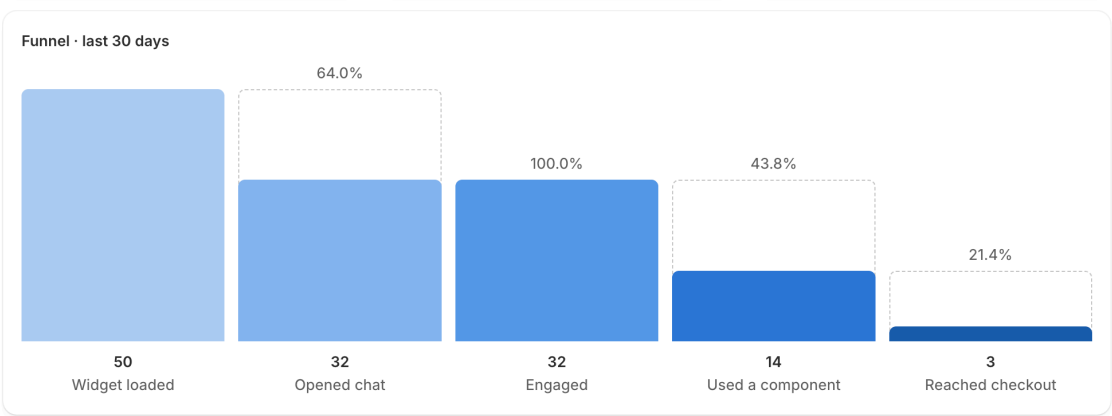
Aprendo una conversazione il commerciante ritrova messaggi, prodotti mostrati e azioni del cliente in ordine cronologico (figura 3.14) e può confrontare punteggio ed etichette con lo scambio effettivo, fino all'aggiunta al carrello e al clic verso il pagamento, verificando così il giudizio automatico sul singolo caso.

Ho verificato questo percorso su Dawn e Horizon, e il risultato dimostra che la combinazione tra conversazione, componenti e carrello del tema è realizzabile nel contesto del prototipo. L'efficacia sul comportamento d'acquisto resta invece da verificare, perché ChatCart non ha ricevuto traffico di clienti reali; restano aperti i limiti discussi nella sezione 3.5.



(a) Scelta della variante (b) Carrello nella chat

Figura 3.12: Scelta del formato di un prodotto e carrello dopo l'aggiunta. Nel carrello il cliente controlla quantità e subtotale e accede al pagamento con il pulsante dedicato. Le schermate provengono dalla stessa prova sul negozio di sviluppo.



Recent engaged conversations

All caught up. Conversations are ready to analyze after 30 minutes idle. Analyze conversations

Conversation	When	Msgs	Outcome	Score	Shopper wanted	Assistant
Crea la mia routine	3 minutes ago	4	Added to cart	Not analyzed	—	—
come funzionano spedizioni e resi?	5 minutes ago	2	—	Not analyzed	—	—
come funzionano spedizioni e resi?	5 minutes ago	2	—	Not analyzed	—	—
Spedizioni e resi	6 minutes ago	2	—	Not analyzed	—	—
ho un budget di 50 euro	10 minutes ago	4	Added to cart	Not analyzed	—	—
cosa vendete?	3 days ago	4	Added to cart	5/5	Looking for a specific product	Led the shopper to cart or checkout
Spedizioni e resi	3 days ago	8	—	4/5	Looking for a specific product	Answered clearly
mi serve un detergente delicato	4 days ago	6	Added to cart	5/5	Price-conscious	Helped pick a size or variant
Crea la mia routine	4 days ago	6	Reached checkout	5/5	Looking for a specific product	Led the shopper to cart or checkout
cosa è questa collezione?	4 days ago	2	—	5/5	Browsing for ideas	Answered clearly
cosa fa questo?	4 days ago	2	—	5/5	Looking for a specific product	Answered clearly
Spedizioni e resi	4 days ago	2	—	1/5	Asking about shipping or returns	Dodged a question
cosa vendete? che linee di prodotti avete?	4 days ago	12	Added to cart	4/5	Comparing options	Dodged a question

Figura 3.13: Riepilogo dei passaggi d'uso e conversazioni recenti. L'elenco distingue le conversazioni ancora da analizzare da quelle con punteggio, intenzione del cliente e giudizio sull'assistente. Conteggi e percentuali si riferiscono alle prove sul negozio di sviluppo e non misurano l'efficacia su clienti reali.

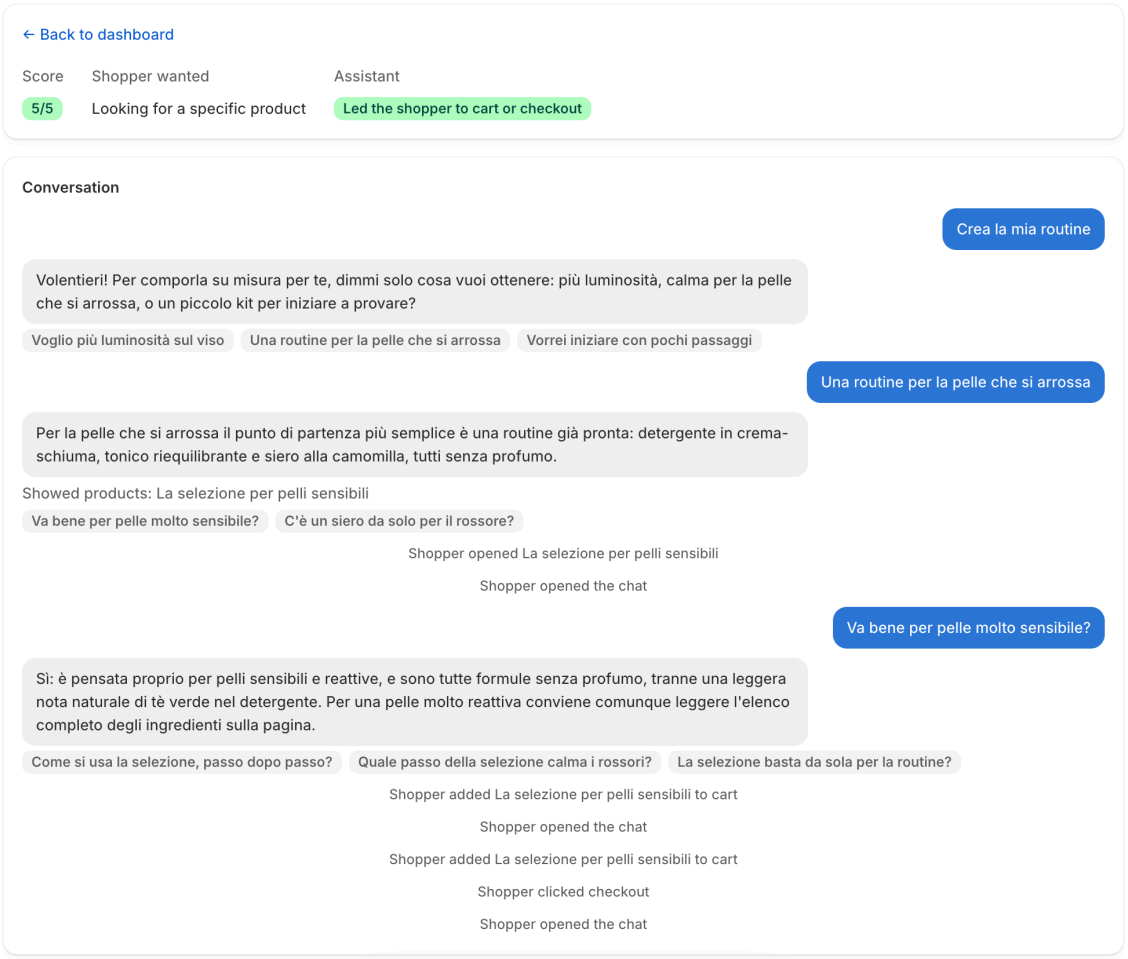


Figura 3.14: Una conversazione di prova nella vista del commerciante. In alto compaiono punteggio ed etichette; la cronologia conserva risposte, suggerimenti, riferimenti ai prodotti e azioni del cliente. L'evento finale di accesso al pagamento indica il clic, non un acquisto completato.

3.7.2 Risultati quantitativi

Valuto i risultati sulla versione finale del codice, del 7 novembre, distinguendo tre dimensioni: la copertura dei requisiti, la copertura delle righe di codice e la quantità dei prodotti realizzati. Nessuna di queste misure riguarda l'utilità commerciale o l'accuratezza del modello.

La tabella 3.12 riguarda soltanto i venti requisiti principali presentati nella sezione 3.2.2, con gli stessi identificativi. Considero soddisfatto un requisito quando la versione finale ne realizza il comportamento richiesto; per quelli affidati alle istruzioni, però, la valutazione si basa sulla soluzione implementata e sulle prove manuali, senza garantire che il modello rispetti la regola in ogni conversazione.

Stato	**Numero**	**Requisiti della selezione**
Soddisfatti	16	RFOB2, RFOB3, RFOB5, RFOB6, RFOB9, RFOB10, RFOB11, RFOB12, RFOB15, RFOB22, RFOB24, RFDE3, RFDE7, RQOB2, RQDE1, RVOB2.
Parziali	1	RFOB13.
Non soddisfatti	3	RFDE5, RFDE8, RVOP1.
Totale	**20**	La base di conteggio è la selezione esposta, non l'insieme originario dei 45 requisiti.

Tabella 3.12: Copertura dei venti requisiti principali nella versione finale.

RFOB13 è parziale perché il codice impone l'arresto al componente, ma non tutte le combinazioni tra testo e componente richieste dalle istruzioni. Io e il *tutor* abbiamo escluso dal perimetro lo stato dell'ordine (RFDE5) e la prova su Hydrogen (RVOP1), mentre il confronto con le sessioni senza assistente (RFDE8) richiede dati reali che il negozio di sviluppo non offre.

La copertura delle righe attraversate dalle prove è complessivamente del 63,1%, ma la media nasconde quanto le prove si concentrino sulle cuciture (tabella 3.13).

Area	**Righe coperte**	**Copertura**
Logica dell'agente	673 / 679	99,1%
Dashboard lato *server*	550 / 554	99,3%
Conversazioni	114 / 116	98,3%
Rotte del *widget*	49 / 56	87,5%
Carrello e sincronizzazione	394 / 403	97,8%

Area	Righe coperte	Copertura
Tipi e schemi condivisi	204 / 204	100%
Schede e messaggi del *widget*	526 / 697	75,5%
Avvio e pannello del *widget*	0 / 538	0%
Stato della conversazione e librerie del *widget*	222 / 385	57,7%
Componenti della *dashboard*	0 / 298	0%
Autenticazione, amministrazione e avvio	9 / 332	2,7%
Script di supporto	0 / 85	0%
Totale	**2.741 / 4.347**	**63,1%**

Tabella 3.13: Copertura delle righe per area, misurata con Vitest sulla versione finale. L'avvio e il pannello del *widget* comprendono pulsante di apertura, intestazione, campo di testo e pannello del carrello.

Agente, interrogazioni della *dashboard* e sincronizzazione del carrello superano il 97%, mentre i componenti visivi della chat e della *dashboard* dipendono dalle prove manuali. Nella versione finale passano tutti i 174 casi in 17 file, insieme al controllo dei tipi e all'analisi statica.

Il prodotto comprende 7.550 righe non vuote in 95 file, *test* esclusi: 2.626 nel *server*, 4.367 nel *widget*, di cui 1.772 di CSS, e 269 nelle definizioni condivise. La documentazione finale comprende il README con i diagrammi e la presentazione del 9 novembre; le specifiche e i piani intermedi hanno guidato le modifiche descritte nella sezione 3.1.2. Nella sezione 4.1 valuto questi risultati rispetto agli obiettivi.

4 Valutazione retrospettiva

Valuto lo *stage* a posteriori: in che misura ho soddisfatto gli obiettivi del piano di lavoro e i miei, che cosa mi ha lasciato in termini di conoscenze e abilità e quanto il corso di studi mi abbia preparato al lavoro che ho svolto.

4.1 Soddisfacimento degli obiettivi

Per valutare gli obiettivi ho confrontato la formulazione del piano di lavoro con il comportamento della versione finale del codice, la stessa su cui ho misurato i risultati della sezione 3.7.2. Considero raggiunto un obiettivo quando il prototipo realizza ciò che il piano descrive, e segnalo come scostamento ogni differenza tra la formulazione e la soluzione implementata, anche quando l'ho introdotta di proposito. La tabella 4.1 riporta lo stato degli obiettivi del piano (tabella 2.1 e tabella 2.2) e dei miei obiettivi personali (tabella 2.3), con la sezione che ne documenta la realizzazione.

Il prototipo soddisfa tutti gli obiettivi obbligatori, che il piano definisce vincolanti, e le 304 ore rientrano nell'intervallo da 300 a 320 ore fissato dai vincoli del progetto (sezione 2.4). Halue aveva indicato come criterio di successo un prototipo funzionante dall'inizio alla fine su un negozio di esempio, accompagnato da una valutazione onesta di ciò che funziona e di ciò che non funziona. ChatCart funziona sul negozio di sviluppo nella versione pubblicata su Vercel, e la valutazione l'ho presentata al *team* l'ultimo giorno di *stage*. Quattro obiettivi raggiunti presentano però differenze dalla formulazione del piano, che discuto di seguito insieme agli obiettivi esclusi.

Il piano descriveva il *widget* come responsabile della sola visualizzazione, così che ogni decisione restasse nell'*agent layer*. Nella versione finale il *widget* legge e scrive da sé il carrello di Shopify e lo mantiene allineato al tema, perché ho stabilito che l'agente non veda né modifichi il carrello e che il cliente acquisti soltanto con i pulsanti dei componenti (sezione 3.4.2). Lo scostamento sposta nel *browser* una

ID	Stato	Evidenza
OB1	Raggiunto, con scostamento	Il *widget* mostra conversazione e componenti, e gestisce anche il carrello (sezione 3.4.2).
OB2	Raggiunto	Cinque strumenti di lettura interrogano la Storefront API (sezione 3.4.1).
OB3	Raggiunto	L'agente risponde con testo, con una domanda di chiarimento o con un componente (sezione 3.4.1).
OB4	Raggiunto, con scostamento	Tre componenti e i messaggi suggeriti, che in alcuni casi sono due o mancano (sezione 3.3.2).
OB5	Raggiunto	Sei eventi, *funnel* e analisi delle conversazioni, misurati senza traffico reale (sezione 3.4.3).
DE1	Raggiunto	Paese e lingua del cliente in ogni lettura del catalogo, prezzi nella valuta del mercato (sezione 3.4.2).
DE2	Non raggiunto	Escluso con il *tutor* dal perimetro del progetto.
FA1	Non raggiunto	Escluso con il *tutor* dal perimetro del progetto.
FA2	Non raggiunto	Escluso con il *tutor* dal perimetro del progetto.
OP1	Raggiunto	Un'applicazione completa, dal modello ufficiale al rilascio (sezione 3.2.1).
OP2	Raggiunto	L'agente, il contratto tipizzato e la divisione delle regole tra istruzioni e codice (sezione 3.3.1).
OP3	Raggiunto	L'architettura dell'agente e la gestione del negozio con assistenti IA attraverso le Admin API (sezione 3.1.3).
OP4	Raggiunto	304 ore svolte in prevalenza da remoto, con il metodo della sezione 3.1.2.

Tabella 4.1: Lo stato degli obiettivi del piano di lavoro e dei miei obiettivi personali nella versione finale.

parte della logica, con un costo di manutenzione preciso: un adattatore per ogni famiglia di temi, da aggiornare quando un tema cambia le proprie interfacce. In cambio nessuna risposta del modello produce da sola un effetto sull'acquisto. Lo considero coerente con l'intento del piano, che voleva separare la presentazione dalle decisioni, perché il *widget* esegue soltanto le azioni che il cliente conferma e l'agente resta l'unico a decidere che cosa mostrare.

Per OB4 il piano chiedeva tre messaggi suggeriti per ogni risposta. ChatCart li genera con una seconda chiamata al modello, accetta due suggerimenti quando uno dei tre non è utilizzabile e mostra la risposta senza suggerimenti quando la chiamata fallisce o ne produce meno di due. Ho scelto di proposito questa regola, perché pre-

ferivo una risposta completa senza un elemento accessorio a una risposta in ritardo (sezione 3.4.1). Il modulo di iscrizione alla *newsletter* citato nel piano era soltanto un esempio, e le schede prodotto, la scheda informativa e i collegamenti alle collezioni soddisfano la richiesta di almeno due o tre componenti.

OB5 è raggiunto come strumentazione, ma la strumentazione non basta a rispondere alla domanda per cui il piano la richiedeva, cioè se l'assistente sia efficace. Sul negozio di sviluppo le misure contano le mie conversazioni di prova, e il *funnel* si ferma al clic verso il pagamento, perché ChatCart non legge gli ordini e nessun ordine risulta collegato a una conversazione. Verificare se l'assistente porta più clienti all'acquisto, il beneficio che lo studio citato nella sezione 2.2 attribuisce a un assistente efficace, richiede traffico reale e un confronto con le sessioni senza assistente, il requisito RFDE8 che la sezione 3.7.2 riporta come non soddisfatto. Anche DE1 ha un limite: catalogo, prezzi e risposte seguono la lingua e il mercato del cliente, mentre i testi fissi del *widget* esistono in italiano e in inglese, e ogni altra lingua richiede un nuovo file di traduzione.

DE2, FA1 e FA2 non sono raggiunti. Io e il *tutor* li abbiamo esclusi dal perimetro dopo aver deciso di dedicare la seconda metà dello *stage* all'esperienza d'uso e alla sperimentazione (sezione 3.1.1), che Halue considerava la parte di maggior valore. Il piano prevedeva questa possibilità, perché considerava le estensioni solo dopo il completamento degli obiettivi obbligatori; DE1 è l'unica estensione che ho realizzato. Lo stato degli ordini avrebbe richiesto inoltre di riconoscere il cliente e di accedere ai suoi dati, e la sezione 3.5 descrive che cosa servirebbe per realizzare i tre obiettivi esclusi.

Valuto allo stesso modo i prodotti attesi del piano (tabella 4.2). Ho consegnato al *team* il codice completo, con la documentazione che lo accompagna.

Il documento di architettura ha preso la forma della documentazione del codice invece che di un documento separato, e la libreria dei componenti fa parte del *widget* invece che di un pacchetto riutilizzabile da altre applicazioni. La relazione tecnica è l'unico prodotto atteso che non ho realizzato come documento a sé: i risultati del progetto li ho esposti al *team* nella presentazione finale.

Il *tutor* mi aveva chiesto inoltre una presentazione formale al *team* per l'ultimo giorno di *stage*. Prima della presentazione ho fatto il punto sull'intero progetto, e durante la presentazione ho svolto la retrospettiva che Halue prevede alla fine di ogni *stage* (sezione 2.1): ho illustrato che cosa ha funzionato, che cosa non ha funzionato e per quali ragioni, a partire dagli obiettivi esclusi e dalle sperimentazioni abbandonate.

Prodotto atteso	Consegna
Documento di architettura	Il README del codice e la cartella della documentazione, con i diagrammi dell'architettura, dei casi d'uso, del percorso di un messaggio e della sincronizzazione del carrello.
Codice sorgente	Il codice completo del *server*, del *widget*, dell'estensione e delle definizioni condivise (sezione 3.7.2).
Libreria dei componenti	Le schede prodotto con la scelta della variante, la scheda informativa, i collegamenti alle collezioni e il pannello del carrello, interni al *widget*.
Strumentazione delle metriche	Gli eventi, il *funnel*, la cronologia e l'analisi delle conversazioni nella *dashboard* (sezione 3.4.3).
Relazione tecnica sui risultati e sui *test*	Sostituita dalla presentazione finale al *team* e dalla documentazione del codice.

Tabella 4.2: I prodotti attesi del piano di lavoro e ciò che ho consegnato.

Il materiale della presentazione riassume in forma semplificata la documentazione del codice, e per questo non l'ho consegnato come prodotto a sé. Il suo valore è stato il confronto con il *team*, le cui domande si sono concentrate sulle scelte di progettazione e sul modo in cui ho istruito l'agente (sezione 2.3).

4.2 Maturazione professionale

Lo *stage* mi ha lasciato conoscenze tecniche, abilità nel modo di lavorare e un modo diverso di usare gli assistenti IA. Considero quest'ultimo l'apprendimento più importante, perché gli strumenti di IA fanno già parte del lavoro quotidiano di chi sviluppa, come mostra il ciclo di lavoro di Halue (sezione 1.5), e perché li avevo già usati spesso prima dello *stage*, ma mai con la consapevolezza che ho maturato durante il progetto. Lo tratto per ultimo, perché riprende le conoscenze e le abilità che descrivo prima.

Sul piano delle conoscenze, ho imparato come è fatta un'applicazione Shopify dall'inizio alla fine. Ho imparato anche React Router 7, il *framework* su cui si basa il modello di Shopify, e le sue differenze rispetto a Next.js, il *framework* per React che conoscevo già. In Next.js i componenti eseguiti sul *server* leggono i dati al proprio interno, mentre in React Router ogni rotta dichiara un *loader*$_{\text{G}}$, la funzione che prepara i dati sul *server* prima che la pagina venga mostrata, e un'*action*$_{\text{G}}$, la funzione che riceve le modifiche inviate dai moduli e dalle richieste. Il componente riceve quindi i dati già pronti, e ogni rotta raccoglie in un solo file letture, scritture

e interfaccia. In ChatCart la rotta che riceve gli eventi dal *widget* dichiara soltanto un'*action*, perché accetta solo scritture e non mostra alcuna pagina.

Sulla progettazione di agenti, l'oggetto di OP2, ho imparato a trattare il modello come un componente il cui risultato va guidato e controllato. Le istruzioni ne orientano il giudizio sul dialogo, il codice impone le regole che non devono dipendere da quel giudizio, e il contratto tipizzato verifica la forma dei dati prima che arrivino all'interfaccia (sezione 3.3.2, sezione 3.4.1). Dove collocare ciascuna regola è stata una delle decisioni più frequenti del progetto.

Il riordino del codice mi ha insegnato infine a riconoscere i *design pattern* in una soluzione già scritta e a introdurli quando semplificavano il codice (sezione 3.3.1). Ho capito soprattutto il loro valore come strumento di comunicazione. Un *pattern* condensa in un nome un problema ricorrente, la sua soluzione e le conseguenze. Nella presentazione finale ho descritto la sincronizzazione del carrello con tre nomi, *adapter*, *decorator* e *observer*, che chi conosce i *pattern* associa subito a una struttura e a un comportamento, mentre descrivere la stessa soluzione senza quei nomi avrebbe richiesto di spiegare un modulo alla volta.

Sul piano delle abilità, il lavoro da remoto mi ha insegnato a comunicare per iscritto in modo mirato. Una chiamata senza un obiettivo può durare un'ora senza produrre una decisione, e il tempo del *tutor* era limitato. Ho imparato quindi a definire prima di ogni confronto l'intenzione e il risultato atteso, a distillare dal lavoro svolto soltanto i dettagli utili alla decisione e a formulare ogni domanda in modo che la risposta sbloccasse un passo preciso. Superata la difficoltà iniziale a chiedere in tempo (sezione 3.1.1), da remoto ho comunicato in modo più efficace, perché il poco tempo a disposizione mi obbligava a preparare di più ogni scambio.

Ho imparato anche come si prendono le decisioni in un *team* strutturato. Le scelte che incidevano sul progetto le abbiamo prese insieme: il *tutor* ha proposto l'*adapter* per il carrello e io ne ho scelto l'implementazione, abbiamo ridotto insieme il perimetro escludendo DE2, FA1, FA2 e la personalizzazione visiva, e alcune decisioni sull'esperienza d'uso le abbiamo discusse con il fondatore. Il mio compito era arrivare a ogni confronto con le alternative già esplorate, e la decisione condivisa mi lasciava poi la responsabilità di realizzarla.

L'ultima abilità riguarda la scelta di che cosa escludere. Nelle sperimentazioni (sezione 3.5) ho rinunciato a funzioni che avevo già realizzato, come gli inviti contestuali e la personalizzazione visiva, quando il beneficio non giustificava la complessità. In un progetto di ricerca e sviluppo anche il lavoro scartato produce un risultato, perché

stabilisce quali strade non conviene percorrere, e ho imparato a considerarlo parte del risultato del progetto.

L'uso degli assistenti IA è l'abitudine che è cambiata di più. In Halue l'IA entra in ogni attività, e l'interesse dell'azienda è emerso anche nell'incontro conclusivo con il fondatore, nel mio ultimo giorno in sede, la cui prima domanda ha riguardato proprio il modo in cui avevo usato gli strumenti di programmazione con l'IA. Prima dello *stage* li usavo in due modi opposti: nei progetti personali e universitari scrivevo tutto il codice a mano, mentre nel lavoro da *freelance*, dove contava la velocità, affidavo agli assistenti intere funzionalità. Durante lo *stage* ho trovato un uso intermedio, che si è evoluto in tre fasi:

- all'inizio chiedevo agli assistenti di trovare le pagine pertinenti della documentazione, che poi leggevo io, e di spiegarmi i concetti nuovi con esempi, come le rotte di React Router;
- in seguito li ho usati per discutere alternative progettuali e costruire prototipi in qualche ora, come il primo *registry* degli strumenti;
- verso la fine assegnavo loro compiti sempre più piccoli e definiti, come un riordino limitato a una sola funzionalità, con indicazioni precise sul risultato atteso.

Riassumo questa evoluzione come meno potere e più direzione: l'assistente decideva sempre meno, mentre io stabilivo con sempre più precisione che cosa dovesse fare. La ragione di questa scelta riguarda i processi cognitivi con cui si acquisisce la padronanza di una tecnologia nuova, un tema che avevo approfondito prima dello *stage*, quando conciliare l'università da pendolare con il lavoro da *freelance* mi ha costretto a studiare i metodi di apprendimento più efficienti, per sostenere gli esami nei tempi previsti senza ridurre l'impegno verso i miei clienti. La tassonomia di Bloom$_{G}$, nella revisione di Anderson e Krathwohl[25], ordina i processi cognitivi in sei livelli di complessità crescente: ricordare, comprendere, applicare, analizzare, valutare e creare. Quando l'assistente scrive il codice, chi programma si limita a valutarlo e rinuncia al livello più alto, quello in cui si costruisce la padronanza. Per questo ho affidato agli assistenti ciò che non dovevo imparare, come i dati del negozio di prova e le prove nel *browser*, e ho tenuto per me la progettazione e l'implementazione definitiva, che erano l'oggetto dello *stage*. Usare l'IA in modo responsabile e mirato significa per me stabilire prima quale parte del lavoro devo svolgere da solo.

La mia esperienza conferma l'opinione condivisa nel *team* di Halue, secondo cui nelle

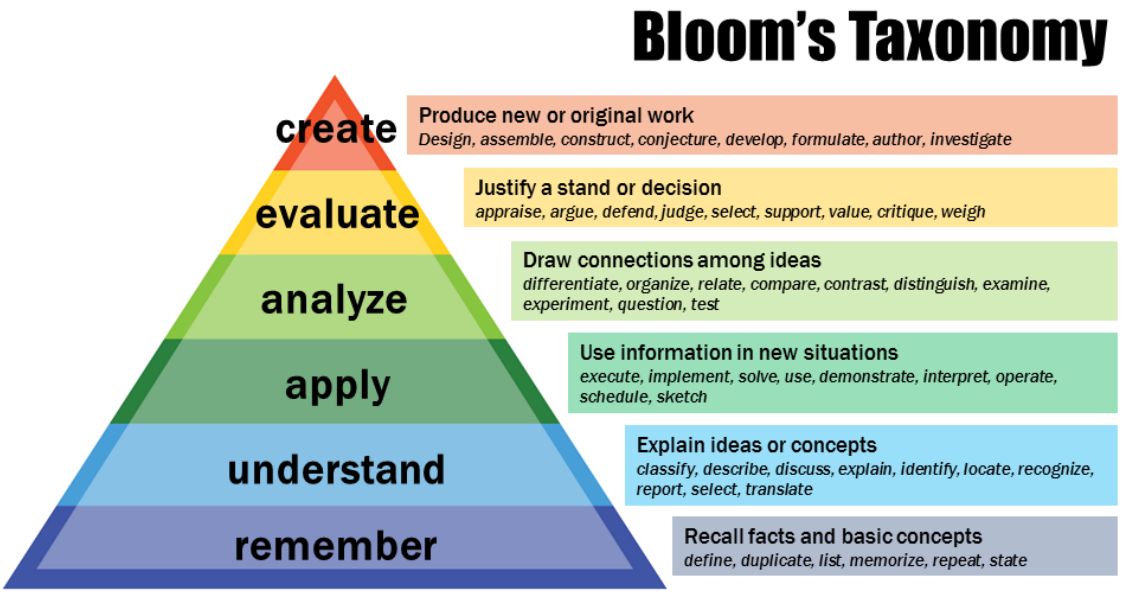


Figura 4.1: I sei livelli della tassonomia di Bloom nella revisione di Anderson e Krathwohl, dal più semplice in basso al più complesso in cima: per ciascuno, la capacità che esprime e i verbi d'azione con cui si formulano gli obiettivi di apprendimento di quel livello. Fonte: https://rotel.pressbooks.pub/readwritesuccess/chapter/understanding-blooms-taxonomy/.

mani di un programmatore esperto l'IA è un buon programmatore *junior*, ma non sostituisce un *senior* (sezione 1.5). Gli assistenti producevano rapidamente codice funzionante, mentre le scelte su cui si regge il progetto, come il contratto tra agente e *widget*, gli adattatori del carrello e la divisione delle regole tra istruzioni e codice, richiedevano un giudizio sull'intero sistema che spettava a me. Ho ottenuto i risultati migliori quando il compito era già definito con precisione, e definirlo richiedeva di conoscere il progetto.

4.3 Formazione universitaria e lavoro

Il corso di studi mi ha dato le basi su cui ho costruito gran parte del lavoro svolto durante lo *stage*. Due insegnamenti in particolare si sono rivelati decisivi: Ingegneria del Software e Programmazione ad Oggetti.

Ingegneria del Software è stato l'insegnamento più impegnativo dei tre anni e quello che mi è servito di più. Nel suo progetto didattico ho lavorato per mesi, in modo continuativo, in un gruppo di sette studenti, con un'organizzazione formale che comprendeva l'analisi dei requisiti, i diagrammi dei casi d'uso e un *way of working*$_{\text{G}}$, cioè l'insieme di regole con cui il gruppo pianifica, svolge e verifica il proprio lavoro.

Da quel progetto vengono le basi di gestione del lavoro con cui ho organizzato lo *stage*, il modo in cui ho scritto requisiti e casi d'uso (sezione 3.2.2) e l'impostazione delle prove automatiche (sezione 3.6). Il riscontro dei docenti sul lavoro del gruppo mi ha fatto capire l'importanza dei *design pattern*, che in ChatCart ho poi applicato a problemi concreti.

Lo *stage* mi ha mostrato anche come dosare quel rigore. Nel progetto didattico l'analisi dei requisiti precedeva la realizzazione, mentre in un progetto esplorativo io e il *tutor* abbiamo ricavato i requisiti iterando sul prototipo. Della formazione ricevuta ho mantenuto la tracciabilità, cioè il legame esplicito tra ogni requisito, il caso d'uso da cui deriva e l'obiettivo del piano, e l'ho applicata a un'analisi costruita durante lo sviluppo.

Programmazione ad Oggetti mi ha dato la base dei linguaggi a tipi statici. Il corso usava C++, e anche se ho scritto ChatCart in stile funzionale, l'abitudine a ragionare sui tipi mi ha guidato nella scelta di TypeScript in modalità rigorosa e dei tipi derivati dagli schemi Zod, su cui si fonda il contratto tra agente e *widget* (sezione 3.3.2). Anche i *design pattern*, che avevo studiato nella loro forma a oggetti, in ChatCart hanno preso una forma funzionale, con interfacce espresse come tipi e la creazione degli strumenti affidata a una funzione.

Le lacune riguardano due ambiti. Il corso di studi mi ha dato poca formazione e poca pratica sullo sviluppo di applicazioni web moderne, con i *framework* e le architetture che ho dovuto imparare durante lo *stage*. La lacuna principale riguarda però gli agenti di IA. Non avevo alcuna base su come si progetta un agente, su come si istruisce un modello linguistico, su come se ne validano le uscite e su quali pratiche consolidate seguire, e ho affrontato l'intero ambito per la prima volta. È la lacuna che pesa di più, perché i modelli linguistici entrano ormai sia negli strumenti con cui si programma sia nei prodotti che si realizzano.

Lo *stage* nasceva dall'incontro di tre esigenze (sezione 2.5): la ricerca di Halue sul commercio agentico, i limiti degli assistenti che rispondono soltanto con testo e il mio interesse per l'ecosistema Shopify e per gli agenti. In otto settimane e 304 ore ho realizzato ChatCart, un'applicazione Shopify in cui un agente risponde ai clienti con componenti interattivi, legati all'agente da un contratto tipizzato, accompagnata dalla sincronizzazione del carrello con il tema del negozio e da una *dashboard* che misura le conversazioni. Il prototipo soddisfa tutti gli obiettivi obbligatori, e gli obiettivi esclusi dipendono dalla scelta, condivisa con il *tutor*, di dedicare la seconda

metà dello *stage* alla sperimentazione. Per Halue ChatCart è una base documentata da cui riprendere la ricerca sul commercio agentico.

Per me lo *stage* ha colmato la lacuna che mi aveva portato a sceglierlo: oggi so costruire un'applicazione Shopify su misura, progettare un agente e decidere quali regole affidare alle istruzioni e quali al codice. Ho portato nel progetto il rigore appreso in Ingegneria del Software, dosandolo per un lavoro esplorativo, e ho affrontato per la prima volta gli agenti di IA, l'ambito in cui partivo più scoperto. Queste competenze entrano direttamente nel mio lavoro in proprio, dove intendo inserire agenti come quello di ChatCart in diverse parti dei negozi che seguo e gestirli con strumenti agentici attraverso le Admin API. Considero l'esperienza di tirocinio in Halue molto positiva: gli studi mi hanno insegnato il rigore, il lavoro da *freelance* la capacità di passare in fretta dall'idea al prodotto, e questo *stage* il modo per farli convivere nello stesso progetto.

Acronimi e abbreviazioni

Elenco in ordine alfabetico gli acronimi e le abbreviazioni introdotti nei capitoli 1, 2 e 3. Quando presente, il collegamento rimanda all'approfondimento nel glossario.

ADR: *Architecture Decision Record*.

API: *Application Programming Interface*.

AWS: Amazon Web Services.

B2B: *Business-to-Business*.

B2C: *Business-to-Consumer*.

CLI: *Command Line Interface*.

CMS: *Content Management System*.

CRM: *Customer Relationship Management*.

CSS: *Cascading Style Sheets*.

ERP: *Enterprise Resource Planning*.

FAQ: *Frequently Asked Questions*.

HTML: *HyperText Markup Language*.

HTTP: *HyperText Transfer Protocol*.

IA: intelligenza artificiale.

iPaaS: *Integration Platform as a Service*.

JSON: *JavaScript Object Notation*.

JWT: *JSON Web Token*.

LLM: *Large Language Model*.

MCP: *Model Context Protocol*.

ORM: *Object-Relational Mapper*.

SAL: stato di avanzamento dei lavori.

SemVer: *Semantic Versioning*.

SSE: *Server-Sent Events*.

UCP: *Universal Commerce Protocol*.

UI: *User Interface*.

UML: *Unified Modeling Language*.

UX: *User Experience*.

Glossario

Raccolgo in ordine alfabetico i termini tecnici introdotti nel testo. Alla prima occorrenza nel corpo della relazione, il termine e il pedice G rimandano alla voce corrispondente.

Action In React Router, la funzione che una rotta dichiara per ricevere ed elaborare sul *server* le modifiche inviate da moduli e richieste, prima che la pagina venga aggiornata. Affianca il *loader*, che si occupa invece delle letture.

adapter *Design pattern* che rende compatibile un componente con l'interfaccia che il codice cliente si aspetta, senza modificare né l'uno né l'altro. In ChatCart ogni famiglia di temi ha un adattatore con un'interfaccia comune, e un adattatore di ripiego copre i temi non riconosciuti.

ADR Documento che registra una decisione di architettura: il contesto in cui nasce, le alternative considerate, la scelta e le sue conseguenze. Conserva le motivazioni della scelta anche quando cambia chi lavora al progetto.

API Interfaccia con cui un programma rende disponibili dati o operazioni ad altri programmi, secondo un contratto definito. In Shopify, la Admin API permette di amministrare il negozio; la Storefront API espone le funzioni rivolte all'esperienza d'acquisto.

App proxy Funzione di Shopify che espone un percorso dell'applicazione sotto il dominio del negozio e inoltra le richieste al *server* dell'applicazione aggiungendo una firma che ne attesta la provenienza. Permette a un *widget* di chiamare il proprio *server* senza uscire dal dominio del negozio.

B Impact Assessment Questionario gratuito con cui un'azienda misura il proprio impatto sociale e ambientale in cinque aree: *governance*, lavoratori, clien-

ti, comunità e ambiente. Il punteggio che ne risulta è anche la base per la certificazione B Corp.

B2B Modello commerciale in cui un'azienda vende prodotti o servizi ad altre aziende. Nel commercio elettronico può richiedere cataloghi, listini e condizioni di acquisto diversi per ciascun cliente.

B2C Modello commerciale in cui un'azienda vende prodotti o servizi al consumatore finale. È il contesto dell'interfaccia conversazionale di ChatCart, rivolta a chi acquista nel negozio.

Brain dump Tecnica di organizzazione personale che consiste nello scrivere o registrare, senza ordine né filtro, tutto ciò che si ha in mente (attività, idee, dubbi), per poi ricavarne le azioni da svolgere. È il primo passo del metodo Getting Things Done di David Allen, che lo chiama raccolta.

Caso d'uso Descrizione di un'interazione completa tra un sistema e un attore, cioè un ruolo esterno che lo usa per raggiungere uno scopo: comprende lo scenario principale, in cui tutto va come previsto, e gli scenari alternativi, con le eccezioni e gli errori. I casi d'uso si rappresentano spesso con un diagramma UML, che mostra gli attori, il confine del sistema e i casi d'uso collegati a ciascun attore; da essi si ricavano i requisiti. Tra due casi d'uso la relazione «*include*» indica un comportamento sempre compreso nell'altro, la relazione «*extend*» un comportamento facoltativo che vi si innesta in un punto preciso.

Chatbot Programma che conversa con l'utente in linguaggio naturale, per iscritto o a voce. Nei negozi *online* risponde alle domande dei visitatori; quelli più recenti si basano su un modello linguistico e possono consultare i dati del negozio.

Checkout Procedura con cui il cliente conclude l'acquisto: indirizzo, spedizione e pagamento. In Shopify è ospitata dalla piattaforma, e il negozio vi accede con il contenuto del carrello.

Commercio agentico Insieme delle esperienze d'acquisto in cui un agente di intelligenza artificiale accompagna il cliente, o agisce per suo conto, nella ricerca, nella scelta e nell'acquisto di un prodotto. L'agente può appartenere al negozio oppure a una terza parte che raggiunge il negozio tramite protocolli come MCP e UCP.

Commit Registrazione di un insieme di modifiche nella storia di un archivio Git, con autore, data e messaggio descrittivo. La storia dei *commit* permette di ricostruire lo stato del codice in ogni momento.

Conventional Commits Convenzione per i messaggi di *commit* di Git: ogni messaggio inizia con un tipo (`feat`, `fix`, `refactor`, `docs`, `test`, `chore` e altri), eventualmente seguito da un ambito tra parentesi, e da una breve descrizione. Rende la storia leggibile da persone e programmi, per esempio per generare le note di rilascio o calcolare il numero di versione secondo SemVer.

CRM Insieme di pratiche e sistemi per gestire le relazioni di un'azienda con i propri clienti. Un sistema CRM raccoglie, per esempio, contatti, interazioni commerciali e richieste di assistenza.

Cucitura (*seam*) Punto in cui il codice di un'applicazione incontra un sistema che non controlla, come una piattaforma esterna, una base di dati o un modello. Nei *test* è il punto in cui si può sostituire il sistema esterno con un equivalente controllato, senza modificare il codice che lo usa.

Dashboard Pagina che riassume in un'unica vista dati e indicatori utili a chi la consulta. La *dashboard* di ChatCart è incorporata nell'amministrazione del negozio e mostra al commerciante il *funnel*, le conversazioni e la loro analisi.

Data tool Strumento di lettura dell'agente di ChatCart: interroga le Storefront API (prodotti, collezioni, pagine del negozio) e restituisce il risultato soltanto al modello, senza che il cliente lo veda. Si contrappone al *display tool*.

decorator *Design pattern* che aggiunge un comportamento a un oggetto o a una funzione avvolgendolo in un altro con la stessa interfaccia, senza modificarne il codice e senza che chi lo usa se ne accorga.

dependency injection *Design pattern* in cui un modulo riceve dall'esterno gli oggetti da cui dipende, invece di crearli da sé. Rende sostituibili le dipendenze, per esempio con versioni finte nei *test*, senza modificare il modulo.

Design pattern Soluzione riconosciuta e documentata a un problema di progettazione che si ripresenta in contesti diversi, descritta in modo indipendente dal linguaggio. Ne è un esempio l'*adapter*, che rende compatibili due interfacce senza modificarle.

Design system Insieme coerente di principi, regole visive e componenti riusabili per progettare un'interfaccia. Comprende elementi come colori, tipografia,

spaziature e comportamenti dei componenti, così da mantenere riconoscibile l'identità del prodotto o del marchio.

Display tool Strumento di visualizzazione dell'agente di ChatCart: il suo risultato sono i dati di un componente che il *widget* disegna nella chat, come le schede prodotto. La sua invocazione chiude la risposta dell'agente.

E-commerce Commercio di prodotti o servizi attraverso canali digitali. Un negozio elettronico comprende il catalogo, la selezione dei prodotti, il carrello e il pagamento, oltre ai collegamenti con i sistemi che gestiscono ordini e logistica.

ERP Sistema gestionale che integra dati e processi di diverse aree aziendali, come amministrazione, acquisti, magazzino e vendite. Il collegamento con il negozio permette, per esempio, di sincronizzare ordini e disponibilità dei prodotti.

factory *Design pattern* in cui una funzione, o un oggetto, si occupa di creare gli oggetti di cui il resto del codice ha bisogno, nascondendo come vengono costruiti e con quali dipendenze. In ChatCart crea a ogni richiesta gli strumenti dell'agente, legati al negozio e al mercato del cliente.

Founder & Senior Consultant Ruolo che unisce la fondazione dell'azienda all'attività di consulenza esperta. In Halue lo ricopre Valentino Baraldo.

Funnel Sequenza dei passi che portano un visitatore fino a un obiettivo, per esempio dall'arrivo nel negozio all'acquisto, con il numero di persone che raggiunge ciascun passo. Il rapporto tra un passo e il precedente indica dove si perdono più visitatori.

GraphQL Linguaggio di interrogazione per API e insieme di regole per eseguire le richieste su uno schema di dati tipizzato. Il programma che invia la richiesta specifica i campi di cui ha bisogno; Shopify lo adotta per la Admin API e la Storefront API.

Headless Architettura che separa la gestione dei dati e delle funzioni applicative dall'interfaccia che li presenta. In un negozio di questo tipo l'interfaccia è un'applicazione autonoma che comunica con la piattaforma commerciale tramite API; in un CMS la stessa separazione riguarda i contenuti e la loro presentazione.

Hydrogen *Framework* di Shopify basato su React per costruire interfacce di negozi *headless*. Offre strumenti e componenti per accedere ai dati del negozio tramite

la Storefront API; Oxygen è il servizio di Shopify che può ospitare queste applicazioni.

Interfaccia generativa Interfaccia in cui un modello linguistico decide, a ogni risposta, quali elementi mostrare all'utente oltre al testo. In ChatCart il modello sceglie tra componenti predefiniti e ne fornisce i dati; l'aspetto dei componenti resta fissato dal codice.

iPaaS Piattaforma erogata come servizio per collegare applicazioni e sistemi diversi. Offre connettori e strumenti per trasformare e trasferire i dati, riducendo la necessità di costruire un'integrazione distinta per ogni coppia di sistemi.

Junior Developer Sviluppatore all'inizio del proprio percorso professionale, che realizza e verifica funzionalità con il supporto di colleghi più esperti.

Landing page Pagina di atterraggio, costruita per accogliere chi arriva da una campagna, da un annuncio o da un collegamento esterno e guidarlo verso un'azione, come l'acquisto di un prodotto. Una *landing page* dinamica cambia contenuti in base a dati come la collezione, il mercato o il visitatore.

Linter Strumento di analisi statica che esamina il codice sorgente senza eseguirlo e segnala errori probabili, costrutti sospetti e violazioni delle convenzioni stabilite dal progetto.

Liquid Linguaggio di *template* usato nei temi Shopify. Permette di combinare la struttura delle pagine con i dati del negozio, attraverso variabili, condizioni e cicli; la piattaforma elabora i modelli e genera le pagine da inviare al navigatore.

Livello di ragionamento Parametro di alcuni modelli linguistici che regola quanto testo intermedio, non mostrato all'utente, il modello produce prima della risposta. Un livello più alto migliora di solito l'accuratezza nei compiti a più passi, al prezzo di una latenza e di un costo maggiori.

LLM Modello linguistico di grandi dimensioni, addestrato su ampie raccolte di dati per elaborare e generare testo. In un agente può selezionare strumenti esterni di cui l'applicazione esegue le operazioni; il collegamento a questi strumenti gli permette di usare dati aggiornati che non fanno parte del suo addestramento.

Loader In React Router, la funzione che una rotta dichiara per leggere sul *server* i dati di cui la pagina ha bisogno, prima di mostrarla; il componente della rotta li riceve già pronti.

MCP Protocollo aperto che standardizza la comunicazione tra applicazioni basate su modelli linguistici e servizi esterni. Un *server* MCP può esporre strumenti, risorse e istruzioni; è l'applicazione che ospita il modello a gestire il collegamento e l'esecuzione delle richieste.

Mock Oggetto che nei *test* prende il posto di una dipendenza reale, ne imita l'interfaccia e restituisce valori stabiliti in anticipo, registrando le chiamate ricevute. Si distingue dal *fake*, un'implementazione semplificata ma funzionante della dipendenza.

OAuth Standard aperto per l'autorizzazione delegata: permette a un servizio di concedere a un'applicazione un accesso limitato e revocabile ai dati di un utente, con *token* di accesso al posto della password. Definisce più flussi (*grant*), tra cui l'*authorization code*, le *client credentials* e il *token exchange*, adatti a casi d'uso diversi.

observer *Design pattern* in cui un oggetto, detto soggetto, mantiene un elenco di osservatori e li avvisa a ogni cambiamento del proprio stato, così che restino aggiornati senza interrogarlo di continuo.

ORM *Object-Relational Mapper*: libreria che collega le tabelle di una base di dati relazionale agli oggetti di un linguaggio di programmazione. Prisma, usato da ChatCart, ricava dallo schema i tipi TypeScript e le funzioni di accesso, così gli errori nelle interrogazioni emergono in compilazione.

***Output* strutturato** Modalità in cui un modello linguistico restituisce dati conformi a uno schema dichiarato dal programma, per esempio un oggetto con campi tipizzati, invece di testo libero; il programma valida la risposta contro lo schema prima di usarla.

Project Management Assistant Figura che supporta la gestione dei progetti, contribuendo all'organizzazione delle attività, all'aggiornamento della documentazione e al monitoraggio delle scadenze.

Project/service manager Responsabilità operativa di gestione di un progetto o di un servizio continuativo. Nel metodo di Halue cura la pianificazione delle attività e la capacità disponibile del gruppo di lavoro.

Prompt Testo fornito a un modello linguistico per guidarne la risposta. Il *prompt* di sistema precede la conversazione e ne fissa ruolo, regole e formato delle risposte; il modello lo rilegge a ogni turno.

Pull request Proposta di integrare un insieme di modifiche da un *branch* a un altro in un sistema di versionamento. Raccoglie le differenze nel codice, la discussione di revisione e gli esiti dei controlli automatici prima dell'integrazione.

Redesign Riprogettazione grafica e funzionale di un sito o di un negozio esistente, che ne rinnova l'interfaccia e i percorsi dell'utente, spesso senza cambiare la piattaforma su cui si basa.

registry *Design pattern* che riunisce in un'unica struttura, indicizzata per chiave, le voci di un insieme e le informazioni associate a ciascuna, così che il resto del codice le consulti da un solo punto. In TypeScript il tipo del registro può imporre che ogni voce dell'insieme sia definita.

repository *Design pattern* che concentra in un unico modulo le letture e le scritture di un insieme di dati, nascondendo al resto dell'applicazione il modo in cui sono memorizzati.

Ricerca semantica Ricerca che confronta il significato della richiesta con quello dei documenti, invece delle sole parole in comune. Di solito si basa su rappresentazioni numeriche dei testi, dette *embedding*, in cui testi di significato simile risultano vicini.

SAL Stato di avanzamento dei lavori. Nel contesto descritto indica un rapporto periodico che riassume il progresso del progetto, le scadenze, i rischi e i punti aperti, per condividere la situazione con le persone coinvolte.

Scrum *Framework* per affrontare problemi complessi attraverso cicli brevi di lavoro, chiamati *sprint*, e un confronto regolare sui risultati. Prevede responsabilità, eventi e artefatti definiti; nella sezione 1.4.1 distinguo questo riferimento dall'adattamento che Halue ne fa per il lavoro su commessa.

SemVer Convenzione di versionamento semantico che esprime una versione nella forma `MAJOR.MINOR.PATCH`. Rispetto all'API pubblica dichiarata, il primo numero cambia per modifiche incompatibili, il secondo per nuove funzionalità compatibili e il terzo per correzioni compatibili.

Senior Project Manager Figura con esperienza nella gestione dei progetti, responsabile del coordinamento di pianificazione, risorse e avanzamento delle attività.

Senior Software Engineer Professionista dello sviluppo del *software* con esperienza consolidata, che affronta problemi progettuali complessi e contribuisce alle scelte tecniche e alla revisione del codice.

Shopify Markets Funzione di Shopify per vendere in più paesi dallo stesso negozio. Per ogni mercato si definiscono lingua, valuta, prezzi e indirizzi delle pagine.

Shopify Plus Proposta di Shopify per le aziende di grandi dimensioni, con funzioni aggiuntive rispetto ai piani standard, tra cui quelle per la vendita B2B.

Skill Insieme riusabile di istruzioni e, quando necessari, risorse o strumenti, che guida un assistente basato su IA in un'attività specifica. Serve a rendere esplicito un procedimento, ma non garantisce da solo la correttezza dell'esecuzione.

Società Benefit Società che affianca allo scopo di lucro una o più finalità di beneficio comune indicate nello statuto. La disciplina italiana richiede una gestione che bilanci questi obiettivi e una relazione annuale sull'impatto prodotto; la qualifica non coincide con una certificazione privata di sostenibilità.

Software Engineer Professionista che applica metodi di ingegneria del *software* alla progettazione, realizzazione, verifica e manutenzione di sistemi *software*.

Solution & Technical Architect Figura che traduce le esigenze applicative in una soluzione tecnica, definendo l'architettura del sistema, le tecnologie e le integrazioni tra componenti.

Sprint Ciclo di lavoro di durata fissa, non superiore a un mese nel riferimento Scrum, durante il quale il gruppo persegue un obiettivo concordato. La revisione dei risultati e la retrospettiva permettono di adattare il prodotto e il modo di lavorare al ciclo successivo.

SSE *Server-Sent Events*: standard del *web* con cui un *server* mantiene aperta la risposta a una richiesta HTTP e vi invia una sequenza di eventi testuali. È unidirezionale, dal *server* al *client*, e attraversa i comuni *proxy*, purché non accumulino la risposta prima di inoltrarla.

Staging Ambiente separato dalla produzione in cui collaudare una versione dell'applicazione prima di renderla disponibile agli utenti. Riproduce le condizioni di esercizio per quanto necessario alla verifica, senza coincidere con il servizio effettivamente usato dai clienti.

Stand-up Breve riunione quotidiana, nata nei metodi agili, in cui ogni membro del gruppo riferisce su cosa ha lavorato, su cosa lavorerà e quali ostacoli incontra. Il nome viene dall'abitudine di tenerla in piedi, per mantenerla breve.

Startup Impresa giovane, di solito piccola, nata per sviluppare un prodotto o un servizio innovativo e per verificarne in fretta l'interesse sul mercato, con cicli di sviluppo brevi e ruoli poco formalizzati.

Storefront Interfaccia di un negozio digitale rivolta al cliente, attraverso cui consultare il catalogo, selezionare prodotti e avviare l'acquisto. In Shopify può basarsi sui temi della piattaforma oppure su un'applicazione separata con architettura *headless*.

Story point Unità di stima relativa che un gruppo usa per confrontare le attività in termini di complessità, impegno e incertezza. Non corrisponde a un numero fisso di ore e acquista significato rispetto alle stime dello stesso gruppo; la scala di Fibonacci è una possibile convenzione, non un obbligo di Scrum.

strategy *Design pattern* che incapsula in oggetti intercambiabili le varianti di un algoritmo, così che il codice che lo usa possa sceglierne una durante l'esecuzione senza conoscerne i dettagli.

Streaming Trasmissione di una risposta in frammenti successivi, inviati man mano che vengono prodotti, così che chi la riceve possa mostrarla prima che sia completa. In un assistente basato su un modello linguistico permette di mostrare il testo mentre il modello lo genera.

Tassonomia di Bloom Classificazione degli obiettivi di apprendimento proposta da Benjamin Bloom nel 1956 e rivista nel 2001 da Anderson e Krathwohl, che ordina i processi cognitivi in sei livelli di complessità crescente: ricordare, comprendere, applicare, analizzare, valutare e creare. Ogni livello presuppone i precedenti.

Team lead Responsabilità di guida tecnica di un gruppo di lavoro. In Halue comprende l'analisi funzionale, le stime per il cliente e il coordinamento tecnico degli sviluppatori.

Theme app extension Estensione con cui un'applicazione Shopify aggiunge elementi a un tema senza modificarne il codice. I file dell'estensione sono serviti dalla rete di Shopify e un solo rilascio raggiunge tutti i negozi che usano l'applicazione. Comprende blocchi che il commerciante posiziona in una sezione

e blocchi *app embed*, inseriti una volta in ogni pagina e attivati dall'editor del tema.

Ticket Unità di lavoro registrata in un sistema di tracciamento, con un identificativo, una descrizione e uno stato. Può rappresentare una funzionalità, una modifica o un difetto, e raccogliere stime, discussioni e collegamenti alle modifiche del codice.

Token Unità minima di testo elaborata da un modello linguistico: una parola breve, una parte di parola, un segno di punteggiatura. Il modello legge e genera testo un *token* alla volta; la lunghezza del contesto e il costo di utilizzo si misurano in *token*.

***Token* di accesso** Credenziale che un'applicazione presenta a ogni richiesta a un'API per dimostrare di essere autorizzata, con i permessi (*scope*) che le sono stati concessi. Può essere permanente o avere una scadenza, nel qual caso si rinnova con un secondo *token* (*refresh token*). Da non confondere con il *token* dei modelli linguistici.

Tool calling Meccanismo con cui un modello linguistico chiede al programma che lo ospita di eseguire una funzione, detta strumento, indicandone il nome e gli argomenti. Il programma esegue la funzione e restituisce il risultato al modello, che lo usa per proseguire.

Tunnel Servizio che espone su un indirizzo pubblico un programma in esecuzione su una macchina non raggiungibile da Internet, inoltrandogli le richieste. Nello sviluppo di applicazioni Shopify serve perché Shopify possa chiamare l'applicazione eseguita sul computer dello sviluppatore.

UCP Protocollo aperto per rendere interoperabili i servizi commerciali e gli agenti che operano per conto dell'utente. Definisce modalità condivise per scoprire le capacità di un negozio e interagire con le fasi dell'acquisto; riguarda il dominio commerciale, mentre MCP offre un meccanismo generale per collegare applicazioni e strumenti.

UX/UI Digital Consultant Consulente che progetta l'esperienza utente e l'interfaccia di un prodotto digitale, considerando percorsi d'uso, struttura delle informazioni e aspetto visivo.

Way of working L'insieme delle regole e delle procedure con cui un gruppo di lavoro pianifica, svolge e verifica le proprie attività, dalla gestione delle modifi-

che alle revisioni. Lo si definisce per iscritto, così che ogni membro del gruppo lavori nello stesso modo.

Widget Elemento dell'interfaccia autonomo, inserito in una pagina che non lo ha prodotto. Il *widget* di ChatCart è il riquadro di chat sovrapposto alle pagine del negozio.

Riferimenti

1. Halue S.r.l., sito istituzionale, https://halue.com/it, pagine «Chi siamo», «Approccio» e «Progetti», consultate il 4 novembre 2026.
2. Halue S.r.l., «Società Benefit», https://halue.com/it/societa-benefit, consultata il 10 novembre 2026.
3. Shopify, «Shopify Plus», https://www.shopify.com/plus, consultata il 10 novembre 2026.
4. Shopify, «Halue Srl», elenco dei partner di Shopify, https://www.shopify.com/it/partners/directory/partner/halue-srl, consultata il 10 novembre 2026.
5. L. Fang, Z. Yuan, K. Zhang, D. Donati, M. Sarvary, «Generative AI and Sales Productivity: Field Experiments in Online Retail», arXiv:2510.12049, 2025, https://arxiv.org/abs/2510.12049, consultato il 10 novembre 2026.
6. Shopify, «Sidekick», Centro assistenza, https://help.shopify.com/en/manual/ai-powered-tools/sidekick, consultato il 10 novembre 2026.
7. Shopify, «Storefront MCP», documentazione per sviluppatori, https://shopify.dev/docs/apps/build/storefront-mcp, consultato il 10 novembre 2026.
8. Shopify e Google, *Universal Commerce Protocol*, https://ucp.dev e https://www.shopify.com/it/ucp, consultati il 10 novembre 2026.
9. Dr Kampes, sito del negozio, https://drkampes.com, consultato l'11 novembre 2026.
10. Anthropic, «Claude and your productivity platforms», 16 ottobre 2025, https://www.anthropic.com/news/productivity-platforms, consultato l'11 novembre 2026.
11. OpenAI, «Introducing company knowledge», ottobre 2025, https://openai.com/index/introducing-company-knowledge/, consultato l'11 novembre 2026.
12. Vercel, «AI SDK», documentazione, https://ai-sdk.dev/docs, consultata il 12 novembre 2026.

13. Shopify, «Standard storefront events and actions», documentazione per sviluppatori, https://shopify.dev/docs/apps/build/online-store/standard-events-and-actions, e annuncio del 17 giugno 2026, https://shopify.dev/changelog/standard-storefront-events-and-actions, consultati il 12 novembre 2026.
14. Shopify, «Cart API reference», documentazione per sviluppatori, https://shopify.dev/docs/api/ajax/reference/cart, consultata il 12 novembre 2026.
15. TypeSafe AI, «System One», documentazione, https://docs.typesafe.ai/concepts/system-one, consultata il 12 novembre 2026.
16. Vercel, «How to classify, route, and score with Jev and AI SDK», https://vercel.com/kb/guide/typesafe-jev-and-ai-sdk, consultato il 12 novembre 2026.
17. Shopify, «About app distribution», documentazione per sviluppatori, https://shopify.dev/docs/apps/launch/distribution, consultata il 12 novembre 2026.
18. D. Allen, *Getting Things Done: The Art of Stress-Free Productivity*, Viking, 2001; edizione riveduta Penguin Books, 2015.
19. Shopify, «Libraries and templates», documentazione per sviluppatori, https://shopify.dev/docs/api/libraries-and-templates, consultata il 12 novembre 2026.
20. Shopify, «Token exchange», documentazione per sviluppatori, https://shopify.dev/docs/apps/build/authentication-authorization/access-tokens/token-exchange, consultata il 12 novembre 2026.
21. Shopify, «App proxies», documentazione per sviluppatori, https://shopify.dev/docs/apps/build/online-store/app-proxies, consultata il 12 novembre 2026.
22. Shopify, «Theme app extensions» e «Configure theme app extensions», documentazione per sviluppatori, https://shopify.dev/docs/apps/build/online-store/theme-app-extensions, consultate il 12 novembre 2026.
23. Shopify, «Storefront API», documentazione per sviluppatori, https://shopify.dev/docs/api/storefront, consultata il 12 novembre 2026.
24. E. Ries, *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses*, Crown Business, New York, 2011.
25. L. W. Anderson, D. R. Krathwohl (a cura di), *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*, Longman, New York, 2001.